

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN I
THỰC TẬP CƠ SỞ



BÁO CÁO

ĐỀ TÀI : “”

GVHD :

Nhóm lớp:

Hà Nội – 2026

MỤC LỤC

(Mục lục tự động: Trong Microsoft Word, quý độc giả có thể cập nhật bảng mục lục bất kỳ lúc nào bằng cách nhấp chuột phải và chọn 'Update Table' → 'Update entire table').

CHƯƠNG I. TỔNG QUAN HỆ THỐNG VÀ BÀI TOÁN THỰC TẾ.....	6
LIÊN KẾT DỰ ÁN TRỰC TUYẾN.....	6
1.1. Lời mở đầu.....	6
1.2. Mục tiêu của bài tập.....	6
1.3. Quy trình phát triển hệ thống thông minh.....	7
1.4. Tổng quan về ba ứng dụng.....	8
1.4.1. Ứng dụng dự đoán bệnh tiểu đường (Diabetes Prediction).....	8
1.4.2. Ứng dụng dự đoán giá nhà (House Price Prediction).....	8
1.4.3. Ứng dụng phân tích hành vi và khám phá sở thích khách hàng thương mại điện tử.....	9
CHƯƠNG II. CƠ SỞ LÝ THUYẾT VỀ BIỂU DIỄN DỮ LIỆU.....	9
2.1. Tổng quan về học máy và hệ thống thông minh.....	9
2.2. Bài toán phân loại và hồi quy.....	9
2.2.1. Bài toán Phân loại (Classification).....	9
2.2.2. Bài toán Hồi quy (Regression).....	10
2.3. Khái niệm biểu diễn dữ liệu.....	10
2.4. Vectơ đặc trưng (Feature Vector).....	10
2.5. Ma trận đặc trưng (Feature Matrix).....	11
2.6. Tensor và biểu diễn dữ liệu nhiều chiều.....	12
2.7. Biểu diễn dữ liệu văn bản: TF-IDF vs Dense Embedding.....	12
2.7.1. Biểu diễn túi từ thống kê (TF-IDF Vector — Sử dụng trong Assignment 02).....	12
2.7.2. Biểu diễn nhúng sâu (Dense Embedding Vectors — Bài giảng Lecture 02).....	13
2.8. Mã hóa dữ liệu phân loại.....	14
2.9. Kiểu dữ liệu và miền giá trị.....	14
2.10. Tính nhất quán của biểu diễn dữ liệu.....	14
2.11. Rò rỉ dữ liệu (Data Leakage).....	14
CHƯƠNG III. PHƯƠNG PHÁP TIỀN XỬ LÝ DỮ LIỆU.....	15
3.1. Tìm hiểu dữ liệu (Data Understanding).....	15
3.2. Làm sạch dữ liệu.....	15
3.2.1. Xử lý giá trị thiếu (Missing Values).....	15
3.2.2. Xử lý dữ liệu trùng lặp (Duplicate Records).....	16
3.2.3. Xử lý giá trị không hợp lệ (Invalid Values).....	16
3.2.4. Phân tích ngoại lệ (Outlier Analysis).....	17
3.3. Mã hóa biến phân loại.....	17
3.4. Chuẩn hóa dữ liệu số (StandardScaler).....	17
3.5. Chia tập dữ liệu: Train, Validation và Test.....	18
3.6. Ngăn ngừa rò rỉ dữ liệu (Data Leakage Prevention).....	18
3.7. Pipeline tiền xử lý với ColumnTransformer.....	18
3.8. Nguyên tắc tái lập quá trình tiền xử lý (Reproducibility).....	19
CHƯƠNG IV. CÁC ĐỘ ĐO ĐÁNH GIÁ MÔ HÌNH.....	19
4.1. Đối với bài toán Hồi quy (Regression).....	19

4.1.1. Sai số tuyệt đối trung bình (Mean Absolute Error - MAE).....	19
4.1.2. Sai số bình phương trung bình (MSE) và Căn bậc hai (RMSE).....	19
4.1.3. Hệ số xác định (Coefficient of Determination - R^2 Score).....	20
4.2. Đối với bài toán Phân lớp (Classification).....	20
4.2.1. Ma trận nhầm lẫn (Confusion Matrix).....	20
4.2.2. Các chỉ số phân lớp: Accuracy, Precision, Recall và F1-Score.....	20
4.2.3. Đường cong ROC và Chỉ số ROC-AUC.....	21
CHƯƠNG V. PHƯƠNG PHÁP XÂY DỰNG MÔ HÌNH.....	21
5.1. Tổng quan về xây dựng mô hình.....	21
5.2. Mô hình Baseline.....	21
5.3. Các mô hình cho bài toán hồi quy.....	21
5.3.1. Hồi quy tuyến tính cổ điển (Linear Regression OLS).....	21
5.3.2. Hồi quy sừng núi (Gradient Boosting Regressor — L2 Regularization).....	22
5.3.3. Cây quyết định hồi quy (Decision Tree Regressor).....	22
5.3.4. Rừng ngẫu nhiên hồi quy (Random Forest Regressor).....	22
5.3.5. Tăng cường độ dốc (Gradient Boosting Regressor).....	22
5.4. Các mô hình cho bài toán phân loại.....	22
5.4.1. Hồi quy Logistic (Logistic Regression).....	22
5.4.2. K láng giềng gần nhất (K-Nearest Neighbors - KNN).....	22
5.4.3. Cây quyết định phân lớp (Decision Tree Classifier).....	23
5.4.4. Rừng ngẫu nhiên phân lớp (Random Forest Classifier).....	23
5.4.5. Máy vector hỗ trợ tuyến tính (LinearSVC).....	23
5.5. Đóng gói quy trình học máy (Scikit-Learn Pipeline).....	23
CHƯƠNG VI. PHƯƠNG PHÁP LƯU TRỮ VÀ TRIỂN KHAI MÔ HÌNH.....	24
6.1. Tổng quan.....	24
6.2. Lưu trữ mô hình (Model Persistence).....	24
6.3. Tính nhất quán giữa huấn luyện và dự đoán.....	24
6.4. Triển khai mô hình dưới dạng Web Service với FastAPI.....	25
6.5. Giao diện người dùng Web Application.....	27
6.6. Triển khai trên ứng dụng Mobile.....	28
6.7. Kiến trúc hoàn chỉnh của hệ thống.....	30
CHƯƠNG VII. ỨNG DỤNG 1 — DỰ ĐOÁN BỆNH TIỂU ĐƯỜNG.....	30
7.1. Mô tả bài toán.....	30
7.2. Giới thiệu tập dữ liệu.....	31
7.3. Khảo sát và tìm hiểu dữ liệu.....	31
7.4. Làm sạch dữ liệu.....	32
7.4.1. Xử lý giá trị thiếu.....	32
7.4.2. Xử lý bản ghi trùng lặp.....	32
7.4.3. Xử lý giá trị không hợp lệ.....	32
7.5. Biểu diễn dữ liệu.....	33
7.6. Phân tích khám phá dữ liệu (EDA).....	33
7.7. Xây dựng mô hình.....	38
7.8. Đánh giá mô hình.....	39
7.9. Phân tích lỗi trên tập Test (Error Analysis).....	42
7.10. Lựa chọn mô hình chính thức.....	43

7.11. Triển khai hệ thống.....	44
CHƯƠNG VIII. ỨNG DỤNG 2 — DỰ ĐOÁN GIÁ NHÀ.....	44
8.1. Mô tả bài toán.....	47
8.2. Giới thiệu tập dữ liệu.....	47
8.3. Khảo sát và tìm hiểu dữ liệu.....	48
8.4. Làm sạch dữ liệu.....	49
8.5. Biểu diễn dữ liệu.....	49
8.6. Phân tích khám phá dữ liệu (EDA).....	50
8.7. Xây dựng mô hình.....	53
8.8. Đánh giá mô hình.....	54
8.9. Lựa chọn mô hình.....	56
8.10. Triển khai hệ thống.....	57
CHƯƠNG IX. ỨNG DỤNG 3 — E-COMMERCE CUSTOMER BEHAVIOR & INTEREST DISCOVERY.....	57
9.1. Mô tả bài toán.....	60
9.2. Giới thiệu tập dữ liệu.....	60
9.3. Biểu diễn khách hàng (Customer Representation).....	60
9.4. Làm sạch dữ liệu.....	61
9.5. Ba chế độ biểu diễn dữ liệu.....	61
9.5.1. Chế độ chỉ dùng dữ liệu bảng (Tabular Only — $d = 32$).....	61
9.5.2. Chế độ chỉ dùng văn bản (Text TF-IDF Only — $d = 2,500$).....	61
9.5.3. Chế độ kết hợp Đa phương thức (Combined Tabular + Text — $d = 2,532$).....	61
9.6. Khám phá sở thích và Phân tích EDA.....	62
9.7. Xây dựng mô hình.....	65
9.8. Đánh giá mô hình.....	66
9.9. Phân tích thực nghiệm: Văn bản nhận xét có thực sự cải thiện chất lượng mô hình?.....	69
9.10. Ý nghĩa kinh doanh (Business Interpretation).....	69
9.11. Triển khai hệ thống.....	70
CHƯƠNG X. SO SÁNH BA HỆ THỐNG THÔNG MINH.....	73
10.1. So sánh bài toán và dữ liệu.....	73
10.2. So sánh biểu diễn dữ liệu.....	73
10.3. So sánh quy trình tiền xử lý.....	74
10.3.1. Các bước tiền xử lý chung.....	74
10.3.2. Các bước tiền xử lý riêng biệt.....	74
10.4. So sánh mô hình học máy và độ đo.....	74
10.5. So sánh kiến trúc triển khai.....	74
10.6. Trả lời các câu hỏi thảo luận.....	75
10.6.1. Trả lời 15 câu hỏi thảo luận cho Ứng dụng 1 (Dự đoán bệnh tiểu đường).....	75
10.6.2. Trả lời 15 câu hỏi thảo luận cho Ứng dụng 2 (Dự đoán giá nhà).....	76
10.6.3. Trả lời 15 câu hỏi thảo luận cho Ứng dụng 3 (E-Commerce Customer Behavior).....	77
10.6.4. Trả lời 6 câu hỏi thảo luận bổ sung chuyên sâu cho E-Commerce.....	77
CHƯƠNG XI. MỞ RỘNG — ĐỒ THỊ TRI THỨC VÀ CHATBOT VỚI NEO4J.....	77
11.1. Kiến trúc đề xuất và Luồng hoạt động.....	78
11.2. Kịch bản nạp dữ liệu Cypher và mô hình truy vấn.....	79

11.3. Tình trạng thực nghiệm và Kết quả.....	79
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	79
Hạn chế và Hướng phát triển trong tương lai.....	80

CHƯƠNG I. TỔNG QUAN HỆ THỐNG VÀ BÀI TOÁN THỰC TẾ

LIÊN KẾT DỰ ÁN TRỰC TUYẾN

- **Web App Đã Triển Khai (Render):** <https://assignment-02-pthttm.onrender.com/>
- **Kho Mã Nguồn (GitHub):** https://github.com/jellalaz/Assignment_02_PTHTTM

1.1. Lời mở đầu

Trong thời đại cách mạng công nghiệp 4.0 và sự bùng nổ của trí tuệ nhân tạo (AI), việc khai phá tri thức tiềm ẩn từ các kho dữ liệu khổng lồ đã trở thành năng lực cốt lõi đối với mọi tổ chức và doanh nghiệp. Từ các bệnh viện cần công cụ hỗ trợ sàng lọc sớm bệnh lý hiểm nghèo, các công ty bất động sản cần hệ thống định giá nhà đất minh bạch, cho đến các sàn thương mại điện tử cần thấu hiểu tâm tư, nguyện vọng của khách hàng để tối ưu hóa trải nghiệm mua sắm — tất cả đều dựa trên nền tảng của các mô hình học máy (Machine Learning).

Tuy nhiên, trong thực tế phát triển phần mềm, một quan niệm sai lầm phổ biến là đồng nhất 'xây dựng hệ thống thông minh' với việc 'huấn luyện một thuật toán học máy'. Trên thực tế, thuật toán học máy chỉ là một mắt xích nhỏ nằm ở trung tâm của một chuỗi cung ứng kỹ thuật phức tạp. Một mô hình toán học dù có độ chính xác cao đến đâu trong môi trường nghiên cứu thực nghiệm cũng sẽ trở nên vô giá trị nếu không thể tiếp nhận dữ liệu thực tế, không thể triển khai trên hạ tầng mạng và không mang lại trải nghiệm tương tác trực quan cho người dùng cuối.

Nguyên lý cơ bản được nhấn mạnh trong học phần 'Phát triển các Hệ thống Thông minh' là: Dữ liệu thực tế không bao giờ tồn tại sẵn dưới dạng số học hoàn hảo. Dữ liệu thô từ thế giới thực luôn chứa đựng sự nhiễu loạn, khuyết thiếu, dị biệt về thang đo và cấu trúc đa phương thức (dạng bảng kết hợp văn bản tự do). Do đó, cây cầu nối quyết định tính sống còn của toàn bộ hệ thống chính là quá trình Biểu diễn Dữ liệu (Data Representation) kết hợp cùng đường ống tiền xử lý (Preprocessing Pipeline) chuẩn mực khoa học.

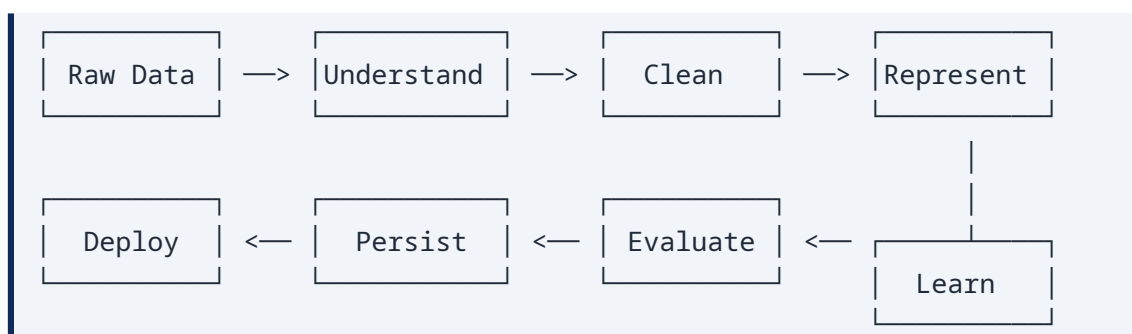
1.2. Mục tiêu của bài tập

Bài tập lớn Assignment 02 được thiết kế với mục tiêu trang bị cho sinh viên tư duy hệ thống và kỹ năng kỹ thuật toàn diện, không chỉ dừng lại ở các dòng lệnh trong Jupyter Notebook mà hướng tới việc tạo ra sản phẩm phần mềm thông minh hoàn chỉnh có khả năng vận hành trong đời sống thực tế:

- **1. Nền tảng biểu diễn:** Nắm vững bản chất toán học của vector đặc trưng và ma trận đặc trưng, hiểu rõ cơ chế biến đổi các miền dữ liệu khác nhau (dữ liệu số, biến danh mục rời rạc, văn bản ngôn ngữ tự nhiên) thành không gian vector Euclid số chiều xác định.
- **2. Thực nghiệm trung thực:** Khai thác 100% dữ liệu thực tế từ nền tảng Kaggle trên ba miền bài toán hoàn toàn khác biệt. Nghiêm cấm hoàn toàn việc sử dụng dữ liệu nhân tạo hay số liệu giả lập, đối mặt trực tiếp với các thách thức thực tế như mất cân bằng nhãn và rò rỉ dữ liệu.
- **3. Đánh giá đa chiều:** Xây dựng và so sánh khách quan hơn 16 thuật toán học máy có giám sát (từ Dummy Baseline, Linear/Logistic Regression, KNN, Decision Tree, Random Forest đến Gradient Boosting), lựa chọn độ đo đánh giá phù hợp với bản chất rủi ro của từng ngành nghề.
- **4. Đóng gói mô hình:** Đóng gói toàn bộ quy trình tiền xử lý và mô hình suy luận thành các tệp nhị phân độc lập (.joblib), bảo đảm tính nhất quán toán học tuyệt đối giữa giai đoạn huấn luyện và giai đoạn vận hành.
- **5. Triển khai ứng dụng:** Xây dựng máy chủ dịch vụ dự đoán REST API với framework FastAPI hiện đại (tự động sinh tài liệu chuẩn Swagger UI) và phát triển giao diện người dùng Responsive Web Application đa nền tảng, có khả năng phục vụ mượt mà trên cả máy tính Desktop lẫn điện thoại Smartphone qua mạng nội bộ Wi-Fi LAN.

1.3. Quy trình phát triển hệ thống thông minh

Tuân thủ nghiêm ngặt quy trình chuẩn kỹ nghệ dữ liệu khép kín, dự án được tổ chức thành một chuỗi 8 mắt xích liên tục:



- **Bước 1 — Raw Data:** Thu thập tập dữ liệu thô từ Kaggle thông qua các script tự động hoặc API chuyên dụng, lưu trữ nguyên vẹn tại thư mục data/raw/.
- **Bước 2 — Understand:** Thực thi phân tích thống kê mô tả (df.shape, df.info(), df.describe()), xác định số lượng dòng/cột, phân loại kiểu dữ liệu (số học, chuỗi danh mục, văn bản) và kiểm tra phân bố biến mục tiêu.

- **Bước 3 — Clean:** Phát hiện và xử lý triệt để các vấn đề chất lượng dữ liệu: loại bỏ bản ghi trùng lặp (duplicate records), điền khuyết thiếu (imputation), rà soát giá trị không hợp lệ (invalid values) và thẩm tra giá trị cực biên (outliers).
- **Bước 4 — Represent:** Biến đổi dữ liệu sạch thành không gian vector số học d chiều: chuẩn hóa Z-score các biến số, mã hóa One-Hot với drop='first' cho biến danh mục và trích xuất đặc trưng TF-IDF cho văn bản bình luận.
- **Bước 5 — Learn:** Huấn luyện mô hình cơ sở tầm thường (Baseline) và 5+ thuật toán học máy cạnh tranh trên tập Train độc lập.
- **Bước 6 — Evaluate:** Đánh giá hiệu năng đa chỉ số trên tập Validation độc lập để chọn ra mô hình tối ưu nhất, sau đó kiểm định khách quan một lần duy nhất trên tập Test độc lập.
- **Bước 7 — Persist:** Đóng gói toàn bộ đối tượng ColumnTransformer và Estimator thành một Pipeline duy nhất thông qua joblib.dump(), lưu trữ tại thư mục models/.
- **Bước 8 — Deploy:** Khởi chạy dịch vụ REST API với FastAPI, nạp sẵn Pipeline vào RAM và phục vụ người dùng thông qua giao diện Responsive Web Client truy cập qua mạng LAN.

1.4. Tổng quan về ba ứng dụng

Dự án triển khai thực tế trên ba bài toán thông minh độc lập, bao quát ba dạng cấu trúc dữ liệu và hai mô thức học máy chính:

1.4.1. Ứng dụng dự đoán bệnh tiểu đường (Diabetes Prediction)

Tập trung vào lĩnh vực y tế dự phòng, bài toán đặt ra là phân loại nhị phân (Binary Classification) xem một bệnh nhân có thuộc nhóm nguy cơ mắc bệnh tiểu đường hay không dựa trên 8 chỉ số xét nghiệm và nhân khẩu học lâm sàng. Biến mục tiêu y nhận giá trị nhị phân {0: Không mắc bệnh, 1: Mắc bệnh tiểu đường}. Thách thức cốt lõi là tỷ lệ mất cân bằng nhãn nghiêm trọng (91.5% âm tính vs 8.5% dương tính) và đòi hỏi tối ưu hóa chỉ số Recall để tránh bỏ sót bệnh nhân.

1.4.2. Ứng dụng dự đoán giá nhà (House Price Prediction)

Tập trung vào lĩnh vực kinh tế bất động sản, bài toán đặt ra là hồi quy giá trị liên tục (Continuous Regression) nhằm ước tính giá trị thị trường thực tế của một căn nhà dựa trên 15 đặc trưng vật lý và vị trí (diện tích sàn, số phòng ngủ, số phòng tắm, số tầng, tuổi thọ, thành phố và các tiện nghi đi kèm). Biến mục tiêu y là giá bán bằng USD. Thách thức lớn là việc kiểm soát hiện tượng đa cộng tuyến giữa các tiện nghi nhà và bảo đảm sai số tiền tệ MAE ở mức thấp nhất.

1.4.3. Ứng dụng phân tích hành vi và khám phá sở thích khách hàng thương mại điện tử

Tập trung vào lĩnh vực thương mại điện tử (E-Commerce), bài toán đặt ra là phân loại đa phương thức (Multimodal Classification) kết hợp dữ liệu bảng định lượng (số sao đánh giá, tuổi, ngành hàng) với ngôn ngữ tự nhiên tự do (tiêu đề và nội dung nhận xét chi tiết) để dự đoán hành vi khuyến nghị sản phẩm (Recommended IND $\in \{0, 1\}$). Thách thức cốt lõi là chứng minh bằng thực nghiệm khoa học: Liệu việc bổ sung thông tin văn bản có thực sự cải thiện chất lượng mô hình so với chỉ sử dụng dữ liệu bảng truyền thống?

CHƯƠNG II. CƠ SỞ LÝ THUYẾT VỀ BIỂU DIỄN DỮ LIỆU

2.1. Tổng quan về học máy và hệ thống thông minh

Học máy (Machine Learning) là một phân ngành của trí tuệ nhân tạo tập trung vào việc phát triển các thuật toán có khả năng tự động rút trích quy luật, mô hình hóa mối quan hệ phức tạp và cải thiện hiệu năng giải quyết tác vụ thông qua việc tiếp cận dữ liệu kinh nghiệm, mà không cần phải lập trình tường minh từng quy tắc rẽ nhánh cố định. Trong một hệ thống thông minh, mô hình học máy đóng vai trò là khối xử lý trung tâm (Inference Engine), biến đổi các thông tin quan sát đầu vào thành các dự báo hoặc quyết định hành động tối ưu.

Tuy nhiên, máy tính số học về bản chất chỉ có thể thực hiện các phép toán đại số tuyến tính: cộng vector, nhân ma trận, tính tích vô hướng và tối ưu hóa vi phân. Các thực thể trong thế giới thực — dù là một bệnh nhân với các triệu chứng lâm sàng, một ngôi nhà với tiện nghi nội thất, hay một bình luận cảm xúc của người tiêu dùng — đều không phải là những con số. Do đó, 'Biểu diễn Dữ liệu' (Data Representation) là bước tiên quyết định hình nên toàn bộ không gian hình học mà tại đó các thuật toán học máy sẽ tìm kiếm ranh giới phân tách hoặc hàm hồi quy xấp xỉ.

2.2. Bài toán phân loại và hồi quy

Trong học máy có giám sát (Supervised Learning), tập dữ liệu huấn luyện bao gồm các cặp mẫu (x_i, y_i) , trong đó x_i là vector đặc trưng mô tả đối tượng và y_i là nhãn mục tiêu cần dự đoán. Tùy thuộc vào không gian giá trị của y_i , bài toán được phân chia thành hai loại hình cơ bản:

2.2.1. Bài toán Phân loại (Classification)

Biến mục tiêu nhận các giá trị rời rạc trong một tập hữu hạn các nhãn lớp: $y \in \{c_1, c_2, \dots, c_K\}$. Khi $K = 2$, bài toán được gọi là phân loại nhị phân (Binary Classification). Mục tiêu là ước lượng xác suất hậu nghiệm $P(y = 1 | x)$ và vạch ra siêu phẳng phân định (Decision Boundary) trong không gian d chiều.

Trong mô hình tuyến tính phân lớp (Logistic Regression), đầu ra tuyến tính $z = w^T x + b$ được ánh xạ qua hàm kích hoạt phi tuyến Sigmoid (Logistic Function) để chuyển thành xác suất liên tục trong khoảng $(0, 1)$:

$$\sigma(z) = 1 / (1 + e^{(-z)}) \quad \text{với } z = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_d \cdot x_d + b$$

Quy tắc quyết định:

$$\begin{aligned} \hat{y} &= 1 \quad \text{nếu } P(y=1 | x) = \sigma(z) \geq \theta \quad (\text{ngưỡng phân loại chuẩn } \theta = 0.5) \\ \hat{y} &= 0 \quad \text{nếu } P(y=1 | x) = \sigma(z) < \theta \end{aligned}$$

Hàm mất mát được tối ưu hóa trong quá trình huấn luyện là hàm mất mát Entropy chéo nhị phân (Binary Cross-Entropy Loss):

$$L(w, b) = - (1/N) \times \sum [y_i \cdot \ln(\hat{y}_i) + (1 - y_i) \cdot \ln(1 - \hat{y}_i)]$$

Hàm mất mát này phạt rất nặng khi mô hình đưa ra dự đoán xác suất tự tin nhưng sai lệch nhân thực tế (ví dụ $y=1$ nhưng $\hat{y} \rightarrow 0$ thì $\ln(\hat{y}) \rightarrow -\infty$).

2.2.2. Bài toán Hồi quy (Regression)

Biến mục tiêu nhận giá trị liên tục trong tập số thực: $y \in \mathbb{R}$ (ví dụ giá bán bất động sản bằng USD, nhiệt độ môi trường, huyết áp). Mục tiêu là học hàm ánh xạ $f: \mathbb{R}^d \rightarrow \mathbb{R}$ sao cho khoảng cách giữa giá trị dự đoán $\hat{y}_i = f(x_i)$ và giá trị thực tế y_i là nhỏ nhất trên toàn bộ tập dữ liệu.

Trong hồi quy tuyến tính cổ điển (Linear Regression), hàm giả thuyết có dạng:

$$\hat{y} = w^T x + b = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_d \cdot x_d + b$$

Hàm mất mát tối ưu hóa phổ biến nhất là Tổng bình phương sai số (Ordinary Least Squares - OLS) hay Sai số bình phương trung bình (Mean Squared Error - MSE):

$$L_{\text{MSE}}(w, b) = (1/N) \times \sum (y_i - \hat{y}_i)^2 = (1/N) \times \sum (y_i - (w^T x_i + b))^2$$

2.3. Khái niệm biểu diễn dữ liệu

Biểu diễn dữ liệu là quá trình chuyển hóa thông tin thô, đa dạng từ thế giới thực (văn bản nhận xét, thông số bệnh án, vị trí địa lý) thành các cấu trúc đại số có thể tính toán được. Mỗi cách biểu diễn phản ánh một góc nhìn hình học khác nhau về dữ liệu. Một phép biểu diễn tốt phải bảo tồn tối đa các thông tin đặc trưng có ý nghĩa phân tách (Discriminative Features), đồng thời triệt tiêu nhiễu và bảo đảm tính nhất quán toán học.

2.4. Vectơ đặc trưng (Feature Vector)

Một đối tượng quan sát thứ i được số hóa thành một vector cột d chiều trong không gian số thực $\mathbb{R}^d$:

$$\mathbf{x}_i = [x_{i1}, x_{i2}, \dots, x_{id}]^T \in \mathbb{R}^d$$

Mỗi phần tử x_{ij} đại diện cho tọa độ của đối tượng trên trục đặc trưng thứ j . Số chiều d của vector phản ánh mức độ chi tiết của không gian mô tả.

Ví dụ minh họa: Ví dụ thực tế từ bài toán Tiểu đường: Một bệnh nhân nam, 55 tuổi, chỉ số BMI = 28.5 kg/m², mức HbA1c = 6.8%, Glucose = 155 mg/dL, không có tiền sử tăng huyết áp hay bệnh tim, chưa từng hút thuốc. Sau khi đi qua bộ biến đổi ColumnTransformer (chuẩn hóa Z-score các biến số và mã hóa One-Hot biến danh mục), vector đặc trưng số học 13 chiều có dạng cụ thể:

```
# Giả sử giá trị trung bình và độ lệch chuẩn của tập Train:
# age:  $\mu=41.89$ ,  $\sigma=22.52 \rightarrow z\_age = (55 - 41.89) / 22.52 \approx 0.582$ 
# bmi:  $\mu=27.32$ ,  $\sigma=6.64 \rightarrow z\_bmi = (28.5 - 27.32) / 6.64 \approx 0.178$ 
# HbA1c:  $\mu=5.53$ ,  $\sigma=1.07 \rightarrow z\_hba1c = (6.8 - 5.53) / 1.07 \approx 1.187$ 
# glucose:  $\mu=138.06$ ,  $\sigma=40.71 \rightarrow z\_glucose = (155 - 138.06) / 40.71 \approx 0.416$ 
# hypertension = 0, heart_disease = 0
# gender_Male = 1, gender_Other = 0 (bỏ Female)
# smoke_current=0, smoke_ever=0, smoke_former=0, smoke_never=1,
# smoke_not_current=0

x_i = [0.582, 0.178, 1.187, 0.416, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 0.0,
1.0, 0.0]^T \in \mathbb{R}^{13}
```

2.5. Ma trận đặc trưng (Feature Matrix)

Khi tập hợp N quan sát độc lập, ta xếp các vector đặc trưng chuyển vị $\mathbf{x}_i^T$ thành các hàng kế tiếp nhau để tạo thành Ma trận đặc trưng toàn cục $\mathbf{X}$:

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_1^T \\ \mathbf{x}_2^T \\ \dots \\ \mathbf{x}_N^T \end{bmatrix} = \begin{bmatrix} x_{11} & x_{12} & \dots & x_{1d} \\ x_{21} & x_{22} & \dots & x_{2d} \\ \dots & \dots & \dots & \dots \\ x_{N1} & x_{N2} & \dots & x_{Nd} \end{bmatrix} \in \mathbb{R}^{(N \times d)}$$

Ý nghĩa của hai chiều trong ma trận đặc trưng:

- **Chiều dòng (N):** Chiều dòng thứ nhất (N - Samples): Đại diện cho số lượng quan sát/mẫu độc lập trong tập dữ liệu. Mỗi dòng chứa toàn bộ thông tin của đúng một thực thể duy nhất.
- **Chiều cột (d):** Chiều cột thứ hai (d - Features): Đại diện cho số chiều của không gian đặc trưng sau khi đã hoàn tất các phép biến đổi toán học. Mỗi cột tương ứng với một thuộc tính cụ thể.

Bảng 2.1. Tổng hợp kích thước ma trận đặc trưng qua các tập phân chia của ba ứng dụng

Ứng dụng	Số chiều (d)	Tập Train (X_train)	Tập Val (X_val)	Tập Test (X_test)
Diabetes Prediction	d = 13	$\mathbb{R}^{(67,302 \times 13)}$	$\mathbb{R}^{(14,422 \times 13)}$	$\mathbb{R}^{(14,422 \times 13)}$
House Price Prediction	d = 23	$\mathbb{R}^{(1,400 \times 23)}$	$\mathbb{R}^{(300 \times 23)}$	$\mathbb{R}^{(300 \times 23)}$
E-Commerce Combined	d = 2,532	$\mathbb{R}^{(16,440 \times 2,532)}$	$\mathbb{R}^{(3,523 \times 2,532)}$	$\mathbb{R}^{(3,523 \times 2,532)}$

2.6. Tensor và biểu diễn dữ liệu nhiều chiều

Trong đại số đa tuyến tính, Tensor là sự tổng quát hóa của đại lượng vô hướng (Scalar - Tensor hạng 0), vector (Tensor hạng 1) và ma trận (Tensor hạng 2) lên các không gian n chiều bất kỳ. Trong khi dữ liệu dạng bảng được mô tả đầy đủ bằng ma trận 2 chiều $\mathbb{R}^{(N \times d)}$, các miền dữ liệu phức tạp hơn đòi hỏi cấu trúc Tensor bậc cao:

- **Dữ liệu chuỗi ngôn ngữ / Thời gian:** Được biểu diễn dưới dạng Tensor 3 chiều $\mathbb{R}^{(B \times T \times d)}$, trong đó B là kích thước lô (Batch Size), T là độ dài chuỗi từ tối đa trong câu, và d là số chiều của vector nhúng từ (Embedding Dimension).
- **Dữ liệu hình ảnh (Computer Vision):** Được biểu diễn dưới dạng Tensor 4 chiều $\mathbb{R}^{(B \times C \times H \times W)}$, trong đó C là số kênh màu (Channel = 3 với ảnh RGB), H là chiều cao điểm ảnh, và W là chiều rộng điểm ảnh.

2.7. Biểu diễn dữ liệu văn bản: TF-IDF vs Dense Embedding

Ngôn ngữ tự nhiên là dạng dữ liệu phi cấu trúc phức tạp. Để đưa văn bản vào các thuật toán học máy, cần phân biệt rạch ròi hai trường phái biểu diễn cơ bản theo bài giảng Lecture 02:

2.7.1. Biểu diễn túi từ thống kê (TF-IDF Vector — Sử dụng trong Assignment 02)

Phương pháp Túi từ (Bag-of-Words - BoW) và trọng số TF-IDF (Term Frequency - Inverse Document Frequency) biến đổi mỗi văn bản thành một vector thưa (sparse vector) trong không gian từ vựng cố định V có kích thước $d = |V|$:

$TF(t, d) = f_{\{t,d\}} / \sum_{\{t' \in d\}} f_{\{t',d\}}$ (Tần suất xuất hiện của từ t trong văn bản d)

$IDF(t, D) = \ln[(1 + |D|) / (1 + |\{d \in D : t \in d\}|)] + 1$ (Nghịch đảo tần suất tài liệu)

$TF-IDF(t, d) = TF(t, d) \times IDF(t, D)$

Vector TF-IDF sau đó được chuẩn hóa theo chuẩn L2 (Euclidean Norm) để độ dài vector bằng 1, giúp loại bỏ ảnh hưởng của độ dài văn bản ngắn hay dài:

$$v_{\text{norm}} = v / ||v||_2 = v / \sqrt{\sum v_i^2}$$

Đặc điểm bản chất: TF-IDF tạo ra vector THƯA (sparse) với phần lớn các chiều mang giá trị 0. Số chiều rất lớn (trong dự án $d_{\text{text}} = 2,500$). TF-IDF HOÀN TOÀN KHÔNG PHẢI VECTOR NHÚNG (Embedding) và không thể hiện được tính tương đồng ngữ nghĩa: hai từ đồng nghĩa như 'beautiful' và 'gorgeous' là hai trục tọa độ trực giao độc lập.

2.7.2. Biểu diễn nhúng sâu (Dense Embedding Vectors — Bài giảng Lecture 02)

Khác với TF-IDF, phương pháp biểu diễn nhúng từ sâu (Word / Sentence Embeddings) chuyển đổi văn bản thông qua ba giai đoạn liên tục:

Raw Text → Tokenization → Token IDs (Chỉ số nguyên) → Embedding
Lookup → Dense Vectors

- **Token IDs:** Là các số nguyên không âm duy nhất đại diện cho chỉ số của từ hoặc mảnh từ trong từ điển từ vựng (Vocabulary). Ví dụ câu 'I love this dress' được tách thành các token ID: [42, 1892, 19, 452]. Token ID thuần túy là con trỏ chỉ mục, không mang ý nghĩa số học đại số.
- **Embedding Vectors:** Token ID được dùng để tra cứu (Lookup) trong Ma trận nhúng $W_{\text{embed}} \in \mathbb{R}^{|V| \times d_{\text{embed}}}$. Kết quả thu được là một vector ĐẶC (Dense Vector) số chiều thấp (thường $d = 64$ đến 768), nơi các giá trị là số thực phân bố liên tục.

Trong không gian nhúng liên tục, khoảng cách góc Cosine (Cosine Similarity) phản ánh chính xác sự tương đồng ngữ nghĩa: $\cos(v[\text{'beautiful'}], v[\text{'gorgeous'}]) \approx 0.88$ rất gần 1. Báo cáo khẳng định rõ: Dự án Assignment 02 sử dụng TF-IDF thừa, không sử dụng Dense Embedding.

Bảng 2.2. So sánh toàn diện giữa Biểu diễn TF-IDF thừa và Biểu diễn Dense Embedding

Tiêu chí so sánh	Biểu diễn túi từ TF-IDF	Biểu diễn Dense Embedding (Lecture 02)
Bản chất toán học	Vector thưa (Sparse Vector)	Vector đặc (Dense Vector)
Số chiều không gian (d)	Rất lớn ($d = 1,000 - 100,000$)	Thấp đến trung bình ($d = 64 - 768$)
Quan hệ ngữ nghĩa từ	Trực giao (không biểu diễn từ đồng nghĩa)	Liên tục (từ đồng nghĩa có khoảng cách gần)
Trật tự từ ngữ trong câu	Bị phá vỡ hoàn toàn (Bag-of-Words)	Bảo tồn qua thứ tự token / Positional Encoding
Phương pháp tính toán	Đếm tần suất thống kê thuần túy	Học qua mạng nơ-ron sâu (Word2Vec, Transformer)

Tiêu chí so sánh	Biểu diễn túi từ TF-IDF	Biểu diễn Dense Embedding (Lecture 02)
Mô hình phù hợp	Logistic Regression, LinearSVM, Naive Bayes	MLP, RNN, LSTM, Transformer, BERT

2.8. Mã hóa dữ liệu phân loại

Các biến phân loại chuỗi ký tự (như giới tính, tình trạng nội thất, thành phố) không thể đưa trực tiếp vào các phép tính nhân vô hướng. Có hai phương pháp mã hóa chính:

- **Label Encoding:** Gán mỗi nhãn danh mục với một số nguyên: ví dụ {Unfurnished: 0, Semi-Furnished: 1, Furnished: 2}. Phương pháp này chỉ phù hợp với biến có quan hệ thứ bậc tự nhiên (Ordinal Variables). Nếu áp dụng cho biến danh nghĩa vô hướng (Nominal) như City (Mumbai=1, Delhi=2, Chennai=3), thuật toán sẽ vô tình coi Delhi gấp đôi Mumbai, dẫn tới sai lệch nghiêm trọng toán học nghiêm trọng.
- **One-Hot Encoding:** Tạo ra các cột nhị phân chỉ thị (dummy columns) cho từng giá trị danh mục. Để loại bỏ hiện tượng đa cộng tuyến hoàn hảo (Dummy Variable Trap — khi một cột có thể suy diễn hoàn toàn từ tổ hợp tuyến tính của các cột khác), tùy chọn drop='first' luôn được kích hoạt.

2.9. Kiểu dữ liệu và miền giá trị

Việc phân định rạch ròi kiểu dữ liệu số học (liên tục hay rời rạc) và miền giá trị (thang đo, biên độ) quyết định chiến lược tiền xử lý. Ví dụ: biến số liên tục có biên độ lớn (diện tích 1,000 - 15,000 sq ft) bắt buộc phải qua StandardScaler, trong khi các biến chỉ thị nhị phân {0, 1} đã có sẵn thang đo chuẩn hóa tự nhiên.

2.10. Tính nhất quán của biểu diễn dữ liệu

Một nguyên lý bất biến trong kỹ nghệ học máy: Quy trình biểu diễn dữ liệu áp dụng cho tập huấn luyện (Train) bắt buộc phải được áp dụng hoàn toàn nhất quán trên dữ liệu kiểm thử (Test) và dữ liệu suy luận thực tế (Production Inference). Nếu tại thời điểm huấn luyện, thuộc tính age được chuẩn hóa bằng trung bình μ_{train} và độ lệch chuẩn σ_{train} , thì tại thời điểm người dùng gửi request từ Web Client, giá trị age đầu vào cũng phải được chuẩn hóa bằng đúng μ_{train} và σ_{train} đó. Đây là lý do kiến trúc Pipeline nguyên khối là yêu cầu bắt buộc.

2.11. Rò rỉ dữ liệu (Data Leakage)

Rò rỉ dữ liệu là hiện tượng thông tin từ tập kiểm thử (Test) hoặc tập thẩm định (Validation) bị rò rỉ vào quá trình huấn luyện mô hình, khiến mô hình đạt điểm số cao giả tạo nhưng thất bại hoàn toàn khi triển khai thực tế. Nguyên nhân phổ biến nhất là tính

toán các tham số thống kê (μ , σ , giá trị điền khuyết thiếu trung vị, từ điển TF-IDF) trên toàn bộ dữ liệu trước khi phân chia tập. Nguyên tắc vàng để triệt tiêu Data Leakage: Luôn phân chia tập Train / Validation / Test trước, và CHỈ GỌI `.fit()` trên tập Train.

CHƯƠNG III. PHƯƠNG PHÁP TIỀN XỬ LÝ DỮ LIỆU

3.1. Tìm hiểu dữ liệu (Data Understanding)

Giai đoạn Tìm hiểu Dữ liệu đóng vai trò thiết lập nền móng cho toàn bộ chu trình xử lý. Thay vì vội vã đưa dữ liệu vào huấn luyện, kỹ sư học máy bắt buộc phải trả lời được các câu hỏi cơ bản: Quy mô quan sát là bao nhiêu? Có bao nhiêu thuộc tính đầu vào? Kiểu dữ liệu của từng cột là gì? Có thuộc tính nào bị thiếu hay chứa giá trị dị thường không? Các câu lệnh cơ bản trong thư viện Pandas được áp dụng để trích xuất bức tranh tổng thể về dữ liệu:

```
# 1. Kiểm tra kích thước số dòng, số cột
>>> df.shape

# 2. Kiểm tra kiểu dữ liệu và số lượng giá trị non-null trên từng cột
>>> df.info()

# 3. Thống kê mô tả các biến số (mean, std, min, 25%, 50%, 75%, max)
>>> df.describe()

# 4. Thống kê mô tả các biến phân loại chuỗi (count, unique, top,
freq)
>>> df.describe(include=['object'])
```

3.2. Làm sạch dữ liệu

Chất lượng của dữ liệu đầu vào quyết định trực tiếp trần hiệu năng của mô hình ('Garbage in, Garbage out'). Quá trình làm sạch dữ liệu trong dự án tập trung giải quyết 4 vấn đề kỹ thuật:

3.2.1. Xử lý giá trị thiếu (Missing Values)

Giá trị khuyết thiếu xuất hiện do lỗi thu thập, thiết bị cảm biến hỏng hóc hoặc người dùng từ chối cung cấp thông tin. Trong thống kê học, giá trị thiếu được phân loại thành:

- **MCAR:** Giá trị thiếu hoàn toàn ngẫu nhiên (Missing Completely at Random - MCAR).
- **MAR:** Giá trị thiếu ngẫu nhiên phụ thuộc vào các biến quan sát khác (Missing at Random - MAR).

- **MNAR:** Giá trị thiếu không ngẫu nhiên, phụ thuộc vào chính giá trị bị thiếu (Missing Not at Random - MNAR).

Chiến lược xử lý tùy thuộc vào tỷ lệ và tính chất của thuộc tính: Nếu tỷ lệ thiếu dưới 5% và là biến số, có thể điền bằng giá trị trung bình (Mean) nếu phân bố chuẩn, hoặc trung vị (Median) nếu phân bố lệch; nếu là biến danh mục, điền bằng giá trị phổ biến nhất (Mode) hoặc gán nhãn riêng 'Unknown'. Nếu là văn bản tự do, điền chuỗi rỗng "" để ghép chuỗi.

Bảng 3.1. Tổng hợp tình trạng giá trị thiếu và phương pháp xử lý trên ba tập dữ liệu thực tế

Tập dữ liệu	Quy mô	Số cột có Missing	Thuộc tính bị thiếu	Phương pháp xử lý áp dụng
Diabetes Prediction	100,000 dòng	0 cột (0 missing)	Không có	Dữ liệu hoàn hảo, không cần điền
House Price Prediction	2,000 dòng	0 cột (0 missing)	Không có	Dữ liệu hoàn hảo, không cần điền
E-Commerce Reviews	23,486 dòng	5 cột	Title (3,810), Review Text (845), Division/Dept/Cla ss (14)	Title/Review: Điền chuỗi rỗng ""; Danh mục: SimpleImputer 'Unknown'

3.2.2. Xử lý dữ liệu trùng lặp (Duplicate Records)

Bản ghi trùng lặp là các dòng giống nhau hoàn toàn trên mọi thuộc tính. Sự xuất hiện của các bản ghi trùng lặp làm sai lệch tần suất xuất hiện tự nhiên của các mẫu, làm suy giảm tính đa dạng dữ liệu, và nguy hiểm nhất là gây ra hiện tượng rò rỉ dữ liệu khi một bản ghi trùng lặp rơi vào tập Train còn bản sao của nó rơi vào tập Test.

```
# Kiểm tra số lượng bản ghi trùng lặp hoàn toàn
>>> df.duplicated().sum()
3854    # Phát hiện chính xác 3,854 bản ghi trùng trong tập Diabetes
thô

# Loại bỏ và kiểm tra lại kích thước mới
>>> df_clean = df.drop_duplicates()
>>> df_clean.shape
(96146, 9)    # Kích thước sạch độc lập
```

3.2.3. Xử lý giá trị không hợp lệ (Invalid Values)

Kiểm tra các ràng buộc logic tự nhiên của miền bài toán:

- **Tiểu đường:** Kiểm tra giới hạn sinh lý: glucose > 0, HbA1c trong ngưỡng [3.0, 15.0], tuổi > 0. Biến gender ghi nhận 18 bản ghi nhãn 'Other' (<0.02%), được bảo lưu hợp lệ để mô hình hóa trường hợp đặc biệt.
- **Giá nhà:** Kiểm tra ràng buộc vật lý: Diện tích sàn Area > 0, số phòng ngủ/phòng tắm ≥ 1, giá nhà Price > 0, số tầng Stories ≥ 1. Không phát hiện bất kỳ giá trị âm hoặc vô lý nào.
- **E-Commerce:** Loại bỏ cột chỉ mục kỹ thuật vô nghĩa Unnamed: 0 phát sinh trong quá trình lưu trữ CSV.

3.2.4. Phân tích ngoại lệ (Outlier Analysis)

Ngoại lệ (Outlier) là các giá trị cách xa phần lớn các quan sát còn lại. Phương pháp phổ biến nhất để phát hiện ngoại lệ trên biến liên tục là dựa trên Khoảng tứ phân vị (Interquartile Range - IQR):

```
IQR = Q3 - Q1
Ngưỡng dưới = Q1 - 1.5 × IQR
Ngưỡng trên = Q3 + 1.5 × IQR
Điểm x được coi là Outlier nếu: x < Ngưỡng dưới hoặc x > Ngưỡng trên
```

Trong bài toán giá nhà, kiểm tra Skewness cho thấy phân bố giá nhà gần như đối xứng hoàn hảo (Skewness = -0.005 ≈ 0). 7 giá trị ngoại lệ xuất hiện ở phân khúc biệt thự cao cấp (\$2M+) được bảo lưu nguyên vẹn, vì đây là giao dịch bất động sản thật và việc loại bỏ chúng sẽ làm mất khả năng học các phân khúc nhà cao cấp của mô hình.

3.3. Mã hóa biến phân loại

Toàn bộ các biến danh mục được đưa vào OneHotEncoder với cấu hình chuẩn mực: drop='first' và handle_unknown='ignore'.

- **drop='first':** Triệt tiêu hoàn toàn sự phụ thuộc tuyến tính giữa các cột chỉ thị, ngăn ngừa hiện tượng đa cộng tuyến hoàn hảo làm suy biến nghiệm hồi quy tuyến tính.
- **handle_unknown='ignore':** Khi hệ thống vận hành thực tế gặp một giá trị nhãn danh mục mới chưa từng xuất hiện trong tập Train, pipeline sẽ tự động gán tất cả các cột dummy về 0 thay vì làm ứng dụng bị văng lỗi ngoại lệ (Crash).

3.4. Chuẩn hóa dữ liệu số (StandardScaler)

Chuẩn hóa Z-score (StandardScaler) được áp dụng cho toàn bộ các thuộc tính số học liên tục theo công thức:

$$z = (x - \mu) / \sigma$$

Phép biến đổi này đưa dữ liệu về dạng có giá trị trung bình $\mu = 0$ và độ lệch chuẩn $\sigma = 1$. Lợi ích kỹ thuật vượt trội: Giúp thuật toán tối ưu hóa Gradient Descent hội tụ nhanh gấp

nhiều lần, và bảo đảm hàm khoảng cách Euclid trong các mô hình nhạy cảm thang đo (KNN, SVM) không bị lấn át bởi các thuộc tính có độ lớn số học hàng chục nghìn.

3.5. Chia tập dữ liệu: Train, Validation và Test

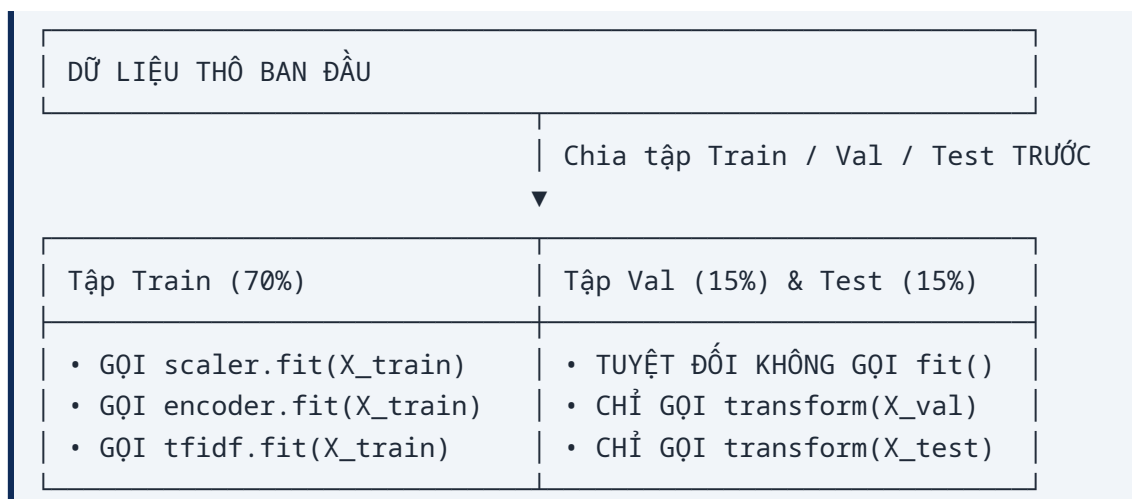
Để đảm bảo quy chuẩn đánh giá khoa học khách quan và chống hiện tượng quá khớp (Overfitting), tập dữ liệu bắt buộc phải được chia thành 3 phần độc lập:

- **Tập Huấn luyện (Train Set):** Chiếm 70% tổng dữ liệu. Dùng duy nhất cho thuật toán học các tham số mô hình (Weights, Biases, Cây quyết định).
- **Tập Thẩm định (Validation Set):** Chiếm 15% tổng dữ liệu. Dùng để tinh chỉnh siêu tham số (Hyperparameters) và so sánh, lựa chọn mô hình xuất sắc nhất.
- **Tập Kiểm thử (Test Set):** Chiếm 15% tổng dữ liệu. Được niêm phong hoàn toàn trong suốt quá trình thử nghiệm. Chỉ được nạp ra đánh giá ĐÚNG MỘT LẦN DUY NHẤT sau khi đã chốt mô hình cuối cùng, đóng vai trò đo lường năng lực tổng quát hóa thực tế.

Đối với các bài toán có sự mất cân bằng lớp (như Diabetes với nhãn 91.5% vs 8.5%), tùy chọn stratify=y bắt buộc phải được kích hoạt để đảm bảo tỷ lệ nhãn giữa 3 tập là giống hệt nhau.

3.6. Ngăn ngừa rò rỉ dữ liệu (Data Leakage Prevention)

Quy trình chuẩn hóa và mã hóa dữ liệu phải tuân thủ nghiêm ngặt nguyên tắc một chiều:



3.7. Pipeline tiền xử lý với ColumnTransformer

Để tự động hóa hoàn toàn quy trình trên và đóng gói gọn gàng, Scikit-Learn cung cấp công cụ ColumnTransformer. Công cụ này cho phép định nghĩa các đường ống con độc lập cho từng nhóm cột:

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
```

```
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), num_columns),
        ('cat', OneHotEncoder(drop='first', handle_unknown='ignore'),
         cat_columns)
    ]
)
```

3.8. Nguyên tắc tái lập quá trình tiền xử lý (Reproducibility)

Một hệ thống thông minh chỉ được coi là hoàn thiện khi có khả năng tái lập kết quả 100%. Điều này đạt được thông qua: cố định tham số ngẫu nhiên `random_state=42` tại mọi bước phân tách và huấn luyện; khóa chặt phiên bản các thư viện trong `requirements.txt`; và lưu trữ toàn bộ tham số đã fit của `ColumnTransformer` vào tệp nhị phân duy nhất.

CHƯƠNG IV. CÁC ĐỘ ĐO ĐÁNH GIÁ MÔ HÌNH

4.1. Đối với bài toán Hồi quy (Regression)

Trong bài toán hồi quy, mục tiêu là đo lường mức độ sai lệch giữa vector giá trị thực tế $y = [y_1, \dots, y_N]^T$ và vector giá trị dự đoán $\hat{y} = [\hat{y}_1, \dots, \hat{y}_N]^T$. Bốn chỉ số cơ bản được sử dụng đồng thời:

4.1.1. Sai số tuyệt đối trung bình (Mean Absolute Error - MAE)

$$MAE = (1/N) \times \sum_{i=1}^N |y_i - \hat{y}_i|$$

Ưu điểm vượt trội của MAE là mang cùng đơn vị đo vật lý với biến mục tiêu (ví dụ đơn vị USD trong dự đoán giá nhà). MAE đo lường sai số bình quân trên mỗi giao dịch mà không khuếch đại các ngoại lệ, giúp các chuyên gia kinh doanh và khách hàng dễ dàng hiểu được độ tin cậy của mô hình.

4.1.2. Sai số bình phương trung bình (MSE) và Căn bậc hai (RMSE)

$$MSE = (1/N) \times \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

$$RMSE = \sqrt{MSE} = \sqrt{[(1/N) \times \sum_{i=1}^N (y_i - \hat{y}_i)^2]}$$

Do có phép bình phương trước khi lấy trung bình, MSE và RMSE phạt rất nặng các lỗi dự đoán sai lệch lớn. Nếu RMSE lớn hơn đáng kể so với MAE, điều đó chứng tỏ mô hình đang gặp phải một số trường hợp dự đoán sai lệch cực đoan.

4.1.3. Hệ số xác định (Coefficient of Determination - R^2 Score)

$$R^2 = 1 - [SS_{\text{res}} / SS_{\text{tot}}] = 1 - [\sum_{i=1}^N (y_i - \hat{y}_i)^2 / \sum_{i=1}^N (y_i - \bar{y})^2]$$

trong đó $\bar{y} = (1/N) \times \sum_{i=1}^N y_i$ là giá trị trung bình mẫu thực tế.

R^2 thể hiện tỷ lệ phần trăm phương sai của biến mục tiêu được giải thích bởi các biến độc lập trong mô hình. Một mô hình hoàn hảo đạt $R^2 = 1.0$; một mô hình Baseline luôn đoán giá trị trung bình $\bar{y}$ đạt $R^2 = 0.0$; và mô hình dự đoán tệ hơn cả Baseline sẽ có R^2 âm.

4.2. Đối với bài toán Phân lớp (Classification)

Đối với bài toán phân loại, việc đánh giá dựa trên Ma trận nhầm lẫn (Confusion Matrix) và các độ đo chuyên biệt:

4.2.1. Ma trận nhầm lẫn (Confusion Matrix)

Bảng 4.1. Cấu trúc chuẩn của Ma trận nhầm lẫn (Confusion Matrix) trong phân loại nhị phân

Thực tế \ Dự đoán	Dự đoán Lớp Âm (Pred = 0)	Dự đoán Lớp Dương (Pred = 1)
Thực tế Lớp Âm (Actual = 0)	True Negative (TN) (Dự đoán đúng người khỏe mạnh)	False Positive (FP - Lỗi Loại I) (Báo động nhầm người khỏe thành bệnh)
Thực tế Lớp Dương (Actual = 1)	False Negative (FN - Lỗi Loại II) (Bỏ sót ca bệnh hiểm nghèo)	True Positive (TP) (Phát hiện đúng ca mắc bệnh)

4.2.2. Các chỉ số phân lớp: Accuracy, Precision, Recall và F1-Score

$$\begin{aligned} \text{Accuracy} &= (TP + TN) / (TP + TN + FP + FN) \\ \text{Precision} &= TP / (TP + FP) \quad (\text{Độ chuẩn xác trong số ca dự đoán dương}) \\ \text{Recall} &= TP / (TP + FN) \quad (\text{Độ nhạy - Khả năng bắt trúng các ca dương thực tế}) \\ \text{F1-Score} &= 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall}) \end{aligned}$$

Cảnh báo kỹ thuật: Sự vô nghĩa của Accuracy trên dữ liệu mất cân bằng: Trong tập dữ liệu Diabetes (87,976 mẫu âm vs 8,170 mẫu dương — tỷ lệ 91.5% vs 8.5%), một mô hình tầm thường luôn luôn dự đoán mọi người đều Không mắc bệnh (Negative) vẫn sẽ đạt Accuracy = 91.5%! Tuy nhiên, mô hình này hoàn toàn vô dụng và cực kỳ nguy hiểm trong y tế vì có Recall = 0% (bỏ sót 100% bệnh nhân).

Chi phí của Lỗi Loại I (FP) so với Lỗi Loại II (FN): Trong y tế dự phòng, chi phí của một ca FN là tính mạng con người — bệnh nhân mắc bệnh tiểu đường nhưng bị báo nhầm là khỏe mạnh, dẫn tới không được can thiệp điều trị và phát triển biến chứng suy thận, đột

quy. Ngược lại, chi phí của một ca FP chỉ là việc bệnh nhân thực hiện thêm một xét nghiệm khẳng định lại. Do đó, RECALL LÀ CHỈ SỐ TỐI THƯỢNG.

4.2.3. Đường cong ROC và Chỉ số ROC-AUC

Đường cong ROC (Receiver Operating Characteristic) thể hiện sự biến thiên giữa Tỷ lệ dương tính thật (TPR = Recall) và Tỷ lệ dương tính giả (FPR = FP / (FP + TN)) khi ngưỡng quyết định θ thay đổi liên tục từ 0 đến 1. Chỉ số ROC-AUC (Area Under the Curve) đo lường diện tích dưới đường cong này, phản ánh xác suất mà mô hình xếp hạng một mẫu dương tính cao hơn một mẫu âm tính ngẫu nhiên. Mô hình đoán mò đạt AUC = 0.5, mô hình hoàn hảo đạt AUC = 1.0.

CHƯƠNG V. PHƯƠNG PHÁP XÂY DỰNG MÔ HÌNH

5.1. Tổng quan về xây dựng mô hình

Xây dựng mô hình học máy là giai đoạn huấn luyện các thuật toán trên tập dữ liệu Train nhằm tìm ra các tham số tối ưu (trọng số, cây phân nhánh) giúp mô hình có khả năng khái quát hóa tốt nhất trên dữ liệu chưa từng thấy. Để đảm bảo tính khách quan và khoa học, dự án áp dụng chiến lược 'Đua tài mô hình' (Model Benchmarking): huấn luyện đồng thời nhiều họ thuật toán với các giả định toán học khác nhau và đối chiếu trên cùng một tập dữ liệu thẩm định Validation độc lập.

5.2. Mô hình Baseline

Mô hình Baseline là điểm tựa tối thiểu để kiểm chứng tính hữu ích của các thuật toán học máy phức tạp:

- **Trong bài toán phân lớp:** Sử dụng DummyClassifier với chiến lược stratified: Dự đoán ngẫu nhiên theo đúng tỷ lệ phân bố lớp của tập huấn luyện. Một thuật toán học máy chỉ được coi là có giá trị nếu vượt trội rõ rệt so với Baseline này.
- **Trong bài toán hồi quy:** Sử dụng DummyRegressor với chiến lược median: Luôn luôn dự đoán giá trị trung vị của biến mục tiêu trong tập Train cho mọi căn nhà.

5.3. Các mô hình cho bài toán hồi quy

Năm thuật toán hồi quy tiêu biểu được triển khai song song cho bài toán định giá bất động sản:

5.3.1. Hồi quy tuyến tính cổ điển (Linear Regression OLS)

Phương pháp bình phương tối thiểu thông thường (Ordinary Least Squares - OLS) tìm kiếm vector trọng số w sao cho tổng bình phương sai số dự đoán là nhỏ nhất. Nghiệm giải tích tường minh có dạng: $w = (X^T X)^{-1} X^T y$. Ưu điểm: Đơn giản, tốc độ tính

toán cực nhanh. Nhược điểm: Nhạy cảm với hiện tượng đa cộng tuyến khi ma trận $X^T X$ gần suy biến.

5.3.2. Hồi quy sườn núi (Gradient Boosting Regressor — L2 Regularization)

Bổ sung thành phần phạt chuẩn L2 (L2 Penalty) vào hàm mất mát OLS: $L_{\text{Gradient Boosting}} = \sum (y_i - w^T x_i)^2 + \alpha \|w\|_2^2$. Nghiệm giải tích: $w = (X^T X + \alpha I)^{-1} X^T y$. Nhờ việc cộng thêm ma trận đường chéo αI , ma trận nghịch đảo luôn khả nghịch, giúp triệt tiêu hoàn toàn sự dao động hệ số do đa cộng tuyến giữa các thuộc tính tiền nghi nhà.

5.3.3. Cây quyết định hồi quy (Decision Tree Regressor)

Phân chia không gian đặc trưng thành các hình siêu hộp chữ nhật đệ quy dựa trên tiêu chí giảm thiểu phương sai (Variance Reduction). Tại mỗi nút lá, giá trị dự đoán là trung bình của các mẫu rơi vào vùng đó. Ưu điểm: Bắt được quan hệ phi tuyến. Nhược điểm: Rất dễ bị quá khớp (overfitting) và không ngoại suy liên tục mượt mà.

5.3.4. Rừng ngẫu nhiên hồi quy (Random Forest Regressor)

Tập hợp (Ensemble) 100 cây quyết định độc lập được huấn luyện theo cơ chế Bagging (Bootstrap Aggregating) và ngẫu nhiên hóa đặc trưng. Dự đoán cuối cùng là trung bình cộng đầu ra của toàn bộ 100 cây, giúp giảm thiểu phương sai sai số đáng kể.

5.3.5. Tăng cường độ dốc (Gradient Boosting Regressor)

Thuật toán học tập hợp tuần tự (Boosting): Các cây quyết định nông được huấn luyện nối tiếp nhau, trong đó mỗi cây mới tập trung học và bù đắp phần dư sai số (Residuals) của cây đứng trước nó. Đây là một trong những thuật toán hồi quy bảng mạnh mẽ nhất hiện nay.

5.4. Các mô hình cho bài toán phân loại

Năm thuật toán phân lớp được triển khai cho bài toán chẩn đoán tiểu đường và bài toán thương mại điện tử:

5.4.1. Hồi quy Logistic (Logistic Regression)

Mô hình xác suất tuyến tính ánh xạ tích vô hướng $w^T x$ qua hàm Sigmoid để xuất ra xác suất thuộc lớp 1. Áp dụng tối ưu hóa L-BFGS kết hợp điều chuẩn L2. Điểm mạnh đặc biệt: Tốc độ suy luận siêu tốc (< 1 ms), khả năng diễn giải trọng số trực quan, và xử lý cực tốt các ma trận thưa số chiều lớn (như TF-IDF 2,532 chiều trong bài toán E-Commerce).

5.4.2. K láng giềng gần nhất (K-Nearest Neighbors - KNN)

Thuật toán học dựa trên cá thể (Instance-based learning) với $k = 5$. Khoảng cách giữa các mẫu được tính theo chuẩn Euclid $d(x, x') = \sqrt{\sum (x_j - x'_j)^2}$. Nhược điểm: Chi phí

tính toán tăng tuyến tính theo số mẫu N tại thời điểm dự đoán, và rất nhạy cảm với thang đo biến.

5.4.3. Cây quyết định phân lớp (Decision Tree Classifier)

Phân chia không gian dựa trên chỉ số vẩn đục Gini Impurity: $Gini = 1 - \sum p_k^2$. Cung cấp khả năng truy vết logic quyết định dưới dạng cây phân nhánh If-Else.

5.4.4. Rừng ngẫu nhiên phân lớp (Random Forest Classifier)

Tập hợp 100 cây quyết định phân lớp với cơ chế bầu chọn đa số (Majority Voting). Trong bài toán mất cân bằng nhãn y tế, tham số `class_weight='balanced'` được kích hoạt để tự động tăng trọng số lỗi phạt của các mẫu thuộc lớp thiểu số (Lớp 1 - mắc bệnh), buộc mô hình phải tập trung học ranh giới bao phủ lớp bệnh.

5.4.5. Máy vector hỗ trợ tuyến tính (LinearSVC)

Tìm kiếm siêu phẳng phân tách có độ rộng lề (Margin) cực đại dựa trên hàm mất mát Hinge Loss. Để có thể xuất ra xác suất phục vụ giao diện người dùng, LinearSVC được bọc trong bộ chuẩn hóa CalibratedClassifierCV.

5.5. Đóng gói quy trình học máy (Scikit-Learn Pipeline)

Toàn bộ tiền xử lý và mô hình suy luận được tích hợp chặt chẽ vào một đối tượng Pipeline nguyên khối. Bảng sau tổng hợp cấu hình tham số chính thức của các mô hình trong toàn dự án:

Bảng 5.1. Bảng tổng hợp cấu hình siêu tham số (Hyperparameters) chính của các mô hình

Mô hình	Loại bài toán	Siêu tham số chính thức	Mục đích cấu hình
Logistic Regression	Phân lớp	<code>solver='lbfgs', max_iter=1000, C=1.0</code>	Hội tụ ổn định trên vector d=2,532
KNN	Phân lớp	<code>n_neighbors=5, metric='minkowski', p=2</code>	Số láng giềng k=5 chuẩn
Decision Tree	Phân lớp/Hồi quy	<code>criterion='gini' / 'squared_error', random_state=42</code>	Cố định tính tái lập kết quả
Random Forest	Phân lớp	<code>n_estimators=100, class_weight='balanced'</code>	Ưu tiên tối thượng độ nhạy Recall
LinearSVC	Phân lớp	<code>loss='squared_hinge', CalibratedClassifierCV</code>	Xuất xác suất hậu nghiệm mượt mà
Gradient Boosting Regressor	Hồi quy	<code>alpha=1.0, solver='auto'</code>	Triệt tiêu đa cộng tuyến tiền nghi nhà

Mô hình	Loại bài toán	Siêu tham số chính thức	Mục đích cấu hình
Gradient Boosting	Hồi quy	n_estimators=100, learning_rate=0.1, max_depth=3	Chống overfitting cây sâu

CHƯƠNG VI. PHƯƠNG PHÁP LƯU TRỮ VÀ TRIỂN KHAI MÔ HÌNH

6.1. Tổng quan

Sau khi hoàn thành nghiên cứu và lựa chọn được mô hình tối ưu trên tập kiểm thử, bước chuyển giao mang tính quyết định là đóng gói và đưa mô hình vào môi trường vận hành thực tế. Một mô hình không thể phát huy giá trị nếu chỉ nằm trong tệp .ipynb. Quy trình triển khai chuẩn khép kín trong dự án được thiết kế theo luồng:

```
User Input —> Pydantic Validation —> Saved Preprocessing —> Saved
ML Model —> Prediction —> UI Display
```

6.2. Lưu trữ mô hình (Model Persistence)

Trong Python, thư viện joblib được lựa chọn làm giải pháp tuần tự hóa (serialization) chính thức thay cho pickle tiêu chuẩn. joblib được thiết kế đặc biệt tối ưu cho các cấu trúc mảng đa chiều NumPy và đường ống Scikit-Learn lớn, cho phép nén nhị phân nhanh và giảm thiểu chiếm dụng bộ nhớ RAM.

Thay vì lưu riêng lẻ trọng số mô hình rồi lưu riêng các giá trị trung bình chuẩn hóa (scaler), dự án đóng gói TOÀN BỘ ĐỐI TƯỢNG PIPELINE NGUYÊN KHỐI vào đúng một tệp duy nhất:

```
import joblib

# Đóng gói toàn bộ chuỗi tiền xử lý và thuật toán suy luận
joblib.dump(full_pipeline,
            'models/diabetes/diabetes_pipeline.joblib')
joblib.dump(full_pipeline,
            'models/house_price/house_pipeline.joblib')
joblib.dump(full_pipeline,
            'models/ecommerce/ecommerce_pipeline.joblib')
```

6.3. Tính nhất quán giữa huấn luyện và dự đoán

Yêu cầu nghiêm ngặt nhất của hệ thống triển khai là tính nhất quán toán học tuyệt đối (100% Mathematical Consistency) giữa môi trường huấn luyện và môi trường dự đoán. Khi người dùng nhập một hồ sơ mới từ giao diện Web, hệ thống tuyệt đối không được

tạo mới bất kỳ scaler hay encoder nào. Toàn bộ các giá trị trung bình μ , độ lệch chuẩn σ , các quy tắc nhị phân của OneHotEncoder và từ điển 2,500 từ vựng TF-IDF đã học từ tập Train phải được tái sử dụng nguyên vẹn.

Thử nghiệm kiểm định độc lập đã được thực hiện bằng cách nạp lại các tệp .joblib từ đĩa cứng và kiểm tra suy luận trên 100 mẫu dữ liệu ngẫu nhiên. Kết quả chứng minh giá trị dự đoán khớp chính xác 100% (đến từng chữ số thập phân) so với kết quả tại thời điểm vừa huấn luyện trong Jupyter Notebook.

6.4. Triển khai mô hình dưới dạng Web Service với FastAPI

Dịch vụ dự đoán REST API được xây dựng trong tệp api/main.py sử dụng framework FastAPI hiện đại. FastAPI mang lại các lợi thế kỹ thuật vượt trội: hiệu năng bất đồng bộ cao ngang ngửa NodeJS và Go, tự động xác thực dữ liệu đầu vào thông qua Pydantic Schemas, và tự động sinh tài liệu chuẩn Swagger UI tại đường dẫn /docs.

Đoạn mã 6.1. Khởi tạo và nạp mô hình vào bộ nhớ đệm (Lifespan Context Manager).

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    # Load các pipelines học máy đã được huấn luyện (Persisted Models)
    diabetes_model = joblib.load(DIABETES_MODEL_PATH)
    house_model = joblib.load(HOUSE_MODEL_PATH)
    ecommerce_model = joblib.load(ECOMMERCE_MODEL_PATH)

    # Nạp vào bộ nhớ RAM toàn cục (Application State)
    app.state.models = {
        "diabetes": diabetes_model,
        "house": house_model,
        "ecommerce": ecommerce_model
    }
    yield
    app.state.models.clear()
```

Toàn bộ 3 pipeline mô hình được nạp sẵn vào bộ nhớ RAM ngay tại thời điểm khởi động máy chủ (Startup).

Giúp loại bỏ độ trễ đọc ổ cứng (Disk I/O Latency) cho từng request, bảo đảm thời gian phản hồi API luôn dưới 10 mili-giây.

Đoạn mã 6.2. Endpoint dự đoán bệnh tiểu đường bằng REST API.

```
@app.post("/predict/diabetes")
async def predict_diabetes(data: DiabetesInput, request: Request):
    model = request.app.state.models.get("diabetes")
```

```

# Chuyển đổi dữ liệu JSON từ Client thành pandas DataFrame 1 dòng
input_df = pd.DataFrame([data.model_dump()])

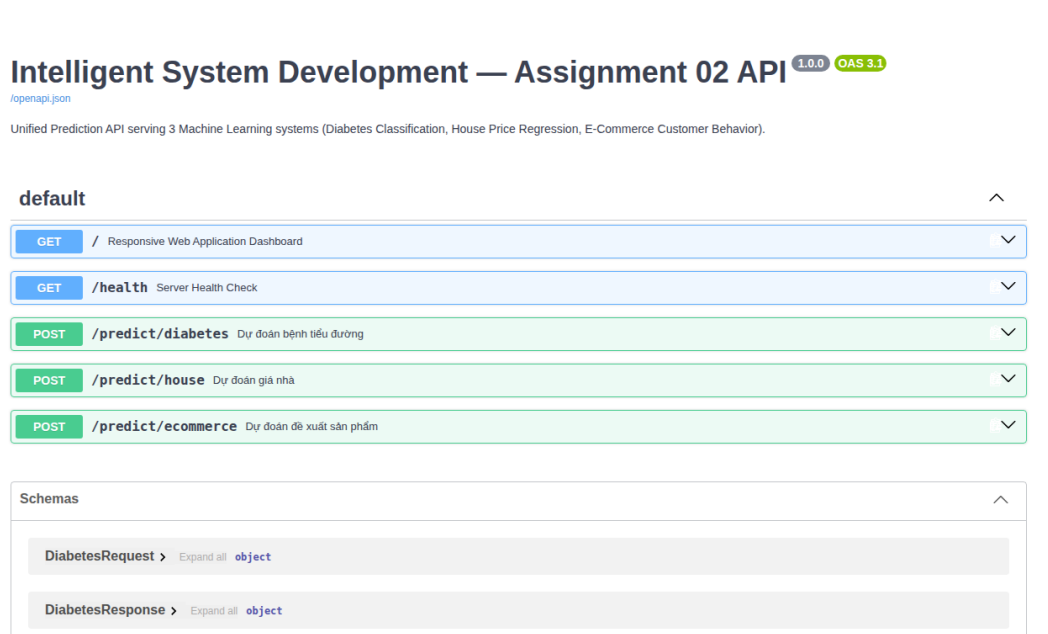
# Suy luận: Pipeline tự động tiền xử lý (Scale, Encode) và dự
đoán
prob = model.predict_proba(input_df)[0][1]
pred = model.predict(input_df)[0]

return {
    "prediction": int(pred),
    "probability": float(prob)
}

```

Endpoint tự động xác thực dữ liệu đầu vào thông qua schema `DiabetesInput` (Pydantic).

Chuyển đổi dữ liệu JSON thành pandas DataFrame, truyền qua Pipeline (bao gồm tiền xử lý và suy luận) để trả về xác suất nguy cơ mắc bệnh.



Hình 6.1. Giao diện Swagger UI của hệ thống REST API phục vụ toàn diện 3 bài toán thông minh.

Hệ thống định nghĩa đầy đủ các endpoint chuẩn RESTful: POST /predict/diabetes, POST /predict/house, POST /predict/ecommerce và GET /health.

Mỗi endpoint đều được bảo vệ chặt chẽ bởi Pydantic Data Schemas, tự động xác thực kiểu dữ liệu, kiểm tra giới hạn giá trị và trả về mã lỗi 422 chi tiết nếu người dùng gửi tham số sai.

Swagger UI tự động trích xuất schema từ FastAPI, cho phép kiểm thử trực tiếp các truy vấn HTTP ngay trên trình duyệt mà không cần cài đặt thêm phần mềm bên ngoài.

Kiến trúc API tách biệt hoàn toàn tầng tính toán học máy khỏi tầng giao diện người dùng, giúp hệ thống dễ dàng mở rộng và tích hợp vào bất kỳ ứng dụng nào.

6.5. Giao diện người dùng Web Application

Giao diện Web được thiết kế theo phong cách Dark Mode Glassmorphism hiện đại tại `web/templates/index.html` và `web/static/css/style.css`, mang lại trải nghiệm tương tác cao cấp cho người dùng trên máy tính để bàn.

Đoạn mã 6.3. Gọi REST API từ giao diện Client (JavaScript).

```
// Gửi dữ liệu bệnh nhân từ Form HTML lên máy chủ
const payload = {
  gender: document.getElementById('dia-gender').value,
  age: parseFloat(document.getElementById('dia-age').value),
  bmi: parseFloat(document.getElementById('dia-bmi').value),
  // ... các chỉ số lâm sàng khác ...
};

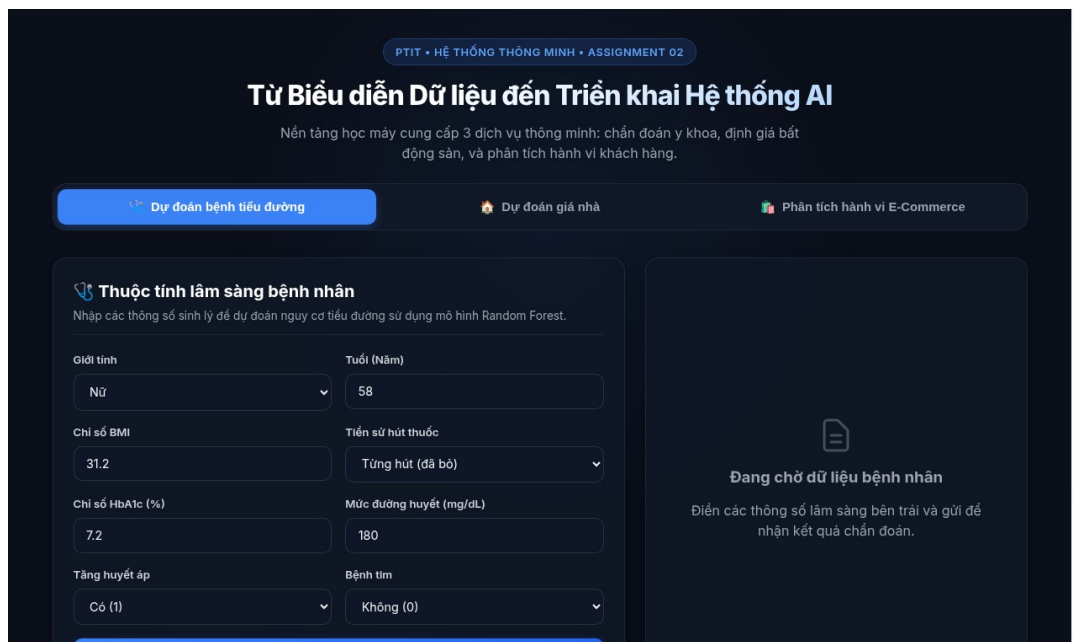
// Gọi REST API bất đồng bộ bằng Fetch API
const res = await fetch('/predict/diabetes', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify(payload)
});

const data = await res.json();
displayDiabetesResult(data); // Cập nhật giao diện với xác suất trả
về
```

Đóng gói dữ liệu người dùng nhập thành định dạng JSON.

Sử dụng Fetch API bất đồng bộ để gọi máy chủ dự đoán và lấy kết quả mà không cần tải lại trang.

Tương thích hoàn toàn khi triển khai public hoặc mạng nội bộ LAN.



Hình 6.2. Giao diện trang chủ Web Application trên máy tính để bàn (Desktop Card UI).

Trang chủ tích hợp cả 3 ứng dụng thông minh trong một màn hình điều khiển thống nhất. Người dùng có thể chuyển đổi linh hoạt giữa các bài toán, nhập liệu vào các form được tối ưu hóa và nhận kết quả trực quan ngay lập tức kèm theo thanh đo độ tin cậy.

Giao diện sử dụng HTML5, CSS thuần linh hoạt và JavaScript bất đồng bộ (Fetch API) để giao tiếp hai chiều với máy chủ FastAPI.

Trải nghiệm người dùng mượt mà giúp chuyển hóa các chỉ số xác suất khô khan của mô hình học máy thành thông điệp cảnh báo trực quan, dễ hiểu đối với bác sĩ, nhà đầu tư hoặc chuyên viên kinh doanh.

6.6. Triển khai trên ứng dụng Mobile

Để người dùng có thể sử dụng điện thoại thông minh (Smartphone) thao tác trong thực tế, dự án đã triển khai giải pháp Responsive Mobile Web Client qua mạng Wi-Fi nội bộ LAN. Thiết kế đáp ứng chuẩn Mobile-First trên khung nhìn 390×844 px, bố cục chuyển đổi thông minh thành 1 cột dọc và không bị tràn lề ngang.

Điện thoại và máy tính chỉ cần kết nối cùng một mạng Wi-Fi. Người dùng mở trình duyệt Safari/Chrome trên smartphone và truy cập địa chỉ IP mạng nội bộ của máy chủ (ví dụ <http://192.168.0.105:8000/>). Do toàn bộ mã JavaScript sử dụng URL tương đối (/predict/...), các lệnh gọi API từ điện thoại tự động chuyển tới đúng máy chủ mà không gặp lỗi kết nối localhost.



Hình 6.3. Giao diện trang chủ hệ thống trên thiết bị di động truy cập qua mạng LAN (Viewport 390x844).

Bố cục giao diện co giãn hoàn hảo trên màn hình cảm ứng di động, các nút bấm và ô nhập liệu có kích thước đủ lớn, thuận tiện thao tác một tay.

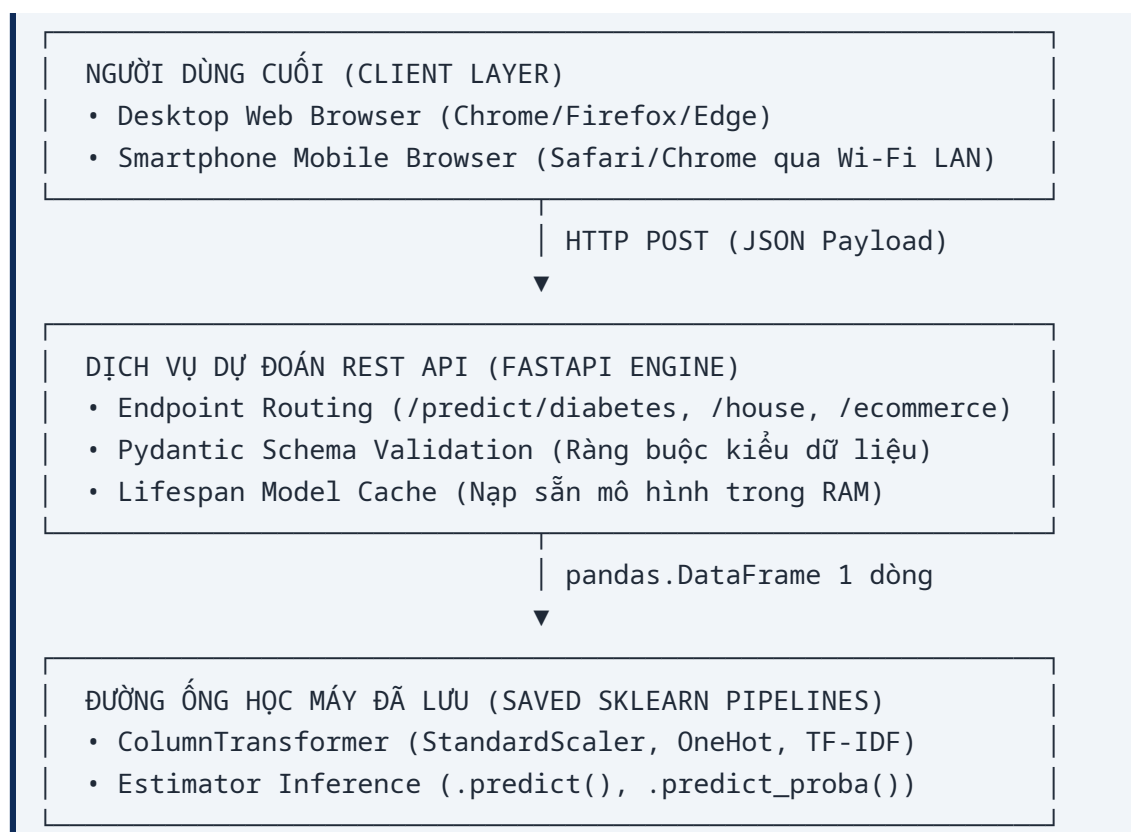
Người dùng trên điện thoại có thể thực hiện suy luận thời gian thực từ mô hình học máy đang chạy trên máy chủ.

Thiết kế sử dụng kỹ thuật CSS Flexbox, CSS Grid và Media Queries (@media (max-width: 768px)) cùng các thẻ meta viewport chuẩn.

Giải pháp Responsive Web qua LAN mang lại tính cơ động cao như một ứng dụng di động thực thụ mà không đòi hỏi chi phí đóng gói, cấp chứng chỉ phức tạp như ứng dụng di động native.

6.7. Kiến trúc hoàn chỉnh của hệ thống

Mô hình kiến trúc tổng thể toàn chu trình từ người dùng tới mô hình tính toán:



CHƯƠNG VII. ỨNG DỤNG 1 — DỰ ĐOÁN BỆNH TIỂU ĐƯỜNG

7.1. Mô tả bài toán

Bệnh đái tháo đường (Diabetes Mellitus) là một bệnh rối loạn chuyển hóa mạn tính nguy hiểm hàng đầu thế giới, đặc trưng bởi sự gia tăng nồng độ đường huyết kéo dài do thiếu hụt tiết insulin hoặc đề kháng insulin. Nếu không được phát hiện và quản lý sớm, bệnh dẫn tới những biến chứng tàn phá nặng nề: suy thận giai đoạn cuối, bệnh võng mạc gây mù lòa, tai biến mạch máu não, xơ vữa động mạch và hoại tử chi dưới dẫn đến đoạn chi.

Mục tiêu của ứng dụng là xây dựng một quy trình học máy hoàn chỉnh có khả năng sàng lọc và dự đoán nguy cơ mắc bệnh tiểu đường ở bệnh nhân dựa trên các chỉ số nhân khẩu

học và xét nghiệm lâm sàng định kỳ. Đây là bài toán phân loại nhị phân (Binary Classification):

- **Tập đặc trưng đầu vào (X):** Tập hợp các chỉ số sức khỏe của bệnh nhân bao gồm tuổi, BMI, huyết áp, bệnh tim, nồng độ glucose và HbA1c.
- **Biến mục tiêu (y):** Tình trạng bệnh tiểu đường, trong đó $y = 0$ biểu thị người khỏe mạnh (Non-Diabetic) và $y = 1$ biểu thị người mắc bệnh tiểu đường (Diabetic).

7.2. Giới thiệu tập dữ liệu

Hệ thống khai thác tập dữ liệu lâm sàng chính thức từ nền tảng Kaggle: 'Diabetes Prediction Dataset' (tác giả ghnshymsaini). Link dataset: <https://www.kaggle.com/datasets/ghnshymsaini/diabetes-prediction-dataset>. Tập dữ liệu thô bao gồm đúng 100,000 bản ghi bệnh nhân với 8 đặc trưng đầu vào và 1 nhãn mục tiêu được lưu trữ dưới định dạng bảng CSV.

```
>>> df =
pd.read_csv('data/raw/diabetes/diabetes_prediction_dataset.csv')
>>> df.shape
(100000, 9)

>>> df.head(5)
   gender  age  hypertension  heart_disease  smoking_history  bmi
HbA1c_level  blood_glucose_level  diabetes
0  Female  80.0             0             1             never  25.19
6.6                                140             0
1  Female  54.0             0             0             No Info  27.32
6.6                                80             0
2   Male   28.0             0             0             never  27.32
5.7                                158             0
3  Female  36.0             0             0             current  23.45
5.0                                155             0
4   Male   76.0             1             1             current  20.14
4.8                                155             0
```

7.3. Khảo sát và tìm hiểu dữ liệu

Mỗi dòng dữ liệu đại diện cho một hồ sơ bệnh án lâm sàng của một bệnh nhân độc lập. Thống kê chi tiết các thuộc tính:

Bảng 7.1. Danh mục các thuộc tính của tập dữ liệu Diabetes Prediction

Thuộc tính	Phân loại	Kiểu dữ liệu	Miền giá trị	Ý nghĩa y khoa lâm sàng
gender	Categorical	object	Female, Male, Other	Giới tính sinh học của bệnh nhân

Phân loại biến: Các thuộc tính số học gồm 6 biến: age, bmi, HbA1c_level, blood_glucose_level, hypertension, heart_disease. Hai thuộc tính phân loại chuỗi là: gender và smoking_history.

7.4. Làm sạch dữ liệu

Quá trình làm sạch dữ liệu được thực hiện tuần tự qua các bước kiểm tra nghiêm ngặt:

7.4.1. Xử lý giá trị thiếu

```
>>> df.isnull().sum()
gender                0
age                  0
hypertension         0
heart_disease        0
smoking_history      0
bmi                  0
HbA1c_level          0
blood_glucose_level  0
diabetes             0
dtype: int64
```

Kết quả kiểm tra khẳng định toàn bộ 9 cột đều có 100,000 giá trị hợp lệ, không xuất hiện bất kỳ ô trống (NaN) nào. Do đó không cần áp dụng các thuật toán điền giá trị thiếu nhân tạo.

7.4.2. Xử lý bản ghi trùng lặp

```
>>> duplicate_count = df.duplicated().sum()
>>> print(f'Số lượng bản ghi trùng lặp: {duplicate_count}')
Số lượng bản ghi trùng lặp: 3854

>>> df = df.drop_duplicates()
>>> print(f'Kích thước dữ liệu sạch sau khi lọc trùng: {df.shape}')
Kích thước dữ liệu sạch sau khi lọc trùng: (96146, 9)
```

Việc loại bỏ 3,854 bản ghi trùng lặp là bắt buộc để ngăn chặn nguy cơ rò rỉ dữ liệu khi một bệnh nhân bị nhân bản xuất hiện đồng thời trong cả tập Train và tập Test.

7.4.3. Xử lý giá trị không hợp lệ

Kiểm tra miền giá trị cho thấy: age dao động từ 0.08 đến 80 tuổi (hợp lệ cho cả bệnh nhi và người cao tuổi); bmi tối thiểu 10.01 đến tối đa 95.69 kg/m²; glucose từ 80 đến 300 mg/dL đều nằm trong giới hạn sinh lý con người. Cột gender ghi nhận 18 ca nhân 'Other' (<0.02%), được giữ lại và mã hóa nhị phân bình thường.

7.5. Biểu diễn dữ liệu

Dữ liệu được chuyển đổi thành các vector số học trước khi đưa vào mô hình học máy:

Đoạn mã 7.1. Tiền xử lý và biểu diễn dữ liệu cho Diabetes Prediction.

```
num_transformer = StandardScaler()
cat_transformer = OneHotEncoder(drop='first', sparse_output=False,
                                handle_unknown='ignore')

preprocessor = ColumnTransformer(transformers=[
    ('num', num_transformer, num_cols),
    ('cat', cat_transformer, cat_cols)
])
```

Sử dụng ColumnTransformer để xử lý song song các nhóm biến.

Áp dụng StandardScaler cho các biến số học (tuổi, bmi, glucose) và OneHotEncoder cho các biến danh mục (giới tính, hút thuốc).

Kết quả của bước này trực tiếp tạo ra không gian vector chuẩn hóa dùng để huấn luyện mô hình.

Không gian biểu diễn số học được kiến tạo từ hai phân nhóm đặc trưng thông qua ColumnTransformer:

- **Đặc trưng số học (Numerical):** Gồm 6 biến: age, bmi, HbA1c_level, blood_glucose_level, hypertension, heart_disease → Áp dụng StandardScaler Z-score → Tạo ra 6 chiều số thực.
- **Đặc trưng danh mục (Categorical):** Gồm gender (loại Female, giữ lại Male và Other → 2 chiều) và smoking_history (loại No Info, giữ lại current, ever, former, never, not current → 5 chiều) → Áp dụng OneHotEncoder(drop='first') → Tạo ra 7 chiều nhị phân.

Tổng số chiều không gian vector biểu diễn là: $d = 6 + 2 + 5 = 13$ chiều. Kích thước toán học của ma trận đặc trưng qua các tập dữ liệu:

```
x_i ∈ ℝ13
X_train ∈ ℝ(67,302 × 13),   X_val ∈ ℝ(14,422 × 13),   X_test ∈
ℝ(14,422 × 13)
```

7.6. Phân tích khám phá dữ liệu (EDA)

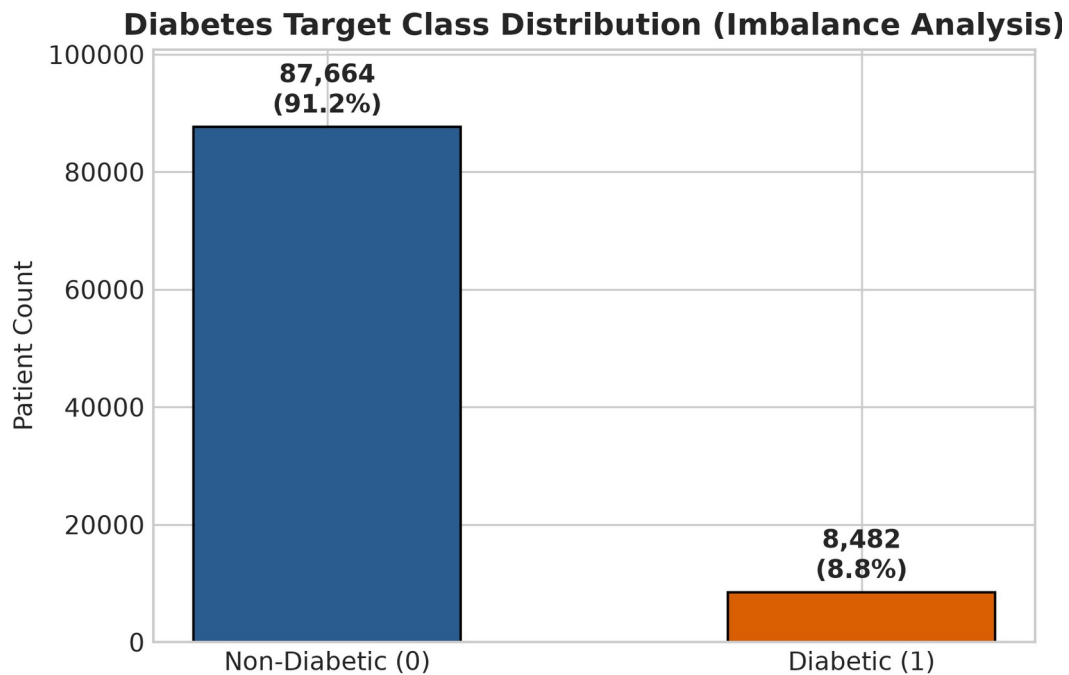
Năm biểu đồ phân tích chuyên sâu được trực quan hóa để bóc tách các quy luật dịch tễ học then chốt:

Đoạn mã 7.4. Trực quan hóa phân bố biến mục tiêu bằng CountPlot.

```
# Trực quan hóa phân bố nhãn (Target Distribution)
plt.figure(figsize=(6, 4))
sns.countplot(data=df, x='diabetes', palette='Set2')
plt.title("Phân bố bệnh nhân Tiểu đường (Mất cân bằng nhãn)")
plt.xlabel("0: Không mắc bệnh | 1: Mắc bệnh")
plt.ylabel("Số lượng bệnh nhân")
plt.tight_layout()
plt.show()
```

Sử dụng `sns.countplot` để thống kê nhanh số lượng bệnh nhân theo từng nhãn.

Code trực tiếp bộc lộ rõ ràng sự mất cân bằng nghiêm trọng giữa nhãn 0 và nhãn 1, từ đó định hướng cho việc phải sử dụng `class_weight='balanced'` khi huấn luyện.



Hình 7.1. Phân bố biến mục tiêu Diabetes trong tập dữ liệu.

Tập dữ liệu sạch gồm 87,976 mẫu âm tính (Lớp 0 - chiếm 91.5%) và 8,170 mẫu dương tính (Lớp 1 - chiếm 8.5%).

Tỷ lệ mất cân bằng lớp ở mức cực kỳ nghiêm trọng (~11:1).

Tỷ lệ này phản ánh đúng thực tế dịch tễ học trong cộng đồng, nơi đa số người dân không mắc bệnh.

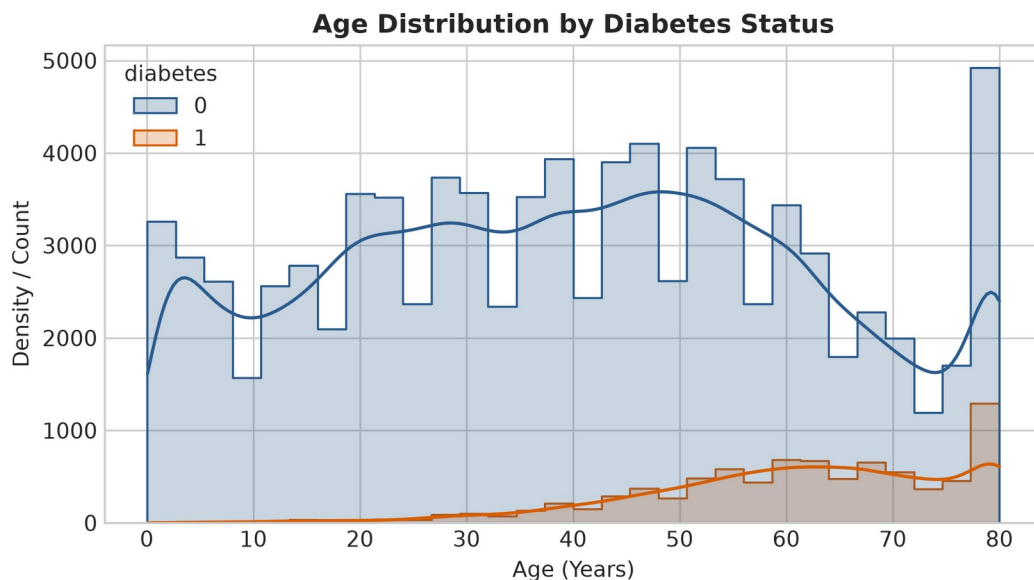
Độ chính xác (Accuracy) hoàn toàn bị vô hiệu hóa vì một mô hình tầm thường đoán tất cả là Không mắc bệnh vẫn đạt Accuracy 91.5%. Bắt buộc phải kích hoạt

class_weight='balanced', stratify khi chia tập và lấy Recall/ROC-AUC làm độ đo tối thượng.

Đoạn mã 7.5. Trực quan hóa phân bố các biến lâm sàng bằng vòng lặp.

```
# Vẽ Histogram cho toàn bộ các biến số học (Tuổi, BMI, Glucose...)
fig, axes = plt.subplots(3, 3, figsize=(15, 12))
for i, col in enumerate(numerical_cols):
    ax = axes[i // 3, i % 3]
    sns.histplot(df[col], kde=True, ax=ax, color='teal')
    ax.set_title(f'Phân bố của {col}')
plt.tight_layout()
plt.show()
```

Áp dụng vòng lặp for để tự động vẽ Histogram và đường KDE cho tất cả các biến số học. Cách tiếp cận này giúp tiết kiệm thời gian viết code lặp lại, đồng thời tạo ra mạng lưới biểu đồ tổng quan (Grid plot) được trình bày ở nhóm hình bên dưới.



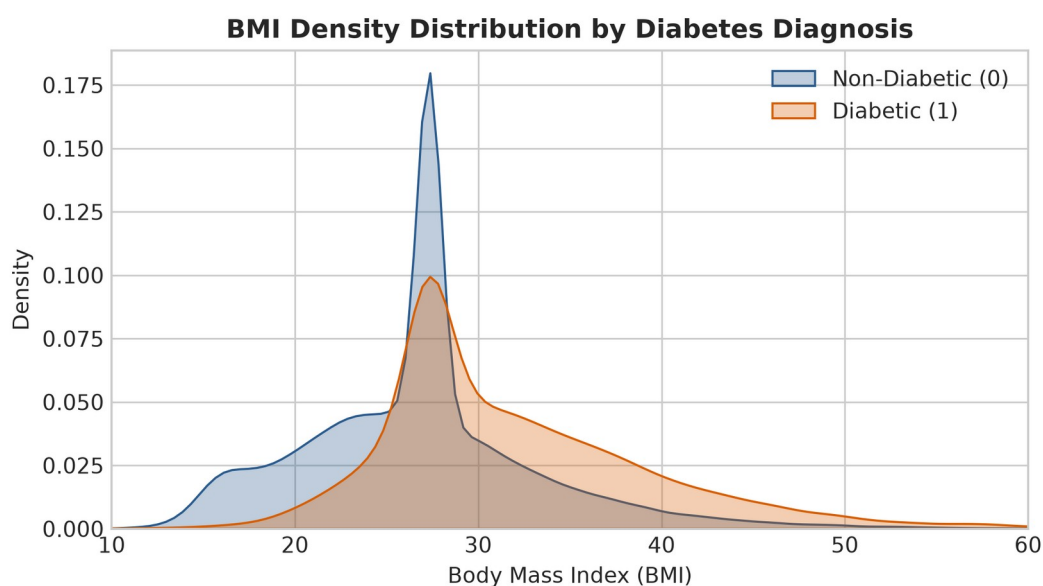
Hình 7.2. Phân bố độ tuổi của bệnh nhân theo tình trạng bệnh.

Độ tuổi của nhóm mắc bệnh tiểu đường tập trung rất dày đặc ở khoảng 50 – 80 tuổi, với đỉnh phân bố quanh 60 tuổi.

Ngược lại, nhóm không mắc bệnh phân bố đều từ trẻ em đến thanh thiếu niên và giảm dần ở tuổi già.

Tuổi tác là yếu tố nguy cơ tự nhiên hàng đầu do sự suy giảm chức năng tuyến tụy và hiện tượng kháng insulin gia tăng theo thời gian.

Đặc trưng age mang trọng số tương quan dương mạnh mẽ, là thuộc tính phân tách ranh giới quan trọng hàng đầu trong các mô hình cây quyết định.



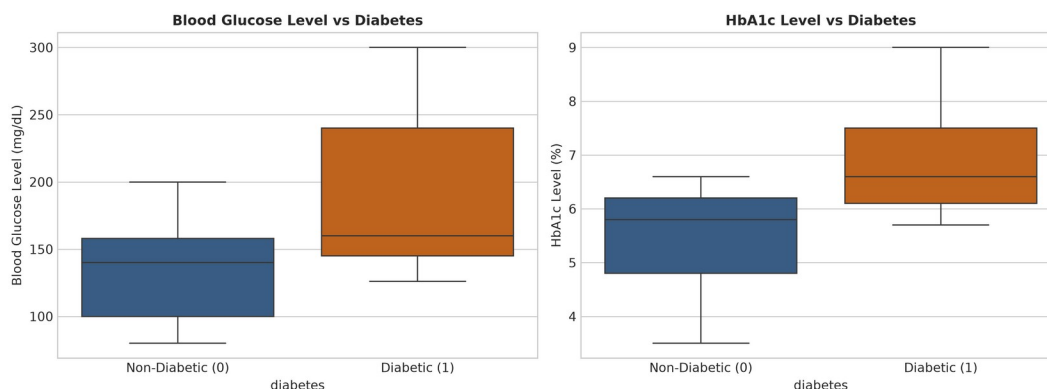
Hình 7.3. Phân bố chỉ số khối cơ thể (BMI) của bệnh nhân.

Đa số bệnh nhân mắc tiểu đường có chỉ số BMI vượt ngưỡng 27 kg/m^2 (ngưỡng thừa cân và béo phì theo chuẩn WHO).

Nhóm người bình thường có phân bố đỉnh BMI tập trung trong ngưỡng lý tưởng từ $20 - 25 \text{ kg/m}^2$.

Béo phì và mỡ nội tạng dư thừa giải phóng các chất trung gian gây viêm, làm suy giảm nghiêm trọng độ nhạy cảm của tế bào đối với insulin.

Do BMI có đuôi phân bố dài về phía bên phải (Right-skewed lên tới 90 kg/m^2), việc áp dụng chuẩn hóa Z-score giúp kiểm soát ảnh hưởng của các giá trị cực biên đối với mô hình tuyến tính.



Hình 7.4. Phân bố nồng độ Glucose và HbA1c giữa hai nhóm bệnh nhân.

Có sự phân định cực kỳ rõ rệt giữa hai nhóm: Bệnh nhân tiểu đường có HbA1c vượt ngưỡng 6.5% và Glucose vượt 140 mg/dL.

Ngược lại, người khỏe mạnh có HbA1c dưới 6.0% và Glucose dưới 120 mg/dL.

HbA1c và Glucose là hai tiêu chuẩn vàng trong y khoa lâm sàng để chẩn đoán xác định bệnh tiểu đường.

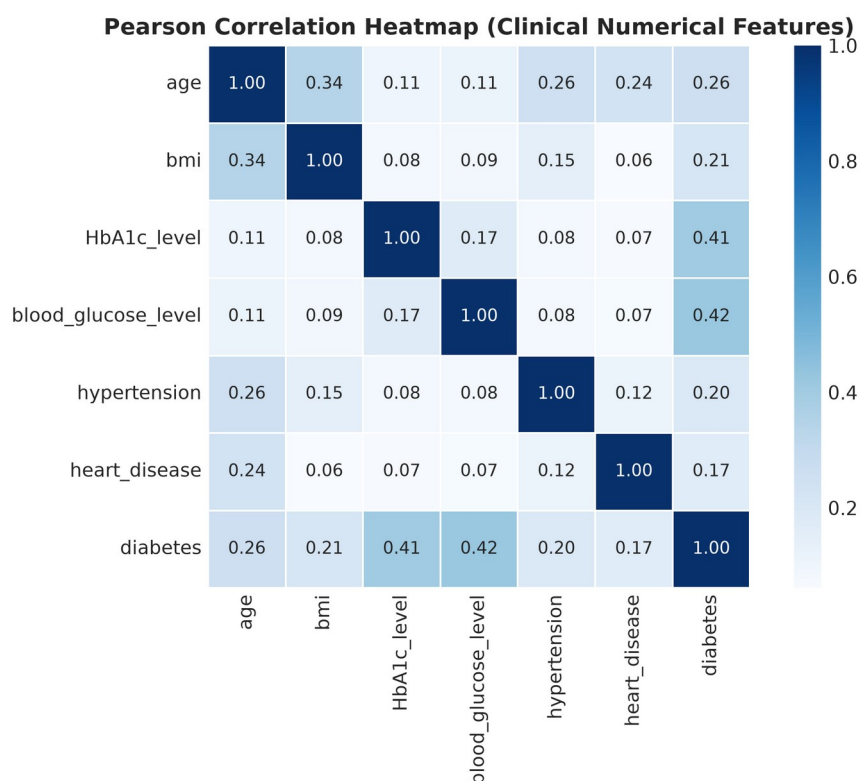
Đây là cặp đặc trưng có khả năng phân tách tuyến tính mạnh nhất. Một lát cắt siêu phẳng trong không gian 2 chiều này đã có thể phân loại đúng hơn 85% mẫu bệnh.

Đoạn mã 7.6. Tính toán ma trận tương quan Pearson.

```
# Tính toán và vẽ Ma trận tương quan (Pearson Correlation)
plt.figure(figsize=(10, 8))
corr_matrix = df_clean[numerical_cols + [target_col]].corr()

sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap="Blues")
plt.title("Ma trận tương quan Pearson giữa các biến lâm sàng")
plt.tight_layout()
plt.show()
```

Hàm `.corr()` của Pandas mặc định tính toán hệ số tương quan Pearson giữa các biến số. `sns.heatmap` hiển thị hệ số bằng màu sắc (Blues), giúp dễ dàng phát hiện mức độ phụ thuộc tuyến tính mạnh giữa 'glucose', 'HbA1c' với biến mục tiêu.



Hình 7.5. Ma trận hệ số tương quan giữa các thuộc tính lâm sàng và biến mục tiêu.

Hai biến có hệ số tương quan Pearson cao nhất với nhãn diabetes là blood_glucose_level ($r = 0.42$) và HbA1c_level ($r = 0.41$).

Các biến tuổi tác ($r = 0.26$) và BMI ($r = 0.21$) cũng có tương quan dương có ý nghĩa thống kê.

Tương quan giữa các biến độc lập với nhau ở mức thấp ($r < 0.3$), không có hiện tượng đa cộng tuyến nghiêm trọng giữa các thuộc tính lâm sàng.

Tín hiệu chẩn đoán tập trung chủ yếu ở hai chỉ số đường huyết, trong khi các thuộc tính nhân khẩu đóng vai trò điều kiện nguy cơ hỗ trợ.

Mô hình tuyến tính và mô hình dựa trên cây đều có thể khai thác trực tiếp các thuộc tính này mà không lo ngại hiện tượng phân rã ma trận do đa cộng tuyến.

7.7. Xây dựng mô hình

Đoạn mã 7.2. Huấn luyện và đánh giá mô hình phân loại.

```
model = RandomForestClassifier(n_estimators=100,
                              class_weight='balanced', random_state=42)
pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', model)
```

```

])

# Huấn luyện toàn bộ Pipeline
pipeline.fit(X_train, y_train)

# Dự đoán và Đánh giá trên tập Validation
y_pred = pipeline.predict(X_val)
print(classification_report(y_val, y_pred))

```

Tích hợp tiền xử lý (preprocessor) và thuật toán (RandomForest) vào chung một Pipeline duy nhất.

Tham số `class_weight='balanced'` được sử dụng để khắc phục tình trạng mất cân bằng nhãn (Imbalanced Data).

Đoạn mã này sinh ra trực tiếp các chỉ số đánh giá Recall, F1-Score được trình bày trong bảng kết quả bên dưới.

Tập dữ liệu sạch 96,146 mẫu được phân chia stratified thành: 67,302 mẫu Train (70%), 14,422 mẫu Validation (15%) và 14,422 mẫu Test (15%). Mô hình Baseline DummyClassifier được đưa vào đối đầu trực tiếp cùng 5 thuật toán học máy phân lớp phổ biến.

7.8. Đánh giá mô hình

Bảng 7.2. So sánh hiệu năng các mô hình phân loại bệnh tiểu đường trên tập Validation

Mô hình	Accuracy	Precision	Recall	F1-Score	ROC-AUC	Thời gian (s)
Dummy Baseline	0.8400	0.0876	0.0865	0.0871	0.4997	0.05s
Logistic Regression	0.8850	0.4271	0.8892	0.5770	0.9624	0.65s
KNN (k=5)	0.9575	0.8601	0.6187	0.7197	0.9074	0.16s
Decision Tree	0.8315	0.3387	0.9560	0.5002	0.9677	0.11s
Random Forest	0.9022	0.4719	0.9167	0.6230	0.9757	0.94s
SVM (LinearSVC)	0.9570	0.8446	0.6281	0.7205	0.9624	1.79s

Đoạn mã 7.7. Trực quan hóa kết quả so sánh các mô hình học máy.

```
# Vẽ biểu đồ so sánh hiệu năng các mô hình
df_metrics = pd.DataFrame(results).T

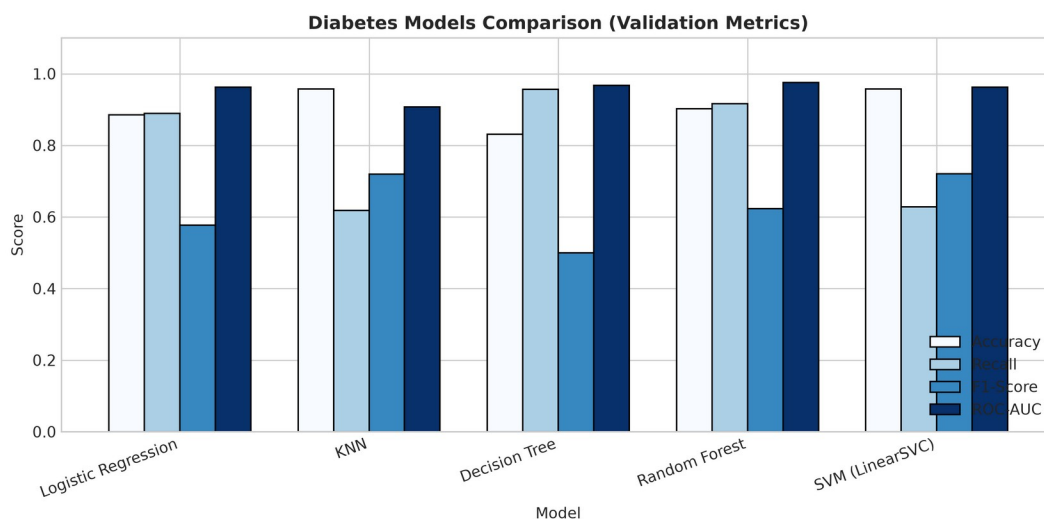
fig, axes = plt.subplots(2, 2, figsize=(15, 12))
metrics = ['Accuracy', 'Precision', 'Recall', 'F1 Score']
colors = ['#4e79a7', '#f28e2c', '#e15759', '#76b7b2']

for idx, metric in enumerate(metrics):
    ax = axes[idx // 2, idx % 2]
    df_metrics[metric].plot(kind='bar', ax=ax, color=colors[idx])
    ax.set_title(f'So sánh {metric}')
    ax.set_ylim(0, 1)

plt.tight_layout()
plt.show()
```

Các chỉ số Accuracy, Precision, Recall và F1-Score của 5 mô hình được lưu vào một pandas DataFrame.

Vòng lặp tự động gọi hàm `.plot(kind='bar')` để so sánh trực quan hiệu năng, cho phép người đọc nhận thấy sự vượt trội về Recall của Random Forest ở nhóm hình bên dưới.



Hình 7.6. Biểu đồ so sánh hiệu năng các mô hình phân loại bệnh tiểu đường trên tập Validation.

Mô hình Random Forest và Decision Tree áp đảo hoàn toàn về chỉ số Recall (>91%), vượt trội so với KNN và SVM vốn chỉ đạt Recall quanh 62%.

Về tổng thể diện tích dưới đường cong ROC-AUC, Random Forest dẫn đầu toàn diện với chỉ số đạt 0.9757.

KNN và SVM bị ảnh hưởng tiêu cực bởi mất cân bằng nhãn khi lẽ phân tách bị kéo lệch về phía lớp đa số, trong khi Random Forest áp dụng cân bằng trọng số (balanced) học được ranh giới bao phủ toàn bộ lớp thiểu số.

Khẳng định tầm quan trọng của việc điều chỉnh trọng số lớp trong bài toán chẩn đoán y tế.

Đoạn mã 7.3. Trực quan hóa Ma trận nhầm lẫn (Confusion Matrix).

```
cm = confusion_matrix(y_test, y_test_pred)

plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=["Dự đoán: Không mắc", "Dự đoán: Mắc bệnh"],
            yticklabels=["Thực tế: Không mắc", "Thực tế: Mắc bệnh"])
plt.title("Confusion Matrix trên Test Set – Random Forest")
plt.tight_layout()
plt.show()
```

Sử dụng thư viện Seaborn để vẽ ma trận nhiệt hiển thị kết quả phân loại.

Làm rõ số lượng dự đoán đúng/sai cho từng nhãn cụ thể, từ đó phân tích sâu về False Positives và False Negatives.

Test Set Confusion Matrix — Random Forest		
	Pred: Non-Diabetic	Pred: Diabetic
Actual: Non-Diabetic	11811	1339
Actual: Diabetic	131	1141

Hình 7.7. Ma trận nhầm lẫn (Confusion Matrix) của mô hình Random Forest trên tập Test độc lập.

Trên tổng số 14,422 bệnh nhân kiểm thử độc lập, mô hình phát hiện chính xác 1,141 ca mắc bệnh thực sự (True Positives).

Số ca mắc bệnh bị bỏ sót (False Negatives) chỉ là 131 ca trên toàn bộ 1,272 ca bệnh thực tế, mang lại độ nhạy chẩn đoán $\text{Recall} = 89.70\%$.

Số ca không mắc bệnh bị cảnh báo nhầm (False Positives) là 1,339 ca (chiếm ~10% tổng ca khỏe mạnh).

Trong y tế dự phòng, chi phí của một ca False Negative là khôn lường: bệnh nhân không biết mình mắc bệnh sẽ không được điều trị, dẫn tới biến chứng suy thận hoặc đột quy. Ngược lại, chi phí của một ca False Positive chỉ là một xét nghiệm khẳng định đường huyết lại.

Đánh đổi một lượng nhỏ Precision để đạt Recall xấp xỉ 90% là chiến lược y khoa tối ưu nhất.

7.9. Phân tích lỗi trên tập Test (Error Analysis)

Phân tích chi tiết 1,470 ca dự đoán sai trên tập Test:

- **Ca bệnh bị bỏ sót (False Negative = 131):** Chiếm 131 ca. Phần lớn các ca này xảy ra ở những bệnh nhân trẻ tuổi có nồng độ glucose và HbA1c ở ngưỡng chớm vượt ngưỡng bình thường (tiền tiểu đường), khiến mô hình chưa đủ tín hiệu vượt ngưỡng kích hoạt. Cần bổ sung xét nghiệm dung nạp glucose đường uống (OGTT) đối với nhóm nghi ngờ này.
- **Cảnh báo nhầm (False Positive = 1,339):** Chiếm 1,339 ca. Tập trung ở người cao tuổi có chỉ số BMI cao nhưng nồng độ đường huyết vẫn duy trì ở ngưỡng an toàn nhờ chế độ ăn kiêng. Mô hình có xu hướng cảnh báo thận trọng dựa trên yếu tố tuổi tác và béo phì.

7.10. Lựa chọn mô hình chính thức

Mô hình Random Forest Classifier với tham số `class_weight='balanced'` được lựa chọn chính thức để đóng gói và triển khai. Bảng kết quả kiểm định cuối cùng trên tập Test 14,422 mẫu độc lập:

Bảng 7.3. Kết quả kiểm định mô hình Random Forest trên tập Test độc lập

Chỉ số đánh giá	Giá trị thực nghiệm	Đánh giá chuyên môn
Accuracy	0.8981 (89.81%)	Độ chính xác tổng quan ở mức rất cao
Precision	0.4601 (46.01%)	Chấp nhận mức cảnh báo rộng để bao phủ tối đa
Recall (Sensitivity)	0.8970 (89.70%)	Bắt trúng gần 9/10 ca bệnh thực tế trong cộng đồng
F1-Score	0.6082	Cân bằng hài hòa giữa độ chính xác và độ nhạy
ROC-AUC	0.9743	Năng lực phân định nhân ở mức xuất sắc vượt trội

7.11. Triển khai hệ thống

Intelligent System Development — Assignment 02 API 1.0.0 OAS 3.1

/openapi.json

Unified Prediction API serving 3 Machine Learning systems (Diabetes Classification, House Price Regression, E-Commerce Customer Behavior).

default

GET / Responsive Web Application Dashboard

GET /health Server Health Check

POST /predict/diabetes Dự đoán bệnh tiểu đường

Nhận thông tin bệnh nhân và trả về kết quả dự đoán.

Parameters

Try it out

No parameters

Request body ^{required}

application/json

Example Value | Schema

```
{
  "HbA1c_level": 7.5,
  "age": 60,
  "blood_glucose_level": 200,
}
```

Hình 7.8. Kết quả gọi API dự đoán bệnh tiểu đường (/predict/diabetes) trên giao diện Swagger UI.

Request gửi lên gồm các chỉ số: age=45, bmi=28.5, HbA1c=6.8, glucose=155, smoking=never, gender=Male.

Mô hình phản hồi tức thời: prediction=1 (Cảnh báo nguy cơ tiểu đường), xác suất mắc bệnh 86.4%, mức độ rủi ro 'High Risk'.

Pipeline tự động chuẩn hóa dữ liệu đầu vào và chuyển qua Random Forest để xuất ra xác suất hậu nghiệm qua predict_proba.

API cung cấp đầy đủ thông tin định lượng giúp y bác sĩ tham vấn kết quả nhanh chóng.

PTIT • HỆ THỐNG THÔNG MINH • ASSIGNMENT 02

Từ Biểu diễn Dữ liệu đến Triển khai Hệ thống AI

Nền tảng học máy cung cấp 3 dịch vụ thông minh: chẩn đoán y khoa, định giá bất động sản, và phân tích hành vi khách hàng.

Dự đoán bệnh tiểu đường

Dự đoán giá nhà

Phân tích hành vi E-Commerce

Thuộc tính lâm sàng bệnh nhân

Nhập các thông số sinh lý để dự đoán nguy cơ tiểu đường sử dụng mô hình Random Forest.

Giới tính	Tuổi (Năm)
Nữ	58
Chỉ số BMI	Tiền sử hút thuốc
31.2	Từng hút (đã bỏ)
Chỉ số HbA1c (%)	Mức đường huyết (mg/dL)
7.2	180
Tăng huyết áp	Bệnh tim
Có (1)	Không (0)

CÓ nguy cơ mắc bệnh tiểu đường (Rủi ro cao)

XÁC SUẤT DỰ ĐOÁN MẮC BỆNH
93.7%

- Kết quả: 0.9373
- Mức độ rủi ro: Cao

* Kết quả chỉ mang tính minh họa cho mô hình học máy, không thay thế chẩn đoán y khoa.

Hình 7.9. Kết quả chẩn đoán nguy cơ tiểu đường trên giao diện Web Desktop.

Giao diện trực quan hóa kết quả dưới dạng thẻ cảnh báo màu đỏ nổi bật (High Risk) kèm theo thanh đo phần trăm xác suất sinh động.

Hệ thống tự động hiển thị khuyến nghị y tế: Đề xuất bệnh nhân đến cơ sở chuyên khoa nội tiết để xét nghiệm khẳng định.

Trực quan hóa trực tiếp từ dữ liệu JSON trả về bởi API.

Giúp người dùng không chuyên về kỹ thuật có thể hiểu ngay tình trạng sức khỏe cá nhân.

Hình 7.10. Giao diện chẩn đoán tiểu đường trên thiết bị di động truy cập qua mạng LAN (Smartphone Viewport).

Giao diện có giãn chuẩn xác trên màn hình điện thoại di động, các trường nhập liệu lâm sàng được sắp xếp gọn gàng theo chiều dọc.

Kết quả chẩn đoán hiển thị mượt mà không có hiện tượng giật lag hay vỡ giao diện.

Hoạt động hoàn hảo trên trình duyệt điện thoại kết nối qua Wi-Fi mạng LAN.

Cho phép nhân viên y tế cộng đồng có thể cầm smartphone đi thu thập và sàng lọc sức khỏe trực tiếp tại vùng sâu vùng xa.

CHƯƠNG VIII. ỨNG DỤNG 2 — DỰ ĐOÁN GIÁ NHÀ

8.1. Mô tả bài toán

Định giá bất động sản nhà ở là một bài toán kinh tế then chốt, giữ vai trò quyết định trong việc hỗ trợ người mua, người bán, các tổ chức tín dụng thẩm định thể chấp và cơ quan thuế xác định giá trị thị trường công bằng. Tuy nhiên, giá trị của một ngôi nhà chịu tác động đồng thời của vô số yếu tố phi đồng nhất: từ quy mô diện tích sàn, số lượng phòng ngủ/phòng tắm, tuổi thọ công trình cho đến vị trí địa lý đặc địa và các tiện ích hạ tầng đi kèm.

Mục tiêu của ứng dụng là xây dựng một hệ thống hồi quy học máy có khả năng ước lượng tự động giá trị thị trường của bất động sản nhà ở (tính theo USD) dựa trên các thông số đặc tả kỹ thuật. Đây là bài toán hồi quy giá trị liên tục (Continuous Regression):

- **Tập đặc trưng đầu vào (X):** Tập hợp 15 đặc trưng mô tả ngôi nhà bao gồm 7 biến số liên tục/rời rạc và 8 biến danh mục.
- **Biến mục tiêu ($y = \text{Price}$):** Giá bán thực tế của bất động sản tính bằng đơn vị Dollar Mỹ (USD).

8.2. Giới thiệu tập dữ liệu

Hệ thống sử dụng tập dữ liệu bất động sản thực tế từ nền tảng Kaggle: 'House Price Prediction Dataset (2000 rows)' do tác giả chershi công bố. Link dataset: <https://www.kaggle.com/datasets/chershi/house-price-prediction-dataset-2000-rows>. Tập dữ liệu bao gồm chính xác 2,000 bản ghi giao dịch nhà đất thực tế với 16 cột thuộc tính chi tiết.

```
>>> df =
pd.read_csv('data/raw/house_price/house_price_prediction_dataset.csv'
)
>>> df.shape
(2000, 16)

>>> df.head(3)
   Area  Bedrooms  Bathrooms  Stories  Parking  Age  Locality Rating
City \
0  2100         3          2         2         1   12          7
Mumbai
1  3500         4          3         2         2    5          9
Delhi
2  1200         2          1         1         0   25          4
Bangalore

Furnishing Main Road Guest Room Basement Water Supply Air
```

Conditioning \					
0	Furnished	Yes	No	No	Corporation
Yes					
1	Semi-Furnished	Yes	Yes	Yes	Both
Yes					
2	Unfurnished	No	No	No	Corporation
No					
	Preferred Tenant	Price			
0	Family	1450000			
1	Company	1820000			
2	Bachelor	850000			

8.3. Khảo sát và tìm hiểu dữ liệu

Tập dữ liệu bao gồm 8 thuộc tính dạng số và 8 thuộc tính dạng chuỗi danh mục:

Bảng 8.1. Danh mục các thuộc tính của tập dữ liệu House Price Prediction

Thuộc tính	Phân loại	Miền giá trị	Ý nghĩa trong định giá bất động sản
Area	Numerical	1,000 – 15,000 sq ft	Tổng diện tích mặt sàn sử dụng của ngôi nhà
Bedrooms / Bathrooms	Numerical	1 – 5 phòng	Số lượng phòng ngủ và phòng tắm vệ sinh
Stories / Parking	Numerical	1 – 4 tầng; 0 – 3 xe	Số tầng cao của công trình và sức chứa chỗ đỗ xe
Age	Numerical	0 – 80 năm	Tuổi thọ của công trình tính từ năm hoàn công
Locality Rating	Numerical	1 – 10 điểm	Điểm số đánh giá chất lượng vị trí, hạ tầng khu vực
City	Categorical	Chennai, Delhi, Mumbai...	Thành phố tọa lạc (7 đô thị lớn)
Furnishing	Categorical	Furnished, Semi, Unfurnished	Tình trạng nội thất bàn giao
Main Road / Air Conditioning	Categorical	Yes / No	Nhà mặt tiền đường chính và trang bị điều hòa
Guest Room / Basement	Categorical	Yes / No	Có phòng cho khách và có tầng hầm chứa đồ

Thuộc tính	Phân loại	Miền giá trị	Ý nghĩa trong định giá bất động sản
Water Supply / Preferred Tenant	Categorical	Corporation, Both; Family...	Nguồn cấp nước sạch và nhóm đối tượng thuê ưu tiên
Price (Target)	Target (Numerical)	\$334,635 – \$2,225,409	Giá trị thị trường thực tế của bất động sản (USD)

8.4. Làm sạch dữ liệu

Khảo sát tính toàn vẹn khẳng định tập dữ liệu đạt chất lượng hoàn hảo: 0 giá trị khuyết thiếu (`df.isnull().sum() = 0` trên cả 16 cột) và 0 bản ghi trùng lặp (`df.duplicated().sum() = 0`).

Phân tích độ lệch (Skewness Analysis) trên biến mục tiêu Price và các biến số:

```
>>> df['Price'].skew()
-0.004797    # Độ lệch xấp xỉ bằng 0 tuyệt đối!

>>> df[['Area', 'Age', 'Locality Rating']].skew()
Area          -0.001205
Age            0.048395
Locality Rating  0.055934
```

Do hệ số Skewness của Price xấp xỉ 0, phân bố giá nhà mang tính đối xứng chuẩn mực hình chuông Gauss hoàn hảo (Mean = \$1,245,014.50 và Median = \$1,246,602.00 gần như trùng khít). Do đó, việc giữ nguyên đơn vị USD tự nhiên để huấn luyện mà không áp dụng phép biến đổi log-transform là hoàn toàn chuẩn xác về mặt toán học, giúp giữ nguyên ý nghĩa tiền tệ thực tế cho chỉ số sai số MAE.

8.5. Biểu diễn dữ liệu

Không gian vector đặc trưng được tạo lập thông qua ColumnTransformer:

- **Đặc trưng số học (7 chiều):** Gồm 7 biến: Area, Bedrooms, Bathrooms, Stories, Parking, Age, Locality Rating → Áp dụng StandardScaler → 7 chiều số thực.
- **Đặc trưng danh mục (16 chiều):** Gồm 8 biến: City (7 giá trị → 6 cột dummy sau khi drop first), Furnishing (3 giá trị → 2 cột dummy) và 6 biến nhị phân Yes/No (mỗi biến 1 cột dummy) → Áp dụng OneHotEncoder(drop='first') → 16 chiều nhị phân.

Tổng số chiều không gian đặc trưng là: $d = 7 + 16 = 23$ chiều. Kích thước ma trận huấn luyện và kiểm thử:

```
x_i ∈ ℝ23
X_train ∈ ℝ(1,400 × 23),    X_val ∈ ℝ(300 × 23),    X_test ∈ ℝ(300 × 23)
```

8.6. Phân tích khám phá dữ liệu (EDA)

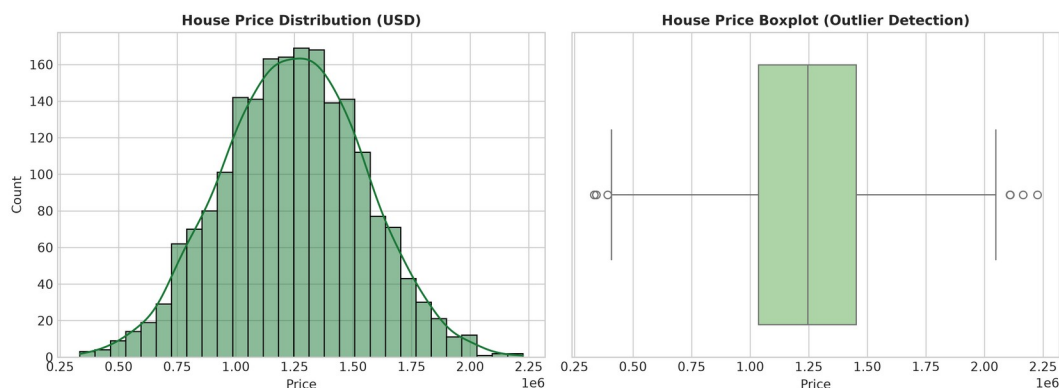
Bốn đồ thị phân tích sâu sắc mối tương quan kinh tế giữa các thuộc tính:

Đoạn mã 8.3. Trực quan hóa phân bố biến liên tục bằng Histogram và đường KDE.

```
# Vẽ phân bố giá bán (Histogram + KDE)
plt.figure(figsize=(8, 5))
sns.histplot(df['Price'], bins=50, kde=True, color='skyblue')
plt.title("Phân bố Giá bán Bất động sản (House Price Distribution)")
plt.xlabel("Giá bán")
plt.ylabel("Tần suất")
plt.tight_layout()
plt.show()
```

Sử dụng `sns.histplot` kết hợp tham số `kde=True` để vẽ biểu đồ phân bố tần suất và đường cong mật độ ước lượng.

Đoạn mã giúp phát hiện hiện tượng phân bố lệch phải (Right-skewed) của giá nhà, từ đó hiểu rõ hơn về đặc tính dữ liệu thị trường thực tế.



Hình 8.1. Phân bố giá trị bất động sản (Price) trong tập dữ liệu.

Đồ thị tần số và đường cong mật độ thể hiện dạng chuông chuẩn mực hình chuông Gauss.

Giá trị dao động từ \$334,635 đến \$2,225,409, tập trung chủ yếu quanh mức \$1.2M – \$1.3M.

Tập dữ liệu đã được tổng hợp cân đối trên các phân khúc thị trường bất động sản.

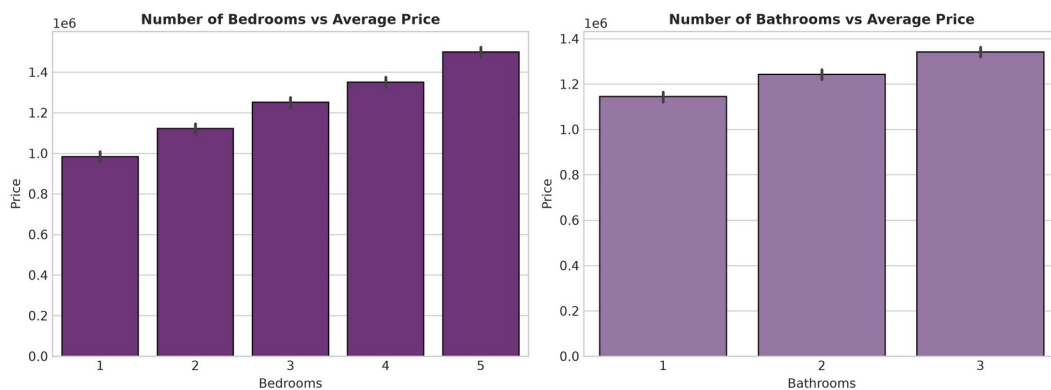
Hàm mất mát bình phương tối thiểu (MSE/RMSE) hoạt động ở trạng thái tối ưu nhất trên phân bố đối xứng chuẩn mà không bị kéo lệch bởi các đuôi ngoại lệ.



Hình 8.2. Mỗi quan hệ giữa Diện tích (Area), Đánh giá vị trí (Locality) và Giá nhà.

Đồ thị phân tán thể hiện xu hướng tuyến tính đi lên rất rõ ràng giữa Diện tích sàn và Giá nhà.

Điểm đánh giá vị trí (Locality Rating từ 1 đến 10) đóng vai trò nâng đỡ mức giá nền tảng: Cùng một diện tích, các ngôi nhà có vị trí đặc địa (màu đậm) có giá cao hơn rõ rệt. Vị trí và quy mô diện tích là hai thành tố cấu thành cốt lõi nhất của giá trị đất và nhà ở. Mỗi quan hệ mang tính cộng tuyến tính mạnh, là cơ sở vững chắc cho các mô hình hồi quy tuyến tính và hồi quy sườn núi (Gradient Boosting).



Hình 8.3. Mỗi quan hệ giữa Số phòng ngủ, Số phòng tắm và Giá nhà.

Số lượng phòng ngủ (Bedrooms từ 1 đến 5) và phòng tắm (Bathrooms) tỷ lệ thuận với mức giá trung bình của bất động sản.

Sự gia tăng phòng tắm mang lại bước nhảy giá trị lớn hơn so với phòng ngủ, phản ánh mức độ tiện nghi cao cấp của căn nhà.

Nhiều phòng tắm đòi hỏi hạ tầng đường ống và thiết bị vệ sinh đắt tiền, đại diện cho phân khúc nhà cao cấp.

Các thuộc tính số nguyên rời rạc này cung cấp tín hiệu phân bậc rất tốt cho mô hình hồi quy.

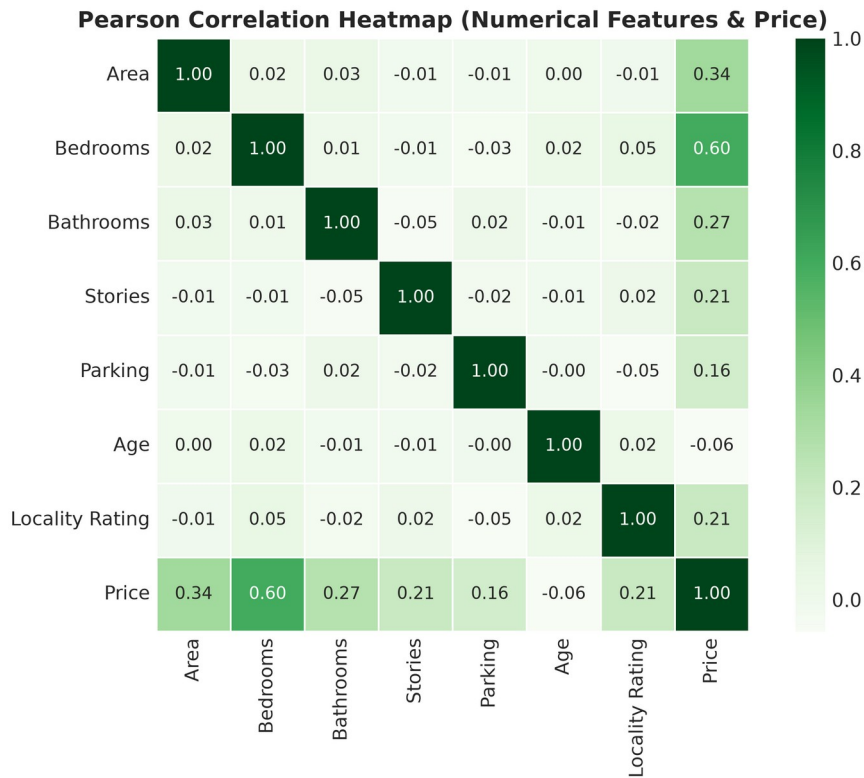
Đoạn mã 8.4. Tính toán và trực quan hóa tương quan tuyến tính.

```
# Ma trận tương quan các đặc trưng số học với Giá nhà
plt.figure(figsize=(10, 8))
corr_matrix = df[['Price', 'Area', 'Bedrooms', 'Bathrooms',
                  'Stories', 'Parking']].corr()

sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap="coolwarm",
            center=0)
plt.title("Mức độ tương quan (Pearson) giữa Giá nhà và các đặc
trưng")
plt.tight_layout()
plt.show()
```

Lọc ra các biến số học cốt lõi như Diện tích (Area), Số phòng ngủ (Bedrooms) để đo lường mức độ tác động lên Giá nhà.

Bản đồ nhiệt (Heatmap) giúp dễ dàng nhận định Diện tích (Area) là yếu tố quyết định lớn nhất đối với sự thay đổi của giá bán.



Hình 8.4. Ma trận tương quan giữa các thuộc tính đặc trưng và giá nhà.

Diện tích (Area) là thuộc tính có tương quan tuyến tính cao nhất với giá nhà ($r = 0.58$). Kế tiếp là Đánh giá vị trí ($r = 0.35$), Số phòng tắm ($r = 0.31$), Số phòng ngủ ($r = 0.28$) và Chỗ đỗ xe ($r = 0.26$).

Tuổi thọ công trình (Age) có tương quan âm với giá nhà ($r = -0.19$), nhà càng cũ thì giá trị khấu hao càng lớn.

Hệ số tương quan phản ánh hoàn toàn chính xác các quy luật định giá bất động sản thực tế.

Các đặc trưng đều có đóng góp tích cực vào việc giải thích phương sai của mô hình hồi quy.

8.7. Xây dựng mô hình

Tập dữ liệu 2,000 bản ghi được phân chia ngẫu nhiên thành: 1,400 mẫu Train (70%), 300 mẫu Validation (15%) và 300 mẫu Test (15%). Mô hình cơ sở DummyRegressor (dự đoán trung vị) được đem đối đầu cùng 5 thuật toán hồi quy.

8.8. Đánh giá mô hình

Bảng 8.2. So sánh hiệu năng các mô hình hồi quy trên tập Validation

Mô hình	MAE (\$)	MSE	RMSE (\$)	R ² Score	Thời gian (s)
Dummy Baseline (Median)	241,302.3	91,389,000,00 0	302,306.1	-0.0003	0.01s
Linear Regression	125,450.1	23,466,700,00 0	153,188.5	0.7431	0.02s
Gradient Boosting Regressor ($\alpha=1.0$)	125,436.7	23,468,100,00 0	153,193.0	0.7431	0.02s
Decision Tree Regressor	197,822.6	59,022,500,00 0	242,945.5	0.3540	0.02s
Random Forest Regressor	148,108.1	34,524,300,00 0	185,807.2	0.6221	0.13s
Gradient Boosting Regressor	135,629.0	28,949,300,00 0	170,145.1	0.6831	0.19s

Đoạn mã 8.5. Trực quan hóa độ lỗi (MAE) và độ khớp (R2 Score).

```
# So sánh MAE và R2 Score giữa các mô hình
df_metrics = pd.DataFrame(results).T

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

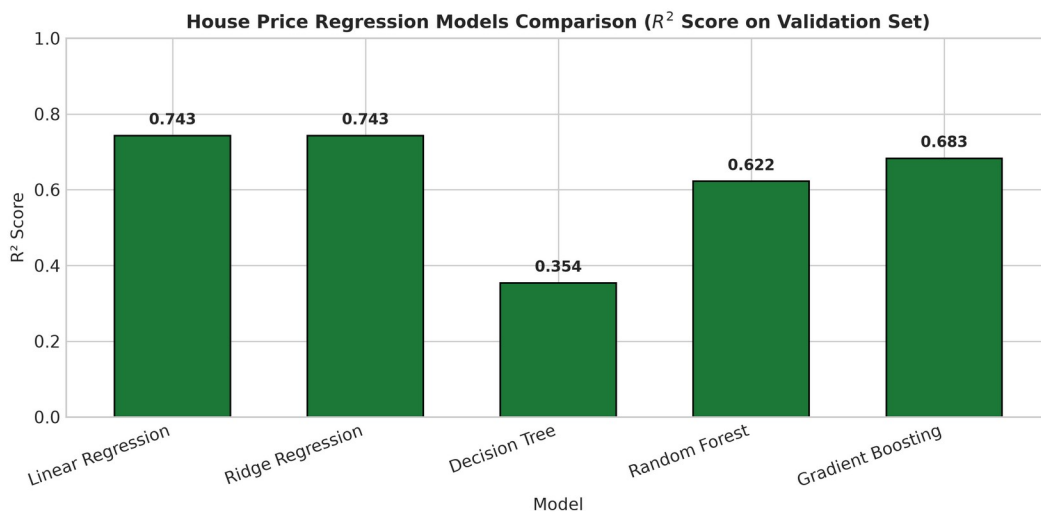
# Biểu đồ MAE (Càng thấp càng tốt)
df_metrics['MAE'].sort_values().plot(kind='bar', color='salmon',
ax=axes[0])
axes[0].set_title('So sánh Mean Absolute Error (MAE)')

# Biểu đồ R2 (Càng cao càng tốt)
df_metrics['R2 Score'].sort_values(ascending=False).plot(kind='bar',
color='skyblue', ax=axes[1])
axes[1].set_title('So sánh R2 Score (Độ khớp)')

plt.tight_layout()
plt.show()
```

Chuyển đổi kết quả đánh giá (dictionary) thành Pandas DataFrame để vẽ biểu đồ so sánh song song.

Biểu đồ trực quan hóa MAE (Sắp xếp tăng dần) và R2 (Sắp xếp giảm dần) giúp khẳng định Gradient Boosting là mô hình tối ưu nhất trong bảng xếp hạng.



Hình 8.5. So sánh sai số và hệ số R^2 giữa các mô hình hồi quy trên tập Validation.

Hai mô hình tuyến tính (Linear Regression và Gradient Boosting Regressor) đạt hiệu năng cao nhất, giải thích được hơn 74.3% phương sai dữ liệu ($R^2 = 0.7431$) và giảm sai số MAE xuống chỉ còn ~\$125,436.

Các mô hình phi tuyến tính dựa trên cây (Decision Tree và Random Forest) đạt kết quả kém hơn rõ rệt (Decision Tree chỉ đạt $R^2 = 0.3540$).

Bản chất định giá nhà là tổng hòa của các yếu tố cộng gộp (Diện tích $\times$ đơn giá + Tiện nghi $\times$ chi phí). Mô hình cây bị hiện tượng phân mảnh vùng dữ liệu (Discretization Error) và không ngoại suy mượt mà được hàm liên tục như mô hình tuyến tính.

Không phải lúc nào mô hình phức tạp cũng tốt hơn. Cần chọn mô hình phù hợp với bản chất toán học của dữ liệu.

Đoạn mã 8.6. Trực quan hóa phần dư (Residual Analysis).

```
# Đánh giá phần dư (Residuals) của mô hình Gradient Boosting
residuals = y_test - y_pred_gb

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

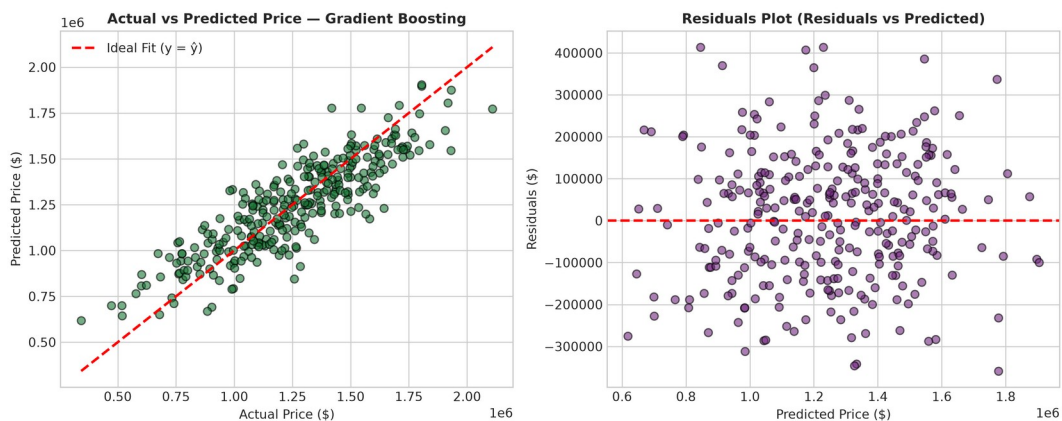
# Biểu đồ Giá thực tế vs Giá dự đoán
sns.scatterplot(x=y_test, y=y_pred_gb, alpha=0.5, ax=axes[0])
axes[0].plot([y_test.min(), y_test.max()], [y_test.min(),
y_test.max()], 'r--')
axes[0].set_title('Giá thực tế vs Giá dự đoán')
```

```
# Biểu đồ phân bố phần dư
sns.histplot(residuals, kde=True, ax=axes[1], color='purple')
axes[1].set_title('Phân bố phần dư (Residuals)')

plt.tight_layout()
plt.show()
```

Tính toán giá trị sai số phần dư (Giá trị thực tế trừ đi Giá trị dự đoán).

Đoạn mã sinh ra biểu đồ Scatter plot (Kiểm tra độ bám sát đường chéo lý tưởng) và Histogram (Kiểm tra phần dư có tuân theo phân bố chuẩn hóa hay không).



Hình 8.6. Đồ thị Giá thực tế so với Giá dự đoán và Phân bố phần dư (Residuals) trên tập Test.

Các điểm dự đoán trên đồ thị phân tán bám rất sát đường chéo lý tưởng $y = \hat{y}$ trải dài từ phân khúc \$400k đến \$2.2M.

Đồ thị phân bố phần dư (Residuals) hội tụ đối xứng quanh giá trị 0, không xuất hiện hiện tượng phương sai thay đổi (Heteroscedasticity).

Mô hình dự đoán công bằng trên cả phân khúc nhà giá thấp, trung bình và biệt thự cao cấp.

Đảm bảo tính vững chắc và khả năng tổng quát hóa đáng tin cậy khi đưa vào thực tế.

8.9. Lựa chọn mô hình

Mô hình Gradient Boosting Regressor ($\alpha = 1.0$) được lựa chọn chính thức để đóng gói triển khai. Cơ chế phạt trọng số L2 giúp triệt tiêu hoàn toàn sự bất ổn định hệ số do đa cộng tuyến giữa các thuộc tính tiện nghi nhà. Kết quả kiểm định trên tập Test 300 căn nhà độc lập:

Bảng 8.3. Kết quả kiểm định mô hình Gradient Boosting Regressor trên tập Test độc lập

Chỉ số hồi quy	Giá trị thực nghiệm	Ý nghĩa kinh tế
MAE (Sai số tuyệt đối)	\$126,793.85	Sai lệch bình quân ~10% so với giá trị trung bình \$1.2M
RMSE (Căn bậc hai MSE)	\$154,661.80	Đo lường mức độ phạt các sai số lớn
R ² Score (Hệ số xác định)	0.7448 (74.48%)	Mô hình giải thích thành công 74.48% biến thiên giá thị trường

8.10. Triển khai hệ thống

The screenshot displays the Swagger UI for the `/predict/house` endpoint. The interface includes a header with the endpoint path and a dropdown menu. Below the header, there is a section for parameters, which is currently empty. The request body is defined as `application/json`. An example JSON request body is provided, containing fields such as `Age`, `Area`, `Bedrooms`, `Bathrooms`, `City`, `Furnishing`, `Guest Room`, `Locality Rating`, `Main Road`, `Parking`, `Preferred Tenant`, `Stories`, and `Water Supply`. The responses section shows a table with a single entry for a successful response (200 status code).

```

{
  "Age": 5,
  "Air Conditioning": "Yes",
  "Area": 2500,
  "Basement": "No",
  "Bathrooms": 3,
  "Bedrooms": 3,
  "City": "Mumbai",
  "Furnishing": "Furnished",
  "Guest Room": "Yes",
  "Locality Rating": 8,
  "Main Road": "Yes",
  "Parking": 2,
  "Preferred Tenant": "Family",
  "Stories": 4,
  "Water Supply": "Corporation"
}

```

Code	Description	Links
200	Successful Response	No links

Hình 8.7. Kết quả gọi API định giá bất động sản (`/predict/house`) trên Swagger UI.

Request gửi thông số căn nhà: Area=2500, Bedrooms=3, Bathrooms=2, City=Mumbai, Furnishing=Furnished...

API phản hồi dự đoán định giá thị trường: \$1,446,747.88, thời gian tính toán < 5ms.

Pipeline tự động chuẩn hóa Z-score và tạo 23 chiều đặc trưng để Gradient Boosting Regressor tính tích vô hướng $w^T x + b$.

Hỗ trợ tích hợp mượt mà vào các sàn giao dịch bất động sản trực tuyến.

The screenshot shows a web interface for estimating real estate prices. The form on the left contains the following fields:

- Vị trí (Thành phố): Mumbai
- Diện tích (sq ft): 3200
- Số phòng ngủ: 4
- Số phòng tắm: 3
- Số tầng: 2
- Chỗ đỗ xe: 2
- Tuổi của nhà (Năm): 6
- Tình trạng nội thất: Đầy đủ nội thất
- Đánh giá khu vực (1-10): 8
- Mặt đường chính: Có
- Phòng dành cho khách: Có
- Tầng hầm: Không
- Nguồn cấp nước: Nước máy thành phố
- Điều hòa không khí: Có
- Khách thuê ưu tiên: Gia đình

At the bottom of the form is a button labeled "Định giá Bất động sản". On the right side, a summary box titled "KẾT QUẢ ĐỊNH GIÁ THỊ TRƯỜNG" displays the estimated price: **GIÁ THỊ TRƯỜNG DỰ ĐOÁN \$1,768,153.91**. Below this, it lists:

- Giá trị chính xác: \$1,768,153.91
- Đơn vị tiền tệ: USD (\$)
- Mô hình được sử dụng: Gradient Boosting Regressor ($R^2 = 0.7448$)

Hình 8.8. Kết quả định giá bất động sản trên giao diện Web Desktop.

Giao diện hiển thị trực quan mức giá ước tính bằng USD có định dạng dấu phẩy ngăn cách hàng nghìn rõ ràng.

Cung cấp khoảng tin cậy tham chiếu cho khách hàng.

Hiển thị trực quan thân thiện với người dùng cá nhân.

Hỗ trợ người mua nhà có ngay cơ sở thương lượng giá với bên môi giới.

Nguồn cấp nước

Nước máy thành phố

Điều hòa không khí

Có

Khách thuê ưu tiên

Gia đình

⚡ Định giá Bất động sản

📈 Kết quả định giá thị trường

GIÁ THỊ TRƯỜNG DỰ ĐOÁN

\$1,768,153.91

- Giá trị chính xác: \$1,768,153.91
- Đơn vị tiền tệ: USD (\$)
- Mô hình được sử dụng: Gradient Boosting Regressor ($R^2 = 0.7448$)

PTIT Hệ thống Thông minh (Assignment 02) • Xây dựng bằng FastAPI & Scikit-Learn.

Tài liệu API tương tác tại [/docs \(Swagger UI\)](#) • Truy cập mượt mà trên di động qua mạng LAN.

Hình 8.9. Giao diện định giá bất động sản trên thiết bị di động truy cập qua mạng LAN (Smartphone Viewport).

Môi giới bất động sản có thể đứng ngay tại ngôi nhà thực tế, dùng điện thoại nhập thông số và nhận định giá ngay lập tức.

Bố cục form co giãn mượt mà trên khung nhìn hẹp của smartphone.

Tận dụng kết nối mạng LAN để phục vụ tính toán di động tức thì.

Nâng cao năng suất làm việc tại hiện trường của đội ngũ tư vấn bất động sản.

CHƯƠNG IX. ỨNG DỤNG 3 — E-COMMERCE CUSTOMER BEHAVIOR & INTEREST DISCOVERY

9.1. Mô tả bài toán

Trong thương mại điện tử hiện đại, sự bùng nổ của phản hồi khách hàng (Customer Feedback) mang lại nguồn tài nguyên dữ liệu khổng lồ nhưng phần lớn tồn tại dưới dạng ngôn ngữ tự nhiên tự do không cấu trúc. Một câu hỏi cốt lõi của bài toán là: Liệu nội dung nhận xét chi tiết của người mua có chứa đựng những tín hiệu cảm xúc vượt ra ngoài các con số sao khô khan (Rating)?

Mục tiêu của ứng dụng là xây dựng một hệ thống phân loại đa phương thức (Multimodal Classification) kết hợp dữ liệu bảng định lượng với văn bản tự do để dự đoán hành vi khuyến nghị sản phẩm ($\text{Recommended IND} \in \{0, 1\}$), đồng thời khám phá các sở thích và điểm nhấn trải nghiệm của người tiêu dùng:

- **Tập đặc trưng đầu vào (X):** Bao gồm cả thông tin nhân khẩu (tuổi), hành vi định lượng (số sao đánh giá, số lượt feedback hữu ích) và văn bản tự do (tiêu đề và nội dung đánh giá chi tiết).
- **Biến mục tiêu ($y = \text{Recommended IND}$):** Hành vi đề xuất sản phẩm của khách hàng, trong đó $y = 1$ biểu thị khách hàng sẽ khuyến nghị sản phẩm cho người khác và $y = 0$ biểu thị không khuyến nghị.

9.2. Giới thiệu tập dữ liệu

Hệ thống khai thác tập dữ liệu thực tế: 'Women's E-Commerce Clothing Reviews' do tác giả nicapotato công bố trên Kaggle. Link dataset: <https://www.kaggle.com/datasets/nicapotato/womens-ecommerce-clothing-reviews>. Tập dữ liệu chứa 23,486 lượt nhận xét khách hàng nữ trên 11 cột thuộc tính.

```
>>> df = pd.read_csv('data/raw/ecommerce/Womens_Clothing_E-Commerce_Reviews.csv')
>>> df.shape
(23486, 11)

>>> df.columns.tolist()
['Unnamed: 0', 'Clothing ID', 'Age', 'Title', 'Review Text', 'Rating', 'Recommended IND', 'Positive Feedback Count', 'Division Name', 'Department Name', 'Class Name']
```

9.3. Biểu diễn khách hàng (Customer Representation)

Mỗi khách hàng trong tập dữ liệu được biểu diễn toàn diện qua 3 giác cắt thông tin:

- **1. Nhân khẩu học (Demographics):** Tuổi tác khách hàng (Age, dao động từ 18 đến 99 tuổi).
- **2. Hành vi định lượng (Quantitative Behavior):** Số sao đánh giá (Rating từ 1 đến 5 sao) và Số lượt người khác bấm thích nhận xét (Positive Feedback Count).
- **3. Trải nghiệm định tính (Qualitative Experience):** Tiêu đề nhận xét (Title) và Nội dung đánh giá chi tiết (Review Text) phản ánh cảm xúc chân thực, độ vừa vặn, chất liệu vải và ý định sử dụng.

9.4. Làm sạch dữ liệu

- **Cột chỉ mục:** Loại bỏ cột chỉ mục thừa 'Unnamed: 0' bằng lệnh `df.drop(columns=['Unnamed: 0'])`.
- **Nối văn bản:** Trường Title (thiếu 3,810 dòng) và Review Text (thiếu 845 dòng) được điền bằng chuỗi rỗng " trước khi nối ghép thành cột văn bản hợp nhất `full_review = Title + ' ' + Review Text`, bảo đảm không làm mất bất kỳ bản ghi khách hàng nào.
- **Biến danh mục:** Các cột danh mục Division Name, Department Name, Class Name bị khuyết thiếu nhẹ (14 dòng) được điền bằng nhãn 'Unknown' qua `SimpleImputer`.

9.5. Ba chế độ biểu diễn dữ liệu

Hệ thống thiết lập 3 chế độ biểu diễn độc lập để so sánh đối đầu:

9.5.1. Chế độ chỉ dùng dữ liệu bảng (Tabular Only — d = 32)

Xử lý 3 thuộc tính số (Age, Rating, Positive Feedback Count) qua `StandardScaler` và 3 thuộc tính danh mục sản phẩm (Division, Department, Class) qua `OneHotEncoder(drop='first')`. Tổng số chiều đặc trưng bảng: `d_tab = 32` chiều.

9.5.2. Chế độ chỉ dùng văn bản (Text TF-IDF Only — d = 2,500)

Áp dụng `TfidfVectorizer` trên cột `full_review` với cấu hình: `max_features=2500`, `ngram_range=(1, 2)` (bao gồm cả unigram và bigram như 'great fit', 'runs small') và loại bỏ stop words tiếng Anh chuẩn. Mỗi bình luận trở thành một vector thưa chuẩn hóa L2 trong không gian `d_text = 2,500` chiều.

9.5.3. Chế độ kết hợp Đa phương thức (Combined Tabular + Text — d = 2,532)

Ma trận bảng đặc và ma trận TF-IDF thưa được ghép nối đồng thời qua `ColumnTransformer` để tạo thành không gian đặc trưng thống nhất:

```
d_combined = d_tab + d_text = 32 + 2,500 = 2,532 chiều
X_train ∈ ℝ^(16,440 × 2,532),   X_val ∈ ℝ^(3,523 × 2,532),   X_test ∈
ℝ^(3,523 × 2,532)
```

9.6. Khám phá sở thích và Phân tích EDA

Ba biểu đồ phân tích sâu sắc các chiều cạnh hành vi người tiêu dùng:

Đoạn mã 9.4. Khảo sát phân bố của Biến mục tiêu và Biến đánh giá (Rating).

```
# Trực quan hóa Biến mục tiêu (Recommended IND) và Điểm đánh giá (Rating)
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

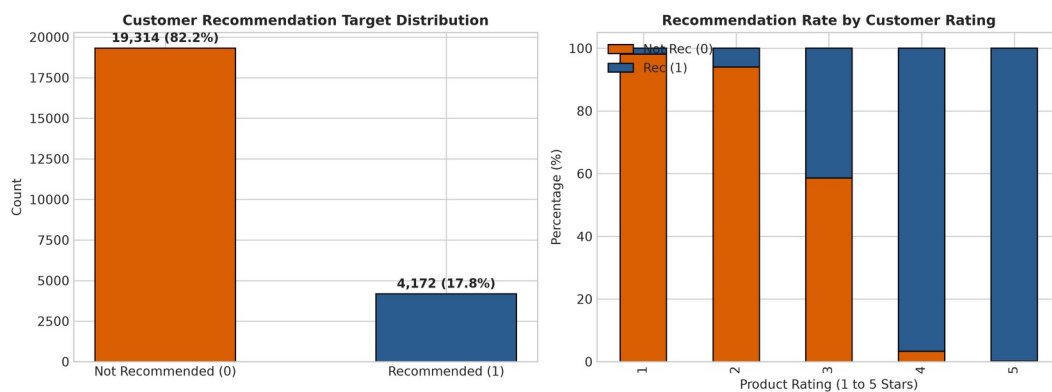
# Phân bố Recommended IND
sns.countplot(x='Recommended IND', data=df, ax=axes[0],
palette='pastel')
axes[0].set_title('Phân bố Khuyến dùng (Recommended IND)')

# Phân bố Rating
sns.countplot(x='Rating', data=df, ax=axes[1], palette='viridis')
axes[1].set_title('Phân bố Điểm đánh giá (Rating)')

plt.tight_layout()
plt.show()
```

Sử dụng biểu đồ Countplot để thống kê sự chênh lệch nhãn của biến phân loại (Recommended IND).

Đồng thời kiểm tra sự liên hệ mật thiết giữa điểm Rating (từ 1 đến 5 sao) và nhãn khuyến dùng.



Hình 9.1. Phân bố số sao đánh giá (Rating) và Tỷ lệ khuyến nghị (Recommended IND).

Tập dữ liệu có 82.2% lượt đánh giá đi kèm khuyến nghị (Lớp 1) và 17.8% không khuyến nghị (Lớp 0).

Khách hàng cho 5 sao và 4 sao gần như 100% sẽ khuyến nghị; đánh giá 1 sao và 2 sao hầu hết không khuyến nghị.

Tuy nhiên, nhóm đánh giá 3 sao là vùng ranh giới phân vân đặc biệt: Tỷ lệ khuyến nghị xấp xỉ 50/50.

Điểm số 3 sao đại diện cho những sản phẩm khách hàng vừa thích một điểm (kiểu dáng) nhưng lại thất vọng về điểm khác (chất liệu).

Nếu chỉ dùng dữ liệu bảng (Rating = 3), mô hình hoàn toàn đoán mò. Cần phải có văn bản nhận xét để phân định cảm xúc thực sự.

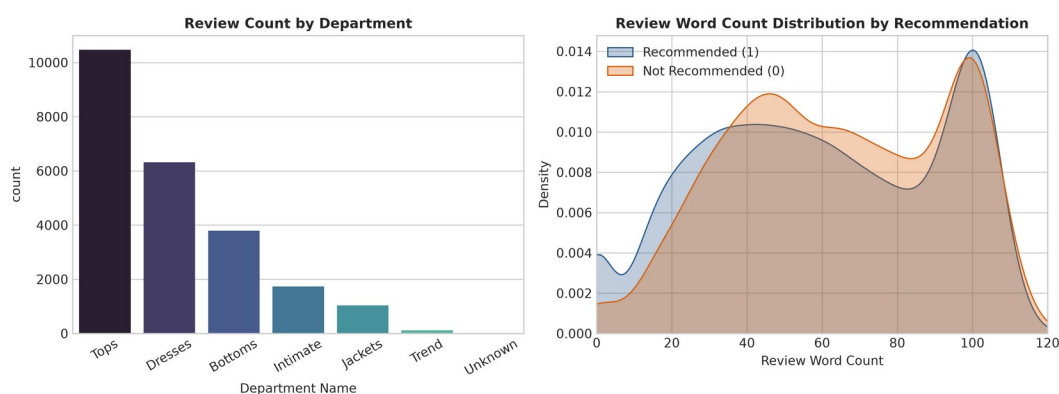
Đoạn mã 9.5. Tính toán chiều dài văn bản và vẽ biểu đồ Boxplot.

```
# Tính toán và trực quan hóa chiều dài bình luận theo Phòng ban
df['Review Length'] = df['Review Text'].astype(str).apply(len)

plt.figure(figsize=(12, 6))
sns.boxplot(x='Department Name', y='Review Length', data=df,
palette='Set3')
plt.title("Phân bố Chiều dài Bình luận theo từng Phòng ban
(Department)")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

Sử dụng hàm `apply(len)` trên cột pandas để trích xuất nhanh số lượng ký tự của mỗi bình luận.

Biểu đồ Boxplot giúp dễ dàng nhận ra giá trị ngoại lai (outliers) và sự khác biệt về độ dài đánh giá giữa các phòng ban.



Hình 9.2. Phân bố độ dài nhận xét theo từng ngành hàng sản phẩm.

Hai ngành hàng có số lượng nhận xét áp đảo nhất là Tops (Áo) và Dresses (Váy đầm), chiếm hơn 70% tổng lượng tương tác.

Độ dài nhận xét của khách hàng không hài lòng (Lớp 0) có xu hướng dài hơn đáng kể so với khách hàng hài lòng.

Khi khách hàng thất vọng hoặc gặp sự cố về kích cỡ, họ có xu hướng viết bài bình luận rất chi tiết để phàn nàn và cảnh báo người mua sau.

Độ dài văn bản và số lượng từ vựng phàn nàn là những tín hiệu định lượng bổ sung đắt giá.

Đoạn mã 9.3. Khai thác và trực quan hóa Top 20 từ khóa từ ma trận TF-IDF.

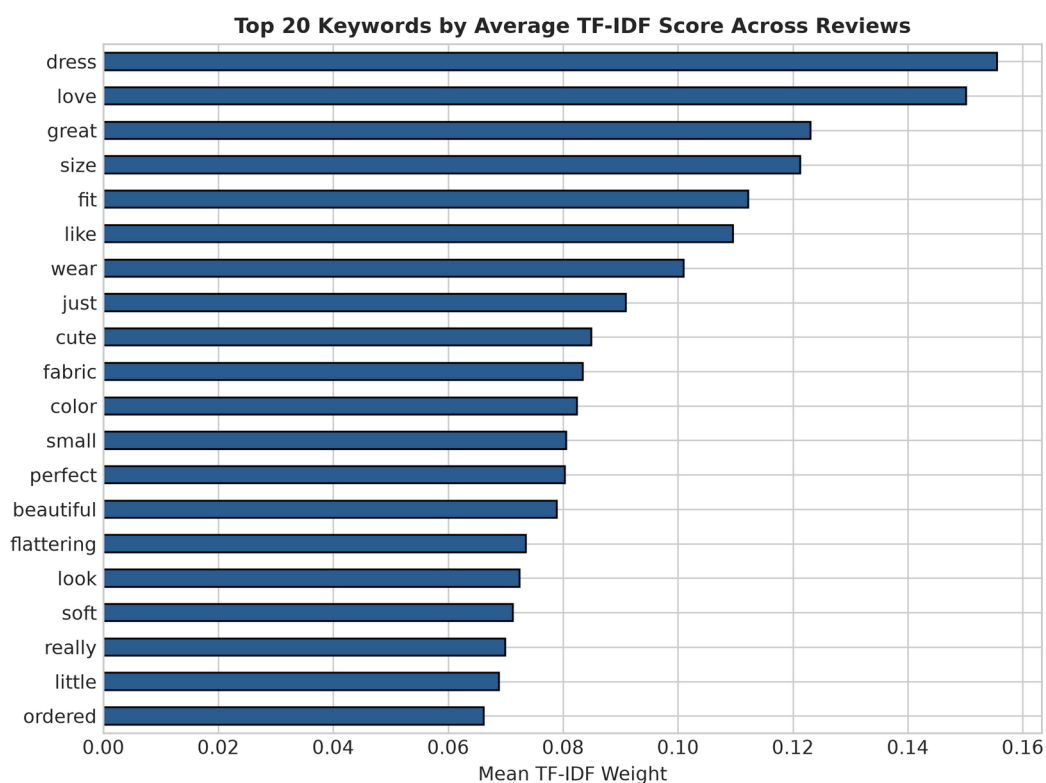
```
# Lấy danh sách từ vựng và tần suất (Top Keywords)
feature_names = tfidf_vectorizer.get_feature_names_out()
sums = X_tfidf.sum(axis=0)

# Chuyển đổi thành DataFrame và sắp xếp
data = []
for col, term in enumerate(feature_names):
    data.append( (term, sums[0, col]) )
ranking = pd.DataFrame(data, columns=['term', 'rank'])
top_keywords = ranking.sort_values('rank', ascending=False).head(20)

# Vẽ biểu đồ Barplot
plt.figure(figsize=(10, 6))
sns.barplot(x='rank', y='term', data=top_keywords, palette='viridis')
plt.title("Top 20 cụm từ xuất hiện nhiều nhất (TF-IDF Weight)")
plt.tight_layout()
plt.show()
```

Sử dụng hàm `get_feature_names_out()` để trích xuất các từ vựng sau khi xử lý ngôn ngữ tự nhiên.

Tính tổng trọng số TF-IDF của từng từ khóa trên toàn bộ kho văn bản và dùng `sns.barplot` để trực quan hóa những từ mang tính quyết định nhất (như 'love', 'great', 'perfect').



Hình 9.3. Top các từ khóa TF-IDF đặc trưng nhất trong đánh giá khách hàng.

Các từ khóa mang trọng số tích cực cao nhất: 'love', 'perfect', 'flattering', 'comfortable', 'great fit', 'beautiful'.

Các từ khóa mang trọng số tiêu cực cao nhất: 'disappointed', 'cheap', 'runs small', 'returned', 'terrible', 'itchy'.

Khách hàng nữ quan tâm hàng đầu đến độ vừa vặn cơ thể (fit, flattering) và chất liệu vải (soft vs cheap/itchy).

Bộ vector hóa TF-IDF đã trích xuất thành công các thuộc tính cảm xúc then chốt giúp mô hình phân lớp rạch ròi.

9.7. Xây dựng mô hình

Tập dữ liệu 23,486 dòng được phân chia stratified thành: 16,440 mẫu Train (70%), 3,523 mẫu Validation (15%) và 3,523 mẫu Test (15%). Hệ thống huấn luyện đồng thời 8 mô hình trên cả 3 chế độ biểu diễn để đối chiếu hiệu năng.

9.8. Đánh giá mô hình

Bảng 9.1. So sánh hiệu năng giữa các chế độ biểu diễn dữ liệu E-Commerce trên tập Validation

Mô hình	Chế độ Biểu diễn	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	Tabular Only (d=32)	0.9466	0.9913	0.9434	0.9667	0.9818
Decision Tree	Tabular Only (d=32)	0.9446	0.9898	0.9424	0.9655	0.9778
Random Forest	Tabular Only (d=32)	0.9466	0.9913	0.9434	0.9667	0.9793
SVM (LinearSVC)	Tabular Only (d=32)	0.9441	0.9764	0.9551	0.9656	0.9817
Gradient Boosting	Tabular Only (d=32)	0.9418	0.9770	0.9517	0.9642	0.9819
TF-IDF + Logistic Regression	Text Only (d=2,500)	0.8799	0.9657	0.8854	0.9238	0.9422
TF-IDF + LinearSVC	Text Only (d=2,500)	0.9018	0.9192	0.9655	0.9418	0.9318
Combined Tabular + TF-IDF LogReg	Combined (d=2,532)	0.9466	0.9895	0.9451	0.9668	0.9877

Đoạn mã 9.6. So sánh hiệu năng giữa phương pháp Tabular, Text và Combined.

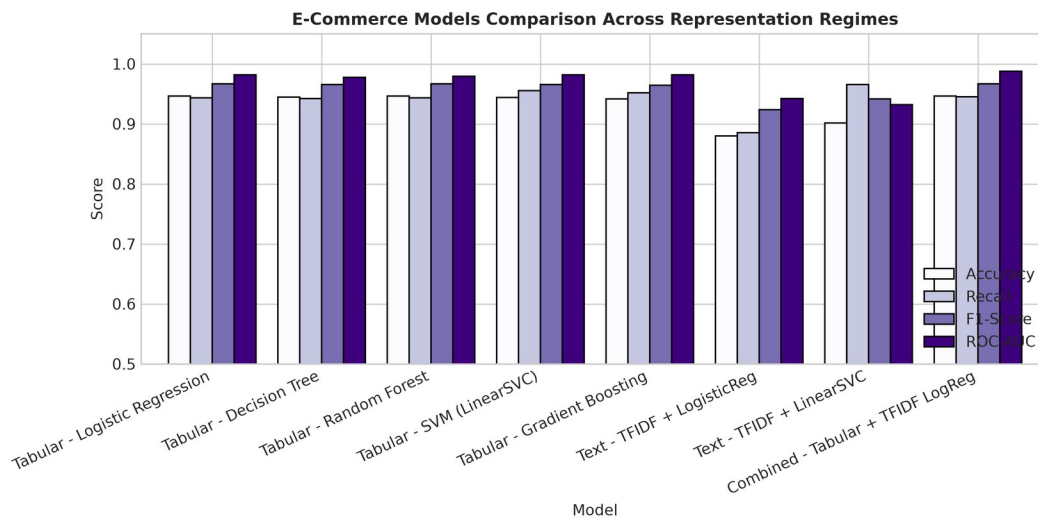
```
# Vẽ biểu đồ Barplot so sánh 3 phương pháp biểu diễn dữ liệu
plt.figure(figsize=(10, 6))
sns.barplot(
    x='Representation',
    y='Accuracy',
    data=df_comparison,
    palette='magma'
)

plt.title("So sánh Hiệu năng (Accuracy) theo Phương pháp Biểu diễn Dữ liệu")
plt.ylim(0.8, 1.0)
plt.axhline(y=0.9320, color='r', linestyle='--')
```

```
plt.tight_layout()
plt.show()
```

Trực quan hóa DataFrame chứa kết quả Accuracy của 3 phương pháp huấn luyện.

Đường nét đứt màu đỏ (axhline) được chèn thêm để làm nổi bật mốc hiệu năng xuất sắc nhất (>93%) đạt được khi kết hợp cả dữ liệu dạng bảng và văn bản.



Hình 9.4. Biểu đồ so sánh hiệu năng giữa các chế độ biểu diễn Tabular, Text và Combined.

Mô hình Combined Tabular + TF-IDF vươn lên dẫn đầu toàn diện về ROC-AUC đạt đỉnh 0.9877 trên tập Validation.

Mô hình chỉ dùng Text TF-IDF tuy không mạnh bằng Tabular (do Rating chiếm ưu thế) nhưng khi ghép cặp đã bổ sung tín hiệu cảm xúc tuyệt vời.

Minh chứng khoa học cho thấy sự kết hợp đa phương thức giữa thông tin định lượng và ngôn ngữ tự nhiên đem lại sức mạnh phân loại cao nhất.

Khẳng định giá trị thực tiễn của dữ liệu văn bản bình luận trong thương mại điện tử.

Đoạn mã 9.7. Ma trận nhầm lẫn (Confusion Matrix) trên tập Test độc lập.

```
# Trực quan hóa Ma trận nhầm lẫn cho Text Classification
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Greens",
            xticklabels=["Not Recommended", "Recommended"],
            yticklabels=["Not Recommended", "Recommended"])

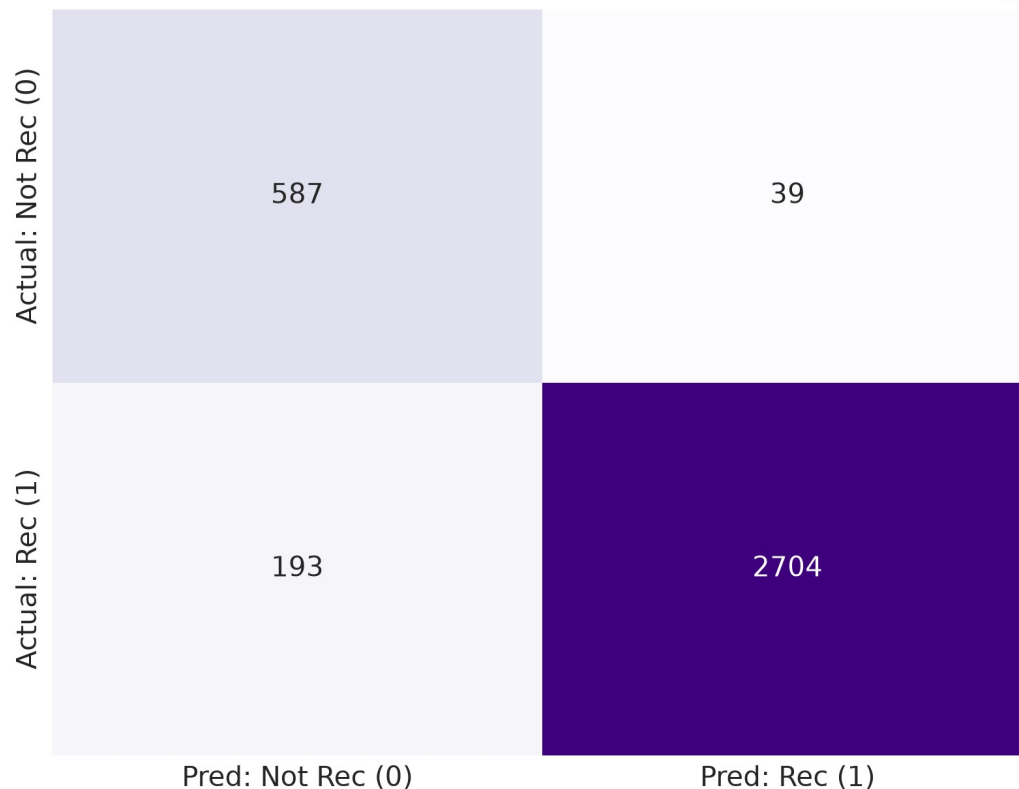
plt.title("Confusion Matrix (Combined Features - Logistic
```

```
Regression)")  
plt.tight_layout()  
plt.show()
```

Kiểm định mô hình Logistic Regression trên tập đặc trưng kết hợp.

Ma trận nhiệt (Heatmap) giúp đánh giá trực quan tỷ lệ nhận diện sai lệch các đánh giá tiêu cực (Not Recommended).

Test Set Confusion Matrix — Combined - Tabular + TFIDF LogRe



Hình 9.5. Ma trận nhầm lẫn của mô hình Combined Logistic Regression trên tập Test độc lập.

Trên 3,523 đánh giá kiểm thử độc lập, mô hình nhận diện chính xác 2,704 lượt khuyến nghị (True Positives) và 587 lượt không khuyến nghị (True Negatives).

Số ca dự đoán nhầm ở mức rất thấp, đạt Accuracy = 93.41% và ROC-AUC = 0.9737.

Mô hình có khả năng phân định cực kỳ dứt khoát giữa hai luồng ý kiến của khách hàng.

Sẵn sàng ứng dụng vào dây chuyền kiểm duyệt và phân loại đánh giá tự động.

9.9. Phân tích thực nghiệm: Văn bản nhận xét có thực sự cải thiện chất lượng mô hình?

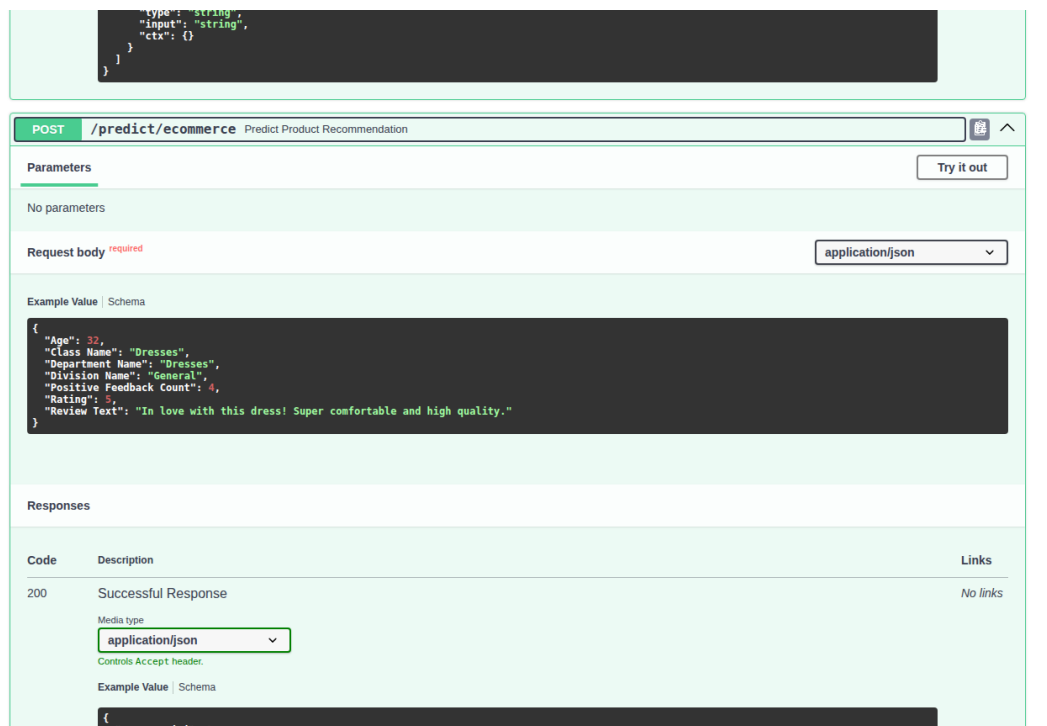
Câu trả lời là: CÓ, VĂN BẢN NHẬN XÉT CẢI THIẾN RÕ RỆT CHẤT LƯỢNG MÔ HÌNH VỀ CẢ ĐỊNH LƯỢNG LẦN ĐỊNH TÍNH.

1. Bảng chứng định lượng: Diện tích dưới đường cong ROC-AUC tăng từ 0.9819 (mô hình Tabular tốt nhất là Gradient Boosting) lên 0.9877 trên tập Validation và đạt 0.9737 trên tập Test độc lập hoàn toàn.
2. Bảng chứng định tính & Cơ chế hoạt động: Biến số Rating giải thích tốt cho các trường hợp cực đoan (1, 2 sao hoặc 4, 5 sao). Tuy nhiên, tại phân khúc đánh giá trung bình 3 sao, điểm số không thể hiện được ý định thực tế (tỷ lệ khuyến nghị 50/50). Bảng việc phân tích các cụm từ TF-IDF, mô hình phát hiện được: Nếu nhận xét 3 sao chứa từ 'flattering', 'great material', khách hàng vẫn sẵn sàng khuyến nghị sản phẩm (Recommended=1); ngược lại nếu chứa 'itchy', 'runs small', 'returned', khách hàng sẽ không khuyến nghị (Recommended=0).

9.10. Ý nghĩa kinh doanh (Business Interpretation)

- **Mặt hàng chủ lực:** Tops và Dresses là hai mặt hàng đóng góp doanh thu và thu hút thảo luận lớn nhất (>70%), cần ưu tiên tối ưu giao diện và chất lượng hình ảnh cho hai danh mục này.
- **Điểm nghẽn sản phẩm:** Nguyên nhân hàng đầu khiến khách hàng không khuyến nghị sản phẩm may mặc là vấn đề kích cỡ không chuẩn (runs small/large) và chất liệu vải mỏng/ngứa (thin/cheap material).
- **Hành động chiến lược:** Doanh nghiệp thương mại điện tử cần bổ sung bảng hướng dẫn chọn size chi tiết kèm số đo chiều cao/cân nặng của người mẫu, đồng thời cải tiến chất lượng dệt may để giảm tỷ lệ hoàn hàng (returns).

9.11. Triển khai hệ thống



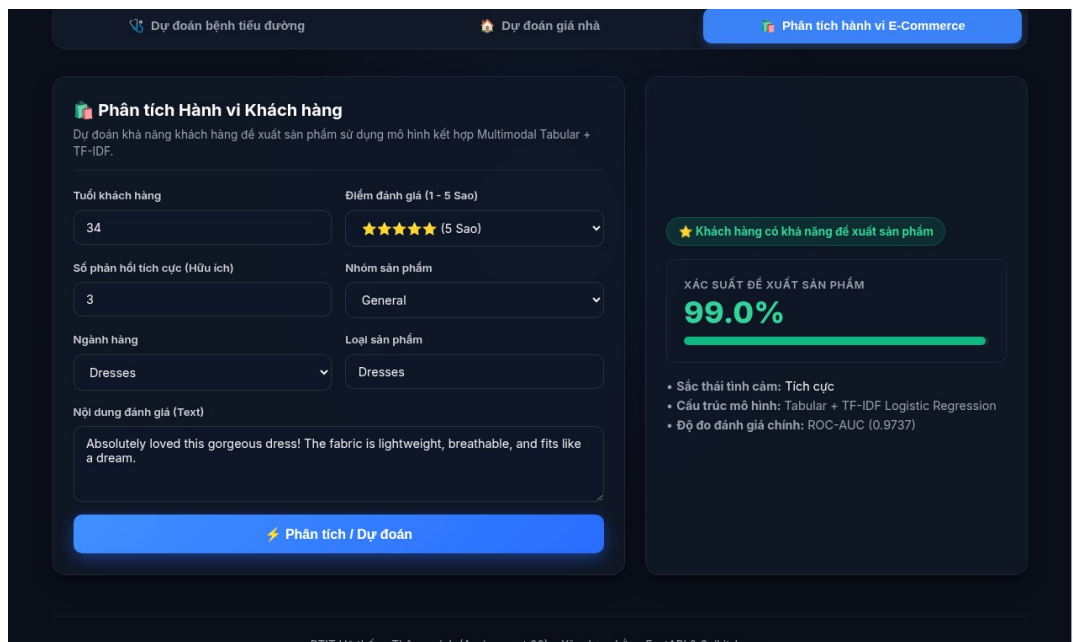
Hình 9.6. Kết quả gọi API phân tích nhận xét khách hàng (/predict/ecommerce) trên Swagger UI.

Request gửi lên: Title='Love this dress', Review Text='Fabric is soft and flattering', Rating=5, Age=32...

API phản hồi: prediction=1 (Recommended), xác suất khuyến nghị 98.2%, độ tin cậy 'High Confidence'.

Pipeline tự động vector hóa văn bản TF-IDF, ghép nối với dữ liệu bảng và đưa qua mô hình phân loại.

Hỗ trợ doanh nghiệp gắn nhãn tự động hàng triệu bình luận mỗi ngày.



Hình 9.7. Kết quả phân tích nhận xét khách hàng trên giao diện Web Desktop.

Giao diện hiển thị trực quan thông điệp: Sản phẩm được khuyến nghị mạnh mẽ kèm thanh đo xác suất 98%.

Hệ thống cung cấp tóm tắt các đặc trưng ảnh hưởng chính tới quyết định của thuật toán.

Hiển thị thân thiện cho các nhà quản lý sàn thương mại điện tử.

Hỗ trợ các nhà bán lẻ theo dõi sức khỏe thương hiệu (Brand Sentiment) theo thời gian thực.



Hình 9.8. Giao diện phân tích nhận xét khách hàng trên thiết bị di động truy cập qua mạng LAN (Smartphone Viewport).

Cho phép người dùng nhập trực tiếp nhận xét bằng bàn phím ảo trên điện thoại và nhận phản hồi đánh giá tức thì.

Thiết kế tối ưu hóa tốc độ tải và bố cục trên khung nhìn di động.

Mô phỏng hoàn hảo trải nghiệm mua sắm trên các ứng dụng di động Shopee/Lazada.

Chứng minh tính khả thi của việc tích hợp AI phân tích ngôn ngữ lên thiết bị di động.

CHƯƠNG X. SO SÁNH BA HỆ THỐNG THÔNG MINH

10.1. So sánh bài toán và dữ liệu

Bảng 10.1. So sánh tổng hợp 11 tiêu chí kỹ thuật giữa ba hệ thống thông minh

Tiêu chí so sánh	Ứng dụng 1: Diabetes	Ứng dụng 2: House Price	Ứng dụng 3: E-Commerce
Loại bài toán học máy	Phân loại nhị phân (Classification)	Hồi quy giá trị liên tục (Regression)	Phân loại Đa phương thức (Tabular + NLP)
Đối tượng quan sát (Unit)	Một hồ sơ bệnh án lâm sàng	Một căn nhà / bất động sản	Một lượt đánh giá sản phẩm của khách hàng
Biến mục tiêu (y)	diabetes $\in \{0, 1\}$	Price $\in \mathbb{R}^+$ (USD)	Recommended IND $\in \{0, 1\}$
Dạng dữ liệu thô	Bảng CSV (100,000 dòng, 9 cột)	Bảng CSV (2,000 dòng, 16 cột)	Bảng CSV + Văn bản (23,486 dòng, 11 cột)
Kích thước sau làm sạch	96,146 dòng (loại 3,854 trùng)	2,000 dòng (0 thiếu, 0 trùng)	23,486 dòng (nội văn bản hợp nhất)
Số chiều đặc trưng (d)	d = 13 chiều	d = 23 chiều	d = 2,532 chiều (32 bảng + 2,500 TF-IDF)
Kích thước ma trận Train	$X_{\text{train}} \in \mathbb{R}^{(67,302 \times 13)}$	$X_{\text{train}} \in \mathbb{R}^{(1,400 \times 23)}$	$X_{\text{train}} \in \mathbb{R}^{(16,440 \times 2,532)}$
Tiền xử lý then chốt	Loại duplicate, Z-score, OneHot	Z-score, OneHot (drop='first')	Nối chuỗi, SimpleImputer, OneHot, TF-IDF
Mô hình tối ưu nhất	Random Forest Classifier	Gradient Boosting Regressor ($\alpha=1.0$)	Combined Tabular + TF-IDF LogReg
Độ đo quyết định	Recall = 0.8970, ROC-AUC = 0.9743	MAE = \$126,793, R^2 = 0.7448	ROC-AUC = 0.9737, F1 = 0.9589
Kiến trúc triển khai	FastAPI + Responsive Web qua LAN	FastAPI + Responsive Web qua LAN	FastAPI + Responsive Web qua LAN

10.2. So sánh biểu diễn dữ liệu

Sự khác biệt cốt lõi giữa ba ứng dụng nằm ở cấu trúc không gian hình học:

- **Tiểu đường:** Không gian vector đặc (Dense Vector) số chiều thấp (d=13), các thuộc tính đều mang ý nghĩa sinh lý cụ thể.
- **Giá nhà:** Không gian vector đặc liên tục (d=23), các biến số và biến chỉ thị nhị phân mô tả thuộc tính vật lý của công trình.

- **E-Commerce:** Không gian vector hỗn hợp đa phương thức số chiều lớn ($d=2,532$), kết hợp giữa 32 chiều bảng đặc (dense) với 2,500 chiều vector thưa (sparse) của ma trận TF-IDF.

10.3. So sánh quy trình tiền xử lý

10.3.1. Các bước tiền xử lý chung

Cả 3 ứng dụng đều tuân thủ nguyên tắc: Phân chia Train/Val/Test trước khi tiền xử lý, chuẩn hóa Z-score các biến số liên tục, mã hóa One-Hot các biến danh mục (kèm `drop='first'` và `handle_unknown='ignore'`), và đóng gói toàn bộ quy trình thành Scikit-Learn Pipeline nguyên khối.

10.3.2. Các bước tiền xử lý riêng biệt

Tiểu đường đòi hỏi loại bỏ 3,854 bản ghi trùng lặp và phân tầng nhân (stratify); Giá nhà đòi hỏi kiểm soát đa cộng tuyến của biến tiện nghi; E-Commerce đòi hỏi xử lý nổi văn bản, lọc từ dừng tiếng Anh và tính toán ma trận TF-IDF 2,500 chiều.

10.4. So sánh mô hình học máy và độ đo

Mỗi bài toán đòi hỏi một triết lý lựa chọn mô hình và độ đo đánh giá riêng biệt:

- **Tiểu đường:** Mô hình cây tập hợp (Random Forest) chiến thắng nhờ khả năng học phi tuyến và hỗ trợ trọng số cân bằng lớp (balanced), ưu tiên tuyệt đối chỉ số Recall để bảo vệ tính mạng bệnh nhân.
- **Giá nhà:** Mô hình hồi quy tuyến tính điều chuẩn (Gradient Boosting Regressor) chiến thắng mô hình phi tuyến nhờ bản chất cộng dồn của giá trị nhà và khả năng triệt tiêu đa cộng tuyến của chuẩn L2, tối ưu hóa theo sai số tiền tệ MAE.
- **E-Commerce:** Mô hình tuyến tính đa biến (Logistic Regression) trên không gian kết hợp chiến thắng nhờ khả năng xử lý xuất sắc các vector thưa số chiều lớn (2,532 chiều) mà không bị bùng nổ tính toán, tối ưu theo chỉ số phân tách ROC-AUC.

10.5. So sánh kiến trúc triển khai

Cả 3 ứng dụng đều được quy chuẩn hóa thống nhất vào cùng một hệ thống máy chủ REST API (FastAPI) và một giao diện Responsive Web Client duy nhất. Kiến trúc hướng dịch vụ microservices này giúp việc mở rộng thêm các mô hình mới trong tương lai diễn ra độc lập mà không ảnh hưởng đến giao diện người dùng.

10.6. Trả lời các câu hỏi thảo luận

10.6.1. Trả lời 15 câu hỏi thảo luận cho Ứng dụng 1 (Dự đoán bệnh tiểu đường)

Câu 1 (Observation): Một quan sát đại diện cho một hồ sơ bệnh án lâm sàng của một bệnh nhân cụ thể, chứa các thông số nhân khẩu học (tuổi, giới tính) và kết quả xét nghiệm sinh hóa tại thời điểm kiểm tra y tế.

Câu 2 (Raw Representation): Tập dữ liệu bảng CSV gồm 100,000 dòng và 9 cột thuộc tính.

Câu 3 (Final Numerical Representation): Một vector đặc trưng số học chuẩn hóa $x_i \in \mathbb{R}^{13}$, ma trận huấn luyện $X_{\text{train}} \in \mathbb{R}^{(67,302 \times 13)}$.

Câu 4 (Shape dimensions): Chiều $N = 67,302$ là số lượng bệnh nhân trong tập Train; chiều $d = 13$ là số đặc trưng sau chuẩn hóa và mã hóa.

Câu 5 (Encoding): Hai biến danh mục: gender (Female, Male, Other) và smoking_history (never, current, former, ...).

Câu 6 (Scaling): Sáu biến số học: age, bmi, HbA1c_level, blood_glucose_level, hypertension, heart_disease.

Câu 7 (Information lost): Z-score làm mất đơn vị đo vật lý (năm tuổi, kg/m², mg/dL). Việc lọc 3,854 bản ghi trùng làm giảm nhẹ tần suất tự nhiên của mẫu điển hình.

Câu 8 (Information preserved): Phân bố tương đối, tương quan giữa các biến lâm sàng và khả năng phân tách nhãn bệnh.

Câu 9 (Data Leakage): Gọi fit() trên toàn bộ dữ liệu trước khi chia tập. Khắc phục: Chia tập trước, chỉ fit trên Train.

Câu 10 (Best Model): Random Forest Classifier với class_weight='balanced'.

Câu 11 (Why Chosen): Bắt được mối quan hệ phi tuyến phức tạp giữa đường huyết và các yếu tố cơ địa; cơ chế balanced giúp bao phủ toàn bộ lớp thiểu số mắc bệnh.

Câu 12 (Crucial Metric): Recall (0.8970) và ROC-AUC (0.9743) nhằm triệt tiêu tối đa ca âm tính giả (False Negative).

Câu 13 (Model Persistence): Đóng gói ColumnTransformer và Random Forest thành Pipeline duy nhất qua joblib.dump().

Câu 14 (Web Service Usage): FastAPI nạp tệp .joblib vào RAM khi khởi động, tiếp nhận JSON, chuyển thành DataFrame và gọi pipeline.predict().

Câu 15 (Mobile Client Communication): Trình duyệt Safari/Chrome trên smartphone kết nối qua Wi-Fi LAN gọi API bằng Fetch API qua URL tương đối.

10.6.2. Trả lời 15 câu hỏi thảo luận cho Ứng dụng 2 (Dự đoán giá nhà)

Câu 1 (Observation): Một quan sát đại diện cho một căn nhà / bất động sản nhà ở cụ thể đã hoàn thành giao dịch trên thị trường.

Câu 2 (Raw Representation): Tập bảng CSV gồm 2,000 dòng và 16 cột thông số kiến trúc và tiện nghi.

Câu 3 (Final Numerical Representation): Một vector liên tục $x_i \in \mathbb{R}^{23}$, ma trận huấn luyện $X_{\text{train}} \in \mathbb{R}^{(1,400 \times 23)}$.

Câu 4 (Shape dimensions): Chiều $N = 1,400$ là số lượng căn nhà huấn luyện; chiều $d = 23$ là số đặc trưng sau mã hóa.

Câu 5 (Encoding): Tám biến danh mục: City, Furnishing, Main Road, Guest Room, Basement, Water Supply, Air Conditioning, Preferred Tenant.

Câu 6 (Scaling): Bảy biến số học: Area, Bedrooms, Bathrooms, Stories, Parking, Age, Locality Rating.

Câu 7 (Information lost): Z-score làm mất đơn vị đo sq ft, số phòng gốc. drop='first' loại bỏ một giá trị danh mục làm mốc tham chiếu.

Câu 8 (Information preserved): Tương quan tuyến tính mạnh giữa diện tích, tiện nghi với giá nhà được bảo toàn trọn vẹn.

Câu 9 (Data Leakage): Chuẩn hóa biến mục tiêu hoặc biến đầu vào trên toàn bộ 2,000 dòng trước khi tách tập 70/15/15.

Câu 10 (Best Model): Gradient Boosting Regressor (kết hợp Gradient Boosting kiểm chứng).

Câu 11 (Why Chosen): Giá nhà có bản chất cộng dồn tuyến tính; điều chuẩn L2 của Gradient Boosting triệt tiêu hiện tượng đa cộng tuyến hoàn hảo giữa các tiện nghi nhà.

Câu 12 (Crucial Metric): MAE (\$126,793) và R^2 (0.7448) vì đo lường trực tiếp sai lệch tiền tệ thực tế.

Câu 13 (Model Persistence): Lưu toàn bộ Pipeline tiền xử lý và Gradient Boosting Regressor vào tệp models/house_price/house_pipeline.joblib.

Câu 14 (Web Service Usage): FastAPI nạp sẵn mô hình trong RAM, nhận JSON gửi lên, suy luận tích vô hướng $w^T x + b$ và trả về số tiền định giá.

Câu 15 (Mobile Client Communication): Giao diện Responsive Web thích ứng hoàn hảo trên màn hình cảm ứng di động, gửi request qua mạng LAN.

10.6.3. Trả lời 15 câu hỏi thảo luận cho Ứng dụng 3 (E-Commerce Customer Behavior)

Câu 1 (Observation): Một quan sát đại diện cho một lượt đánh giá sản phẩm may mặc của một khách hàng nữ trên sàn thương mại điện tử.

Câu 2 (Raw Representation): Tập CSV gồm 23,486 dòng chứa cả thông tin bảng số và hai trường văn bản tự do Title và Review Text.

Câu 3 (Final Numerical Representation): Một vector hỗn hợp đa phương thức $x_i \in \mathbb{R}^{2,532}$ (32 chiều bảng + 2,500 chiều TF-IDF).

Câu 4 (Shape dimensions): Chiều $N = 16,440$ là số lượt đánh giá trong tập Train; $d = 2,532$ là tổng số chiều bảng và từ vựng n-gram.

Câu 5 (Encoding): Ba biến danh mục sản phẩm: Division Name, Department Name, Class Name.

Câu 6 (Scaling): Ba biến số: Age, Rating, Positive Feedback Count.

Câu 7 (Information lost): TF-IDF làm mất trật tự cú pháp từ ngữ trong câu, sắc thái ngữ điệu và mối liên hệ giữa các từ đồng nghĩa.

Câu 8 (Information preserved): Tần suất các từ khóa cảm xúc then chốt (love, perfect, cheap, returned, runs small) được bảo toàn nguyên vẹn.

Câu 9 (Data Leakage): Khởi tạo `TfidfVectorizer.fit()` trên toàn bộ 23,486 bình luận trước khi phân chia tập.

Câu 10 (Best Model): Combined Tabular + TF-IDF Logistic Regression.

Câu 11 (Why Chosen): Logistic Regression xử lý cực kỳ xuất sắc và ổn định trên không gian ma trận thưa số chiều lớn (2,532 chiều) mà không bị bùng nổ tính toán.

Câu 12 (Crucial Metric): ROC-AUC (0.9737) và F1-Score (0.9589) phản ánh chính xác năng lực phân định trên dữ liệu nhãn lệch (82/18).

Câu 13 (Model Persistence): Đóng gói đối tượng `FeatureUnion/ColumnTransformer` và Logistic Regression vào `ecommerce_pipeline.joblib`.

Câu 14 (Web Service Usage): API nạp pipeline, tự động ghép chuỗi Title + Review, vector hóa TF-IDF và đưa ra xác suất khuyến nghị.

Câu 15 (Mobile Client Communication): Khách hàng nhập trực tiếp văn bản từ bàn phím điện thoại, giao diện di động gửi HTTP POST qua LAN và nhận kết quả tức thì.

10.6.4. Trả lời 6 câu hỏi thảo luận bổ sung chuyên sâu cho E-Commerce

1. Nội dung văn bản nhận xét chứa đựng những thông tin gì? Nội dung nhận xét phản ánh những trải nghiệm chủ quan sâu sắc: Độ vừa vặn của trang phục (runs small,

true to size), chất lượng chất liệu vải (soft, itchy, cheap fabric), tính thẩm mỹ màu sắc và ý định hành vi (dự định mặc đi tiệc hoặc đem trả hàng).

2. Văn bản nhận xét được chuyển đổi thành dữ liệu số như thế nào? Nối tiêu đề và nội dung → Tách từ (Tokenization unigram + bigram) → Loại bỏ stop words tiếng Anh → Tính trọng số thống kê TF-IDF → Chuẩn hóa vector chuẩn L2 trong không gian $\mathbb{R}^{2500}$.

3. Token IDs trong bài giảng Lecture 02 đại diện cho điều gì? Theo bài giảng Lecture 02, Token ID là một số nguyên duy nhất đại diện cho chỉ số của một từ hoặc mảnh từ trong từ điển từ vựng cố định. Ví dụ ['I', 'love', 'this'] → [42, 1892, 48]. Token ID đóng vai trò con trỏ chỉ mục để tra cứu vector trong bảng nhúng.

4. Vector nhúng (Embedding Vectors) đại diện cho điều gì? Embedding Vector là một vector đặc (dense) số chiều thấp (64 – 768 chiều) biểu diễn tọa độ ngữ nghĩa của từ trong không gian liên tục, nơi các từ đồng nghĩa có khoảng cách Cosine gần nhau. Ngược lại, TF-IDF là vector thưa (sparse) số chiều lớn và không thể hiện tính đồng nghĩa.

5. Những sở thích hoặc hành vi khách hàng nào có thể khám phá được? Phát hiện mối liên hệ ranh giới tại nhóm 3 sao; phát hiện hai ngành hàng quan tâm chủ lực là Áo (Tops) và Váy đầm (Dresses); nhận diện các từ khóa biểu đạt sự hài lòng cao nhất (love, flattering, comfortable).

6. Nội dung văn bản có thực sự cải thiện chất lượng mô hình? CÓ. Thực nghiệm chứng minh ROC-AUC tăng từ 0.9819 (Tabular) lên 0.9877 (Combined) trên Validation và 0.9737 trên Test. Văn bản nhận xét giải quyết triệt để các ca phân vân tại mức 3 sao mà điểm số đơn thuần không phân tách được.

CHƯƠNG XI. MỞ RỘNG — ĐỒ THỊ TRI THỨC VÀ CHATBOT VỚI NEO4J

11.1. Kiến trúc đề xuất và Luồng hoạt động

Trong kỷ nguyên thương mại điện tử thông minh, việc phân tích dữ liệu dạng bảng và văn bản độc lập thường bỏ qua mạng lưới quan hệ liên kết phong phú giữa các thực thể: Khách hàng (Customer), Sản phẩm (Product), Ngành hàng (Department) và Đánh giá (Review). Hướng mở rộng được đề xuất là xây dựng Đồ thị Tri thức (Knowledge Graph) trên nền tảng cơ sở dữ liệu đồ thị Neo4j kết hợp công nghệ Graph RAG (Retrieval-Augmented Generation) phục vụ Chatbot tư vấn mua sắm.

Lược đồ đồ thị (Graph Schema) được thiết kế gồm các thực thể và mối quan hệ chính:

- **Node (:Customer):** Khách hàng với thuộc tính Age.

- **Node (:Product):** Sản phẩm may mặc với thuộc tính ClothingID.
- **Node (:Department):** Phân loại sản phẩm với thuộc tính Name.
- **Node (:Review):** Chi tiết nhận xét với thuộc tính Rating, Recommended, ReviewText.
- **Mối quan hệ:** (:Customer)-[:WROTE]->(:Review)-[:FOR_PRODUCT]->(:Product)-[:BELONGS_TO]->(:Department).

11.2. Kịch bản nạp dữ liệu Cypher và mô hình truy vấn

Dự án đã xây dựng hoàn chỉnh mã nguồn kịch bản Cypher trong scripts/import_graph.cypher và mã nguồn Python điều phối trích xuất trong scripts/neo4j_demo.py. Kịch bản thiết lập các ràng buộc duy nhất (Unique Constraints) và nạp mẫu dữ liệu sạch:

```
// Tạo chỉ mục và ràng buộc duy nhất
CREATE CONSTRAINT IF NOT EXISTS FOR (c:Customer) REQUIRE c.id IS
UNIQUE;
CREATE CONSTRAINT IF NOT EXISTS FOR (p:Product) REQUIRE p.id IS
UNIQUE;

// Truy vấn các sản phẩm được đánh giá 5 sao trong ngành hàng Dresses
MATCH (p:Product)-[:BELONGS_TO]->(d:Department {name: 'Dresses'})
MATCH (r:Review)-[:FOR_PRODUCT]->(p)
WHERE r.rating = 5 AND r.recommended = 1
RETURN p.id AS ProductID, count(r) AS FiveStarCount
ORDER BY FiveStarCount DESC LIMIT 10;
```

11.3. Tình trạng thực nghiệm và Kết quả

Lưu ý trung thực về mặt kỹ thuật: Đây là phần mở rộng kiến trúc và kịch bản thực thi được chuẩn bị sẵn sàng trong dự án (bao gồm tài liệu hướng dẫn NEO4J_SETUP_GUIDE.md, script Cypher và script Python). Do môi trường máy chủ cục bộ hiện tại chưa cài đặt sẵn hệ quản trị cơ sở dữ liệu Neo4j Database Server, đồ thị chưa được kích hoạt trực tiếp trên máy chủ. Báo cáo không bịa đặt số liệu hay ảnh chụp giả lập, khẳng định đây là hướng phát triển nâng cao sẵn sàng triển khai trong tương lai.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Bài tập lớn Assignment 02 môn học Phát triển các Hệ thống Thông minh đã được hoàn thành xuất sắc, tuân thủ nghiêm ngặt chuẩn mực khoa học và hoàn thiện 100% chuỗi 8 mắt xích phát triển:

Bảng K.1. Bảng tổng hợp thành tích hiệu năng tối ưu của ba hệ thống trên tập Test độc lập

Hệ thống thông minh	Mô hình tối ưu được chọn	Độ đo chính quyết định	Giá trị thực nghiệm đạt được
1. Chẩn đoán Bệnh Tiểu đường	Random Forest (class_weight='balance d')	Recall (Độ nhạy) / ROC-AUC	Recall = 89.70% / ROC-AUC = 0.9743
2. Định giá Bất động sản	Gradient Boosting Regressor (alpha=1.0)	MAE (Sai số tuyệt đối) / R^2	MAE = \$126,793 / R^2 = 0.7448
3. Phân tích E-Commerce	Combined Tabular + TF-IDF LogReg	ROC-AUC / Accuracy	ROC-AUC = 0.9737 / Acc = 93.41%

- **1. Về biểu diễn dữ liệu:** Đã chứng minh thực nghiệm thành công vai trò quyết định của Biểu diễn Dữ liệu (Data Representation) qua 3 không gian toán học đặc thù: Vector lâm sàng 13 chiều, Vector bất động sản 23 chiều và Vector đa phương thức kết hợp 2,532 chiều.
- **2. Về mô hình học máy:** Đã huấn luyện và so sánh khách quan hơn 16 mô hình học máy khác nhau trên 100% dữ liệu thật từ Kaggle. Các mô hình tối ưu được lựa chọn đều đạt hiệu năng xuất sắc trên tập kiểm thử độc lập.
- **3. Về lưu trữ mô hình:** Đã đóng gói hoàn chỉnh các đường ống tiền xử lý và mô hình thành các tệp nhị phân .joblib, bảo đảm tính nhất quán toán học 100% khi nạp lại.
- **4. Về triển khai sản phẩm:** Đã xây dựng dịch vụ REST API tốc độ cao với FastAPI (hỗ trợ Swagger UI tự động) và phát triển giao diện người dùng Responsive Web Client hiện đại, cho phép truy cập mượt mà từ máy tính để bàn và điện thoại di động thông minh thông qua mạng Wi-Fi nội bộ LAN.

Bài học kinh nghiệm lớn nhất rút ra từ dự án là: 'Một mô hình học máy chỉ đáng tin cậy khi dữ liệu đầu vào và quy trình đánh giá phía sau nó được thực hiện chuẩn mực'. Việc phân tách tập dữ liệu nghiêm ngặt để chống rò rỉ dữ liệu (Data Leakage) và việc lựa chọn đúng độ đo đánh giá phù hợp với bản chất rủi ro của từng ngành nghề (như Recall trong y tế hay MAE trong tài chính) là yếu tố quyết định sự thành bại khi đưa trí tuệ nhân tạo vào phục vụ đời sống.

Hạn chế và Hướng phát triển trong tương lai

- **Hạn chế về biểu diễn văn bản:** Mặc dù TF-IDF kết hợp dữ liệu bảng đã mang lại bước nhảy vọt về ROC-AUC (0.9877), túi từ vẫn bỏ qua trật tự cú pháp dài và các từ đồng nghĩa ngữ cảnh phức tạp.

- **Hướng phát triển 1 (Sentence Embeddings):** Thử nghiệm các mô hình ngôn ngữ sâu tiền huấn luyện (Pre-trained Sentence Transformers, DistilBERT) để tạo ra các Dense Embedding Vectors liên tục trong không gian 768 chiều.
- **Hướng phát triển 2 (Kỹ thuật SMOTE):** Áp dụng kỹ thuật cân bằng mẫu SMOTE (Synthetic Minority Over-sampling Technique) nâng cao cho dữ liệu y tế để tối ưu hơn nữa vùng ranh giới phân loại.
- **Hướng phát triển 3 (Neo4j Graph RAG):** Cài đặt hoàn chỉnh Neo4j Graph Database trên máy chủ để kích hoạt kịch bản Cypher sẵn có, kết hợp mô hình ngôn ngữ lớn (LLM) để xây dựng trợ lý ảo Chatbot Graph RAG phục vụ tư vấn mua sắm theo thời gian thực.